COURSE

Context Engineering

CampusX

PROFESSIONAL LEARNING NOTES

Context Engineering Complete Course Notes

A Practical Guide to Building Reliable LLM Context Systems

COMPLETE COURSE

VERSION

2026-09-v01

FIRST EDITION
2026

CX-CONTEXT-ENGINEERING-2026-09-v01

About These Notes

Complete Course Notes

A Practical Guide to Building Reliable LLM Context Systems

Publisher	CampusX • First edition: 2026
Website	learnwith.campusx.in
Support email	support@campusx.in

These notes present the key concepts from this CampusX learning module in a concise, structured and revision-friendly format. They are intended to support focused study, quick revision and future reference.

A request from CampusX. Please treat these notes as your own learning asset and do not copy, pirate or redistribute them. Your support helps us continue creating high-quality educational content.

Study tip. For the best learning experience, we recommend printing a hard copy of these notes for focused study, annotation and revision.

Copyright © 2026 CampusX. All rights reserved. No part of this publication may be reproduced, distributed or transmitted in any form without the prior written permission of the publisher. The case studies and recommendations herein are drawn from publicly reported events and are provided for educational purposes only.

Contents

Introduction to Context Engineering **5**

1 Context and Context Engineering 5

2 Why Context Engineering 14

Structuring Prompt Sections and KV Caching **22**

3 Writing Effective System Instructions 22

4 Structuring the User Prompt Section 30

5 KV Cache Part 1 37

6 KV Cache Part 2 43

Retrieval-Augmented Generation (RAG) **50**

7 RAG Intro 50

8 RAG - Document Loaders 58

9 RAG - Contextual Retrieval and Dynamic Splitting 60

10 RAG - Complete Flow 65

Effective Tool Calling for Proper Context Engineering **71**

11 Tool, Tool Calling and Tool as Context Problem 71

12 Tool Places in Context and Writing Effective Tools 77

13 Deferred Loading 86

14 Tool Results 97

15 Programmatic Tool Calling 113

16 Deciding Which Technique to Use 123

Model Context Protocol (MCP) **128**

17 What Is MCP and the Context Problem It Can Cause 128

18 How MCP Can Become a Context Problem 135

19 Solving the Context Problem in MCP 140

Memory Management **148**

20 Intro to Memory in LLM Apps 148

21 LLMs Don't Have Memory - So How Do They Remember? 151

22 Making Memories 159

23 Reading Memory 166

24 Creating Memories with Libraries and Problems with Libraries 173

25 Creating Memories Using Libraries and Rectifying Their Errors 178

Context Compression **187**

26 Context Compression and Lossless Compression 187

27 Lossy Compression 192

28 Creating Summary and Memories Together 193

Introduction to Context Engineering

1 Context and Context Engineering

1.1 Why Multi-Turn Conversations Seem to "Break" Sometimes

Consider two identical ChatGPT instances given the exact same two prompts:

1. "Who directed Bahubali?" → Both instances correctly answer **SS Rajamouli**.
2. "Which other movies did he direct?" → Instance 1 correctly resolves "he" to SS Rajamouli and lists his filmography. Instance 2 asks "which director are you referring to?" — it fails to resolve "he" at all.

Same model, same prompt, different outcome. The reason lies in how LLMs handle memory.

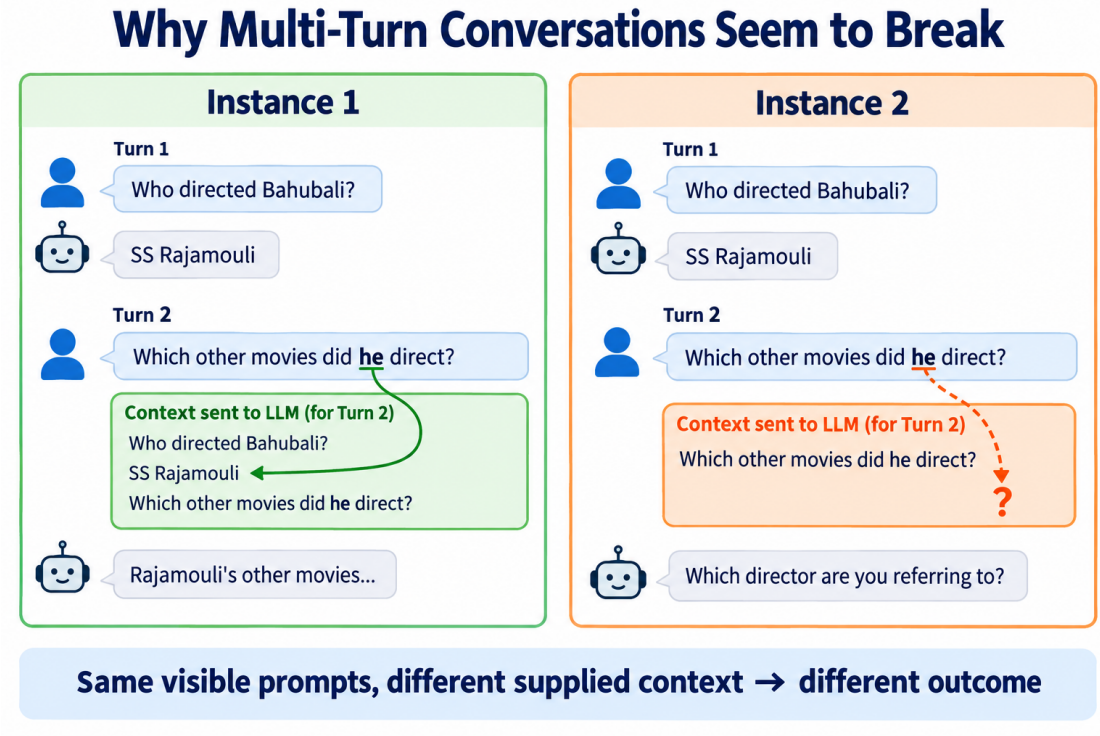

Figure 1. The same follow-up question succeeds when prior conversation context is supplied and fails when that context is missing.

1.2 LLMs Are Stateless

Stateless means an LLM has zero memory. The moment it finishes generating a response, it forgets everything — the query, its own answer, all of it. There is no built-in persistence between calls.

> 🧠 **Analogy — LLMs and Ghajini**
> An LLM behaves like the protagonist in the movie *Ghajini*, who loses his memory every 15 minutes. Except an LLM doesn't even get 15 minutes — the instant it produces an output, its memory is wiped.

This raises an obvious question: if LLMs remember nothing, how do multi-turn conversations and agentic systems (with tool calls, memory retrieval, etc.) work at all?

Answer: context. An LLM doesn't *remember* anything between calls — but if the relevant history is *re-supplied* to it on every call, it behaves as if it remembers.

How This Works Mechanically

Every time you send a prompt to a chatbot, an API call is made to the LLM. What's actually sent in that call is not just your latest message — it's the accumulated message history:

- **Call 1:** `[User prompt 1]`
- **Call 2:** `[User prompt 1, Assistant response 1, User prompt 2]`
- **Call 3:** `[User prompt 1, Assistant response 1, User prompt 2, Assistant response 2, User prompt 3]`

This is confirmed by OpenAI's API documentation: each call passes the full history list, the model's new output is appended to that list, and the updated list is passed again on the next call — a loop. The growing history *is* the context, and it's what lets the model resolve "he" → SS Rajamouli in the working instance. The failing instance simply hadn't been given that prior exchange as context.

CampusX

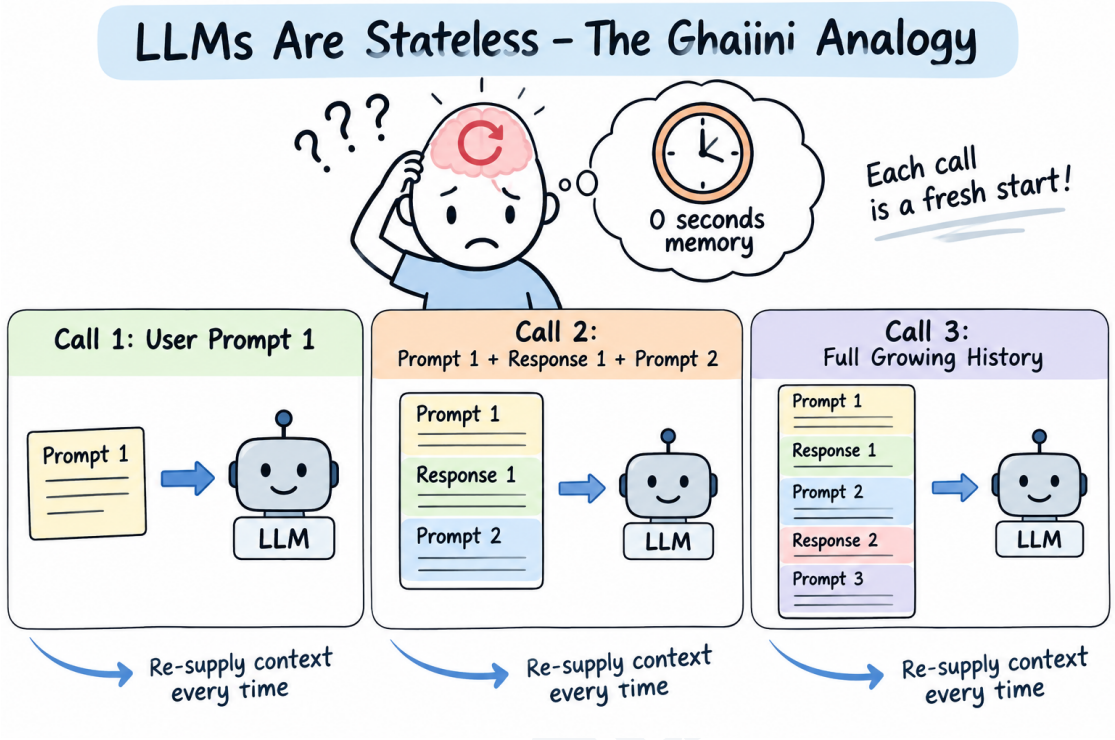

Figure 2. LLMs are stateless: each call starts fresh, so the growing conversation history must be supplied again with every request.

1.3 What Is Context?

The general (non-AI) definition of context: *the information that surrounds a word, phrase, or situation and helps you understand its meaning.*

Example: "I am going to bank" is ambiguous — river bank or financial bank? Adding "…to get some fish" resolves it instantly. That surrounding information is context. The same idea applies to entering a movie mid-scene (you don't know why two characters are fighting) or interpreting sarcasm — you need situational context to interpret meaning correctly.

In LLM/AI terms: Context is the background information, memory, and data passed to an AI system, which the system uses to understand a query and generate a relevant response.

Practical Demonstrations

Purpose: Show that answering a real-time query depends on context, not the model itself.

Essential action: Same prompt — *"What is the current weather of Gurgaon?"* — run on Gemini in Google AI Studio, once with no tools and once with **Grounding with Google Search** enabled.

Observed result: Without grounding, the model says it can't access real-time data. With grounding on, it returns live weather data (high, low, humidity, conditions) plus the sources it pulled from.

Key lesson: LLMs are capable but cut off from the world beyond their training data. Same model, same prompt — only the context changed.

The same principle underlies **RAG (Retrieval-Augmented Generation)**: a user query is used to retrieve relevant chunks from a database; those chunks are passed to the LLM alongside the query as context, and the LLM generates its answer from them.

> 💡 **Insight — Why Context Matters**
> Without adequate context, an LLM either fails to answer or gives a generic, low-value answer. In production, insufficient context is one of the most common causes of poor output — the model isn't "wrong," it's under-informed.

1.4 The Seven Components of Context

Context is commonly (and incorrectly) equated with just "message history." In reality, message history is only one of several components that make up context:

#	Component	What It Is
1	**Message history**	The full back-and-forth of prompts and responses so far in the conversation
2	**Current user prompt**	The query being asked right now
3	**System instruction**	Rules governing the AI's behavior, tone, persona, and output format
4	**Tool/function schemas**	Structured (usually JSON) descriptions of available tools — what they do, their parameters, and data types
5	**Tool calls & their outputs**	The JSON request made to invoke a tool, and the JSON result it returns
6	**Injected information (e.g., RAG)**	Retrieved chunks or external data inserted into the context to ground the response
7	**Memory & parameter settings**	Persisted user-specific information (name, preferences, past instructions) plus model parameters (temperature, top-p, top-k)

CampusX

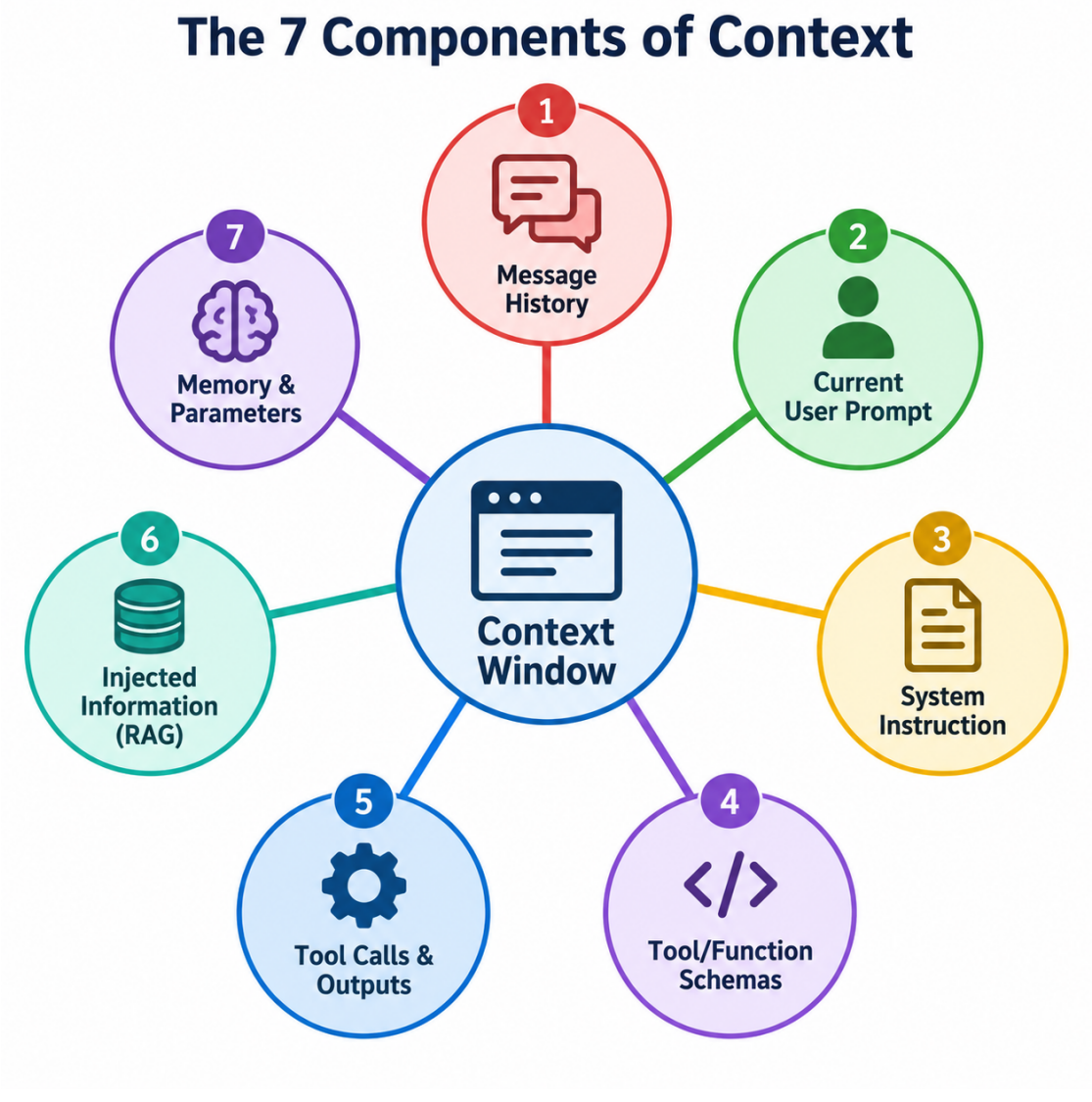

Figure 3. The seven components that can contribute to an LLM's context window.

Message History

The running record of prior prompts and responses (Prompt 1 → Response 1 → Prompt 2 → Response 2 → …). It lets the model resolve follow-up references (e.g., "explain that further") because it can see what "that" refers to.

Current User Prompt

Simply what the user is asking right now — the immediate query the model must respond to, on top of everything else in context.

System Instruction

Functions like a constitution for the AI application — it defines persona, tone, output format, goals, and behavioral guardrails.

Example system instruction: *"You are a helpful, concise, and friendly AI system. Provide accurate, useful, and up-to-date information. Explain complex topics simply. Be polite and encouraging. Respond in English. Keep answers clear and concise. Use examples when helpful."*

The resulting output style (short, example-driven answers) directly reflects these instructions — system instruction shapes not just *what* is said but *how*.

Tool / Function Schemas

LLMs are cut off from the outside world and cannot natively fetch live data or take real-world actions. **Tools** (typically Python functions) extend an LLM's capabilities — e.g., a function that calls a weather API.

To let the LLM use a tool, its schema (usually JSON) is provided as context, describing:

- what the tool does,
- its parameters and their data types,
- which parameters are required,
- what output it returns.

Example schema for a `get_weather` function:

```
{
  "name": "get_weather",
  "description": "Gets the current weather for a given location.",
  "parameters": {
    "location": { "type": "string", "description": "City name" },
    "units": { "type": "string", "description": "celsius or fahrenheit" }
  },
  "required": ["location"]
}
```

The LLM reads this schema to understand what the tool can do and when to invoke it.

CampusX

Tool Calls & Their Outputs

When the model decides to use a tool, it generates a JSON tool call specifying the function name and parameter values. The tool executes and returns a JSON output. **Both the call and its result become part of context**, informing the model's next response.

Example tool call:

```
{ "tool": "get_weather", "location": "Gurugram", "units": "celsius" }
```

Example tool output:

```
{ "status": "success", "location": "Gurugram", "temperature": "...",
  "condition": "...", "humidity": "...", "wind": "..." }
```

Injected Information (RAG)

Chunks retrieved from a database or knowledge base in response to a query. These are injected into context so the model can ground its answer in retrieved facts rather than relying purely on parametric knowledge.

Memory & Parameter Settings

Memory: information accumulated about the user over time — name, workplace, preferences, past instructions — that the assistant recalls and reuses without being told again each time.

Parameter settings: the inference-time configuration of the call itself — temperature, top-p, top-k, max tokens, etc. These shape output style/randomness and are also part of context.

Not All Seven Are Always Present

Which components appear depends on the type of AI system:

- **Simple chatbot** (e.g., basic ChatGPT/Gemini use): message history, current prompt, system instruction, memory, parameters.
- **Agentic system**: adds tool schemas and tool calls/outputs on top of the above.
- **RAG system**: adds injected retrieved content, but typically omits tool schemas/calls.

Worked Example: Customer Support Assistant

A support agent's context, assembled from multiple components, might include:

- Model parameters (max tokens, temperature)
- Tool schemas (`search_knowledge_base`, `get_user_account`)
- System instruction ("You are a helpful customer support assistant for ACME SaaS…")
- Conversation history (user: "I was charged twice for my subscription")
- Tool call outputs (account details from `get_user_account`, refund policy info from `search_knowledge_base`)

Only with all of this assembled can the assistant correctly respond: *"You're eligible for a refund for the duplicate charge."* Remove any one piece and the answer becomes incomplete or impossible.

1.5 What Is Context Engineering?

Context engineering is the practice of deliberately designing, structuring, and optimizing the context provided to an LLM so it produces more accurate, relevant, and reliable output.

It answers two questions:

- **What** should be included in the context?
- **How** should it be structured/sequenced?

> 💡 **Insight — Context as the Differentiator**
> As more companies build on the same underlying models (e.g., all using the same foundation model provider), the model itself stops being a competitive advantage. Two companies can build customer-support chatbots on identical AI — but the one that supplies richer, better-structured context (user data, billing history, product docs) will consistently outperform the one that only passes message history and system instructions. The differentiator shifts from *which model* to *how well context is engineered*.

1.6 Context Engineering vs. Prompt Engineering

Aspect	Prompt Engineering	Context Engineering
Focus	Instructions	Information
Core question	How do I phrase this effectively?	What data do I supply, and how?
Example concern	Structuring few-shot examples, wording of instructions	What documents, tool outputs, or history to include alongside the prompt

> 🧠 **Analogy — Onboarding an Intern**
> Hiring and onboarding a testing intern: first, you explain *how* to test — benchmarks, standard procedures (this is **prompt engineering**, telling the model how output should look). Then, you hand over the codebase — but selectively, not the entire repository; some files are given, some withheld (this is **context engineering**, deciding what data/information the intern — or LLM — actually gets to work with).

> 🧠 **Analogy — The Unstructured Exam Answer** Even a student who has fully studied the material can still perform poorly if they don't structure their exam answer well. Similarly, an LLM can be within its token limit and still produce a poor output if the context it receives is unstructured or poorly organized — staying under the context window limit is not sufficient on its own; the context also has to be well-structured.

In short: Prompt engineering is about instructions. Context engineering is about information — what you provide, and how you structure and deliver it alongside the prompt.

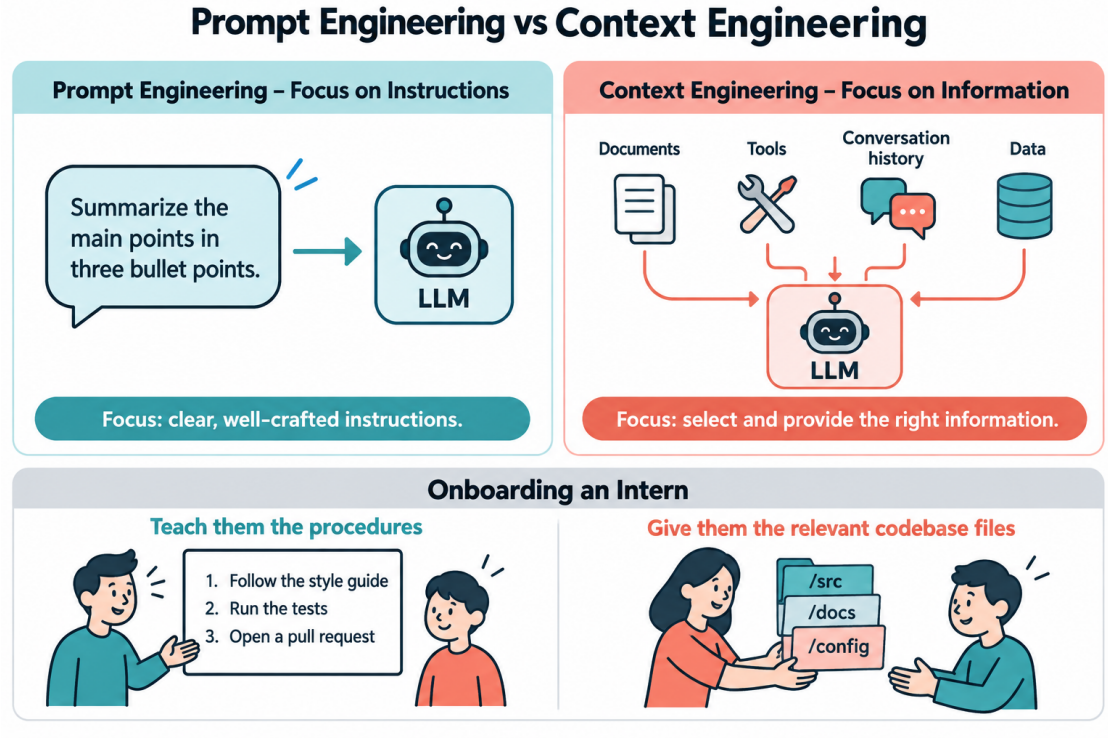

Figure 4. Prompt engineering improves the instructions, while context engineering selects and supplies the information the model needs.

2 Why Context Engineering

Building modern AI systems (agentic applications, agentic RAG, tool-using assistants) creates context-related problems from **two ends**:

1. The **AI system end** — the application itself.
2. The **LLM end** — the model powering that application.

Problem 1: The AI System End — Context Piles Up Fast

Modern AI applications *start* with a large amount of context already baked in, and then *accumulate more* as they run.

At design time, before the application is ever used, a developer typically supplies:

- System instructions
- Tool/function schemas (potentially several tools)
- Model parameters (if tuned)
- Environment details the app runs in

At runtime, once the application is in production and being used, it keeps accumulating further context:

- Message history (the ongoing conversation)
- Tool calls and their outputs
- Injected content (e.g., retrieved chunks in a RAG setup)

Worked Example: A Portfolio-Analysis Agent An agent is set up with three tools — `read_local_file`, `get_stock_data` (via a market data API), and `execute_python` — plus a system instruction and environment details. This is the context that exists *before* a single user interaction happens.

A user then asks: *"Check my portfolio, see how my NVDA shares are doing, and calculate my unrealized profit."*

To answer this single request, the agent:

1. Calls `read_local_file` to read `portfolio.csv` → output (holdings data) becomes context.
2. Calls `get_stock_data` to fetch NVDA's live price → output becomes context.
3. Calls `execute_python` to compute cost, current value, and profit → output becomes context.
4. Generates a response summarizing all of this.

The user then follows up: *"Do the same for AAPL and MSFT… yes, do the rest, and plot a bar chart comparing all three."*

CampusX

To resolve *just* "do the rest" and "all three," the LLM needs the **entire accumulated context** above — every tool schema, every tool output, the system instruction, and the full conversation history — even though only two or three prompts have been exchanged.

> 💡 **Insight — Context Grows Faster Than the Conversation**
> A handful of user turns can produce a very large context footprint once tool schemas, tool outputs, and system instructions are all counted. The problem isn't the number of messages — it's everything silently accumulating behind each message.

Problem 2: The LLM End

The LLM powering the application introduces its own set of constraints, independent of how the application is built.

2a. You Cannot Pass Infinite Context Every LLM has a **context window**: the maximum number of tokens it can process or "remember" while generating an output. This is never infinite — it's a hard limit set by the model.

Representative context window sizes (illustrative figures from the lecture — check current provider docs for up-to-date numbers, as these change frequently):

Model	Context Window
GPT-4o	128K tokens
Claude Opus	200K tokens
Gemini 1.5 Pro	1M tokens
Gemini 1.5 Flash	1M tokens
Mixtral 8x22B Instruct	65K tokens

A larger context window means more context can be supplied per call — but every model still has a ceiling.

What happens when the limit is exceeded? Two common strategies:

1. **Sliding window (FIFO eviction):** Once the limit is reached, the oldest tokens are dropped from memory to make room for new ones. If a new prompt or response pushes the total over the limit, the earliest tokens are evicted first, regardless of their importance.
2. **Summarization:** Instead of evicting old content outright, some systems compress older parts of the context into a summary and keep that summary in context, freeing up space while preserving the gist.

CampusX

2b. The Context Window Isn't Just "Input" A common misconception: if a model has a "128K context window," people assume that means 128K tokens of *input* can be sent. This is incomplete.

The context window is the total budget across **three** components:

$$\text{Context Window} = \text{Input Tokens} + \text{Reasoning Tokens} + \text{Output Tokens}$$

This follows from how transformer attention works: each generated token is predicted based on *all* preceding tokens — including the input, any intermediate reasoning, and the output generated so far. All of that consumes the same token budget.

> 💡 **Insight — Budget the Full Round-Trip, Not Just the Prompt**
> When estimating whether a call will fit in a model's context window, account for input + reasoning + output together — not just the size of the prompt you're sending. A large prompt paired with a long expected output (or a reasoning-heavy model) can hit the limit even when the input alone looks well within range.

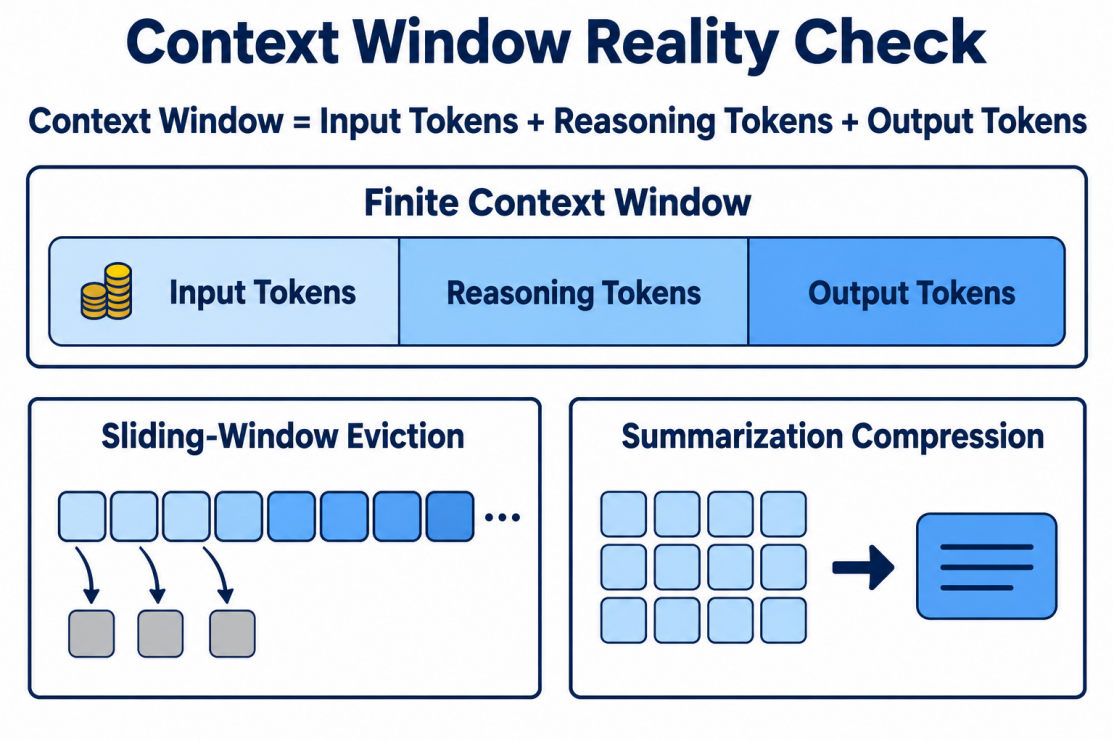

Figure 5. A context window is a finite shared budget for input, reasoning, and output tokens; older context may be evicted or summarized.

2c. Cost of Input LLMs are used in one of two ways: through a chatbot interface, or via API calls to a server. In both cases, mechanically, an API request carrying your input is sent to the model's server, which processes it and returns a response.

Whenever an API call is made, the input tokens sent carry an associated **cost** — every token passed as context is not free.

Both the input and the output of an API call carry a cost — not just the response.

Representative per-million-token pricing cited in the lecture (illustrative — check current provider pricing pages for up-to-date rates):

Model	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)
GPT-4o	$2.50	$10.00
GPT-4 Turbo	$10.00	$30.00
Gemini 1.5	$3.50	$10.50

Every token sent as input — including irrelevant filler, stray punctuation, or unnecessary boilerplate — is billed. If the context passed to the model contains irrelevant information, that's **unnecessary cost being paid for no benefit**.

What actually gets sent as "input" in an API call is, in practice, just the assembled context: system instruction + tool schemas + message history + current prompt. So poorly managed context directly translates into inflated spend — this is the second LLM-end problem: **unnecessary cost of input**.

2d. Context Rot

Context rot: as the amount of context grows, an LLM's ability to effectively use that context degrades — content is technically present in the context window but isn't being used effectively, similar to how something "rots" when it exists but goes unused.

This is commonly felt as: the longer a multi-turn conversation runs, the more generic or degraded a chatbot's responses become — even though nothing about the underlying model has changed.

Two primary causes:

i. Attention Dilution

Every LLM effectively has a fixed **attention budget** that gets distributed across the entire context. As context grows longer, it typically accumulates irrelevant information (e.g., a tool call's output from several turns ago that's no longer relevant to the current query). Because the attention budget must be spread across *everything* in context — relevant and irrelevant alike — genuinely important information ends up receiving less attention than it deserves, simply because it has to "share" the budget with clutter.

> 💡 **Insight — Attention Is a Shared, Limited Resource**
> The problem isn't that irrelevant context is "wrong" — it's that every extra token in context competes for the same fixed attention budget as the tokens that actually matter for the current response. More irrelevant context directly means less effective attention on what's relevant.

ii. Lost in the Middle

A positional bias in how LLMs attend to long contexts: information positioned at the **beginning or end** of the context window tends to get more attention, while information in the **middle** gets comparatively less — regardless of its actual relevance. This is documented in the "Lost in the Middle: How Language Models Use Long Contexts" research paper, which found that model performance is generally highest when relevant information appears at the start or end of the input, and degrades noticeably when the needed information sits in the middle — even for models explicitly built for long contexts.

Together, attention dilution and lost-in-the-middle mean that simply having information *present* in context is not the same as the model *using* it effectively.

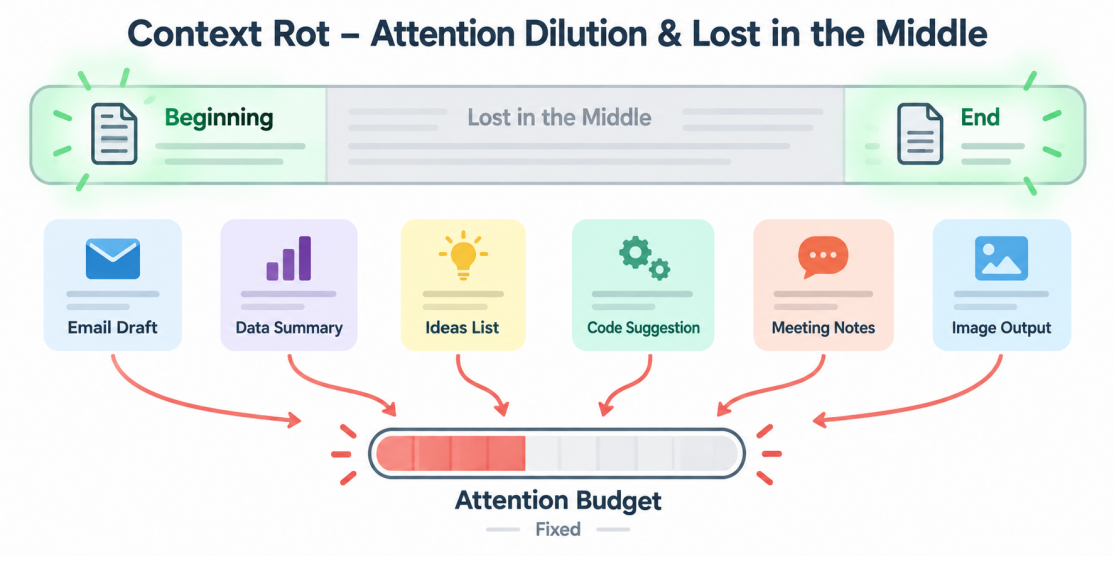

Figure 6. Context rot combines a fixed attention budget with weaker use of information positioned in the middle of a long context.

2.1 How Context Engineering Solves Both Problems

Recap of the two problem sources:

- **AI system end:** context is generated and accumulates constantly, and this accumulation cannot be stopped — the system needs that context to know what to do next.
- **LLM end:** context windows are finite, input carries real cost, and long/cluttered context causes context rot (via attention dilution and lost-in-the-middle).

Since context generation itself can't be prevented, the lever available is **how that context is managed** — which is exactly what context engineering does. By curating, structuring, and removing irrelevant context before it reaches the LLM and by surfacing the right data at the moment it's actually needed rather than front-loading everything — the following problems are addressed:

- Context overflow (exceeding the context window) is avoided.
- Unnecessary cost from irrelevant input tokens is avoided.
- Attention dilution and lost-in-the-middle effects are minimized, since less clutter means the attention budget concentrates on what's actually relevant.

This is why context engineering has become such a widely discussed topic: as more companies build agentic systems on similar underlying models, all of them run into these same two categories of problems, and context engineering is the shared solution.

2.2 Consequences of Skipping Context Engineering

If context is not properly engineered, a modern AI system will typically face:

- **Context rot** — degraded, generic output despite a capable underlying model.
- **Higher cost** — paying for irrelevant tokens repeatedly sent as input.
- **Hallucination** — when the LLM lacks the proper context to answer, it tends to fill in the gaps itself, producing fabricated or unreliable output.
- **Context clash** — the context window contains contradictory information (e.g., one instruction says "use Tool A," another says "don't use Tool A"), leaving the model confused and producing inconsistent or unreliable behavior.
- **Reduced reliability** — overall inconsistency in results, undermining trust in the application.

2.3 A Refined Definition of Context Engineering

Context engineering is the practice of engineering the context sent to an LLM so that:

- Only **relevant** context is passed to the model.
- **Unwanted/irrelevant** context is actively removed before being passed.
- The context window is properly curated, managed, and structured so that the system never experiences context overflow, never pays unnecessary cost, and never suffers from context rot.

Context overflow: sending more context to an LLM than its context window can hold.

🧠 **Analogy — The World-Class Chef**
A world-class chef in a restaurant with poor raw materials, bad spices, and an inadequate kitchen setup will still produce a mediocre dish — no matter how skilled they are. The chef's talent is capped by the quality of what they're given to work with.

🧠 **Analogy — Powerful CPU, Bad RAM**
A very powerful CPU paired with poor RAM performs no better than a weak setup, because the CPU can only work with the data available to it in RAM. Similarly, a powerful LLM given poor-quality context will perform no better than a cheaper, weaker LLM — the model's power is bottlenecked by the quality of its input context, not by its raw capability.

2.4 Two Common Misconceptions

Misconception 1: "More context always means better output."

Not true in general. Excess irrelevant context leads directly to context rot and risks context overflow. It's only true when the *existing* context is sparse or empty — e.g., adding a missing system instruction, a needed output format, or a relevant tool schema genuinely improves output. But once the context already contains what's relevant, adding more (unnecessary) context degrades rather than improves performance.

Misconception 2: "Context windows are getting huge (up to 1M+ tokens), so context management isn't necessary anymore."

Also not true. Even with a very large context window, unmanaged context still causes: increased cost (every token is billed), context rot (attention dilution and lost-in-the-middle still apply), and increased latency — a bigger window doesn't remove any of these problems, it just delays when they show up.

2.5 Golden Rules

⚠ **Golden Rule — Curate, Don't Maximize**
More context is not inherently better. The goal of context engineering is always to pass the *most relevant* context to the LLM and actively remove what isn't needed — not to fill the available context window as much as possible.

⚠ **Golden Rule — Manage Context Regardless of Window Size**
A large context window does not eliminate the need for context engineering. Cost, context rot (attention dilution, lost-in-the-middle), and latency are all still governed by how much — and how relevant — the context actually is, independent of the model's maximum capacity.

CampusX

> ⚠ **Golden Rule — Position Matters, Not Just Presence**
> Information technically being "in context" does not guarantee the model will use it well. Due to lost-in-the-middle effects, place the most critical information near the beginning or end of the context sent to the model where possible, rather than assuming any position is equally effective.

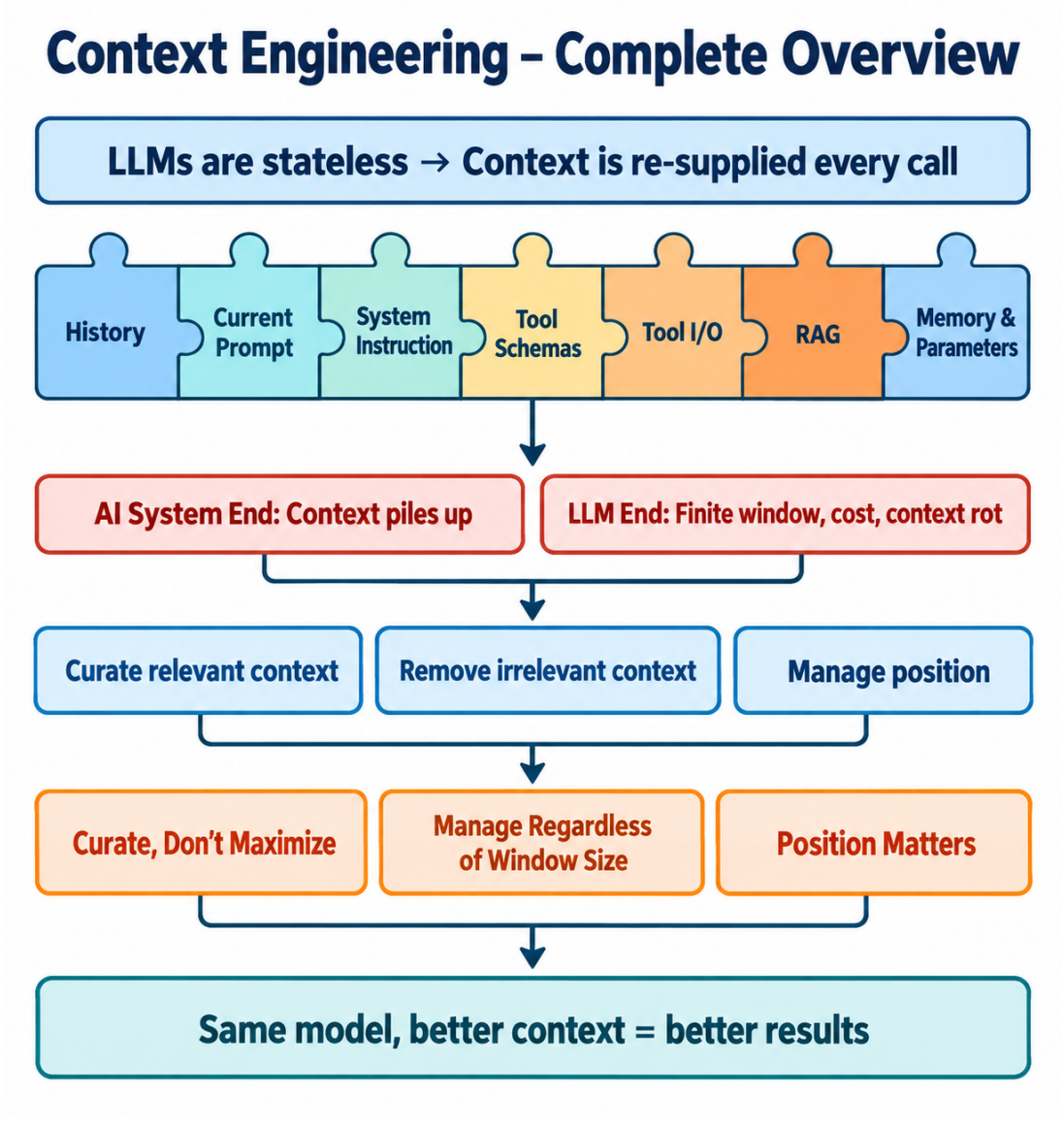

Figure 7. Complete overview of context engineering: assemble the right context, remove irrelevant material, and position important information deliberately.

COURSE MODULE

Structuring Prompt Sections and KV Caching

3 Writing Effective System Instructions

3.1 Two Approaches to Context Engineering

Context engineering can be done in two broad ways:

- **Engineering ways** — writing code/logic that manages context programmatically.
- **Theoretical ways** — applying established frameworks and good practices for structuring context.

This module focuses on the theoretical side: how to write **effective system instructions**, one of the core frameworks for managing context.

Recap — why context engineering matters: it lowers cost, reduces hallucination, prevents *context rot*, and keeps the **signal-to-noise ratio** high (i.e., maximizes relevant information relative to irrelevant information in what's passed to the LLM). Context itself is made up of system instructions, message history, tool calls, memory, and retrieved documents.

CampusX

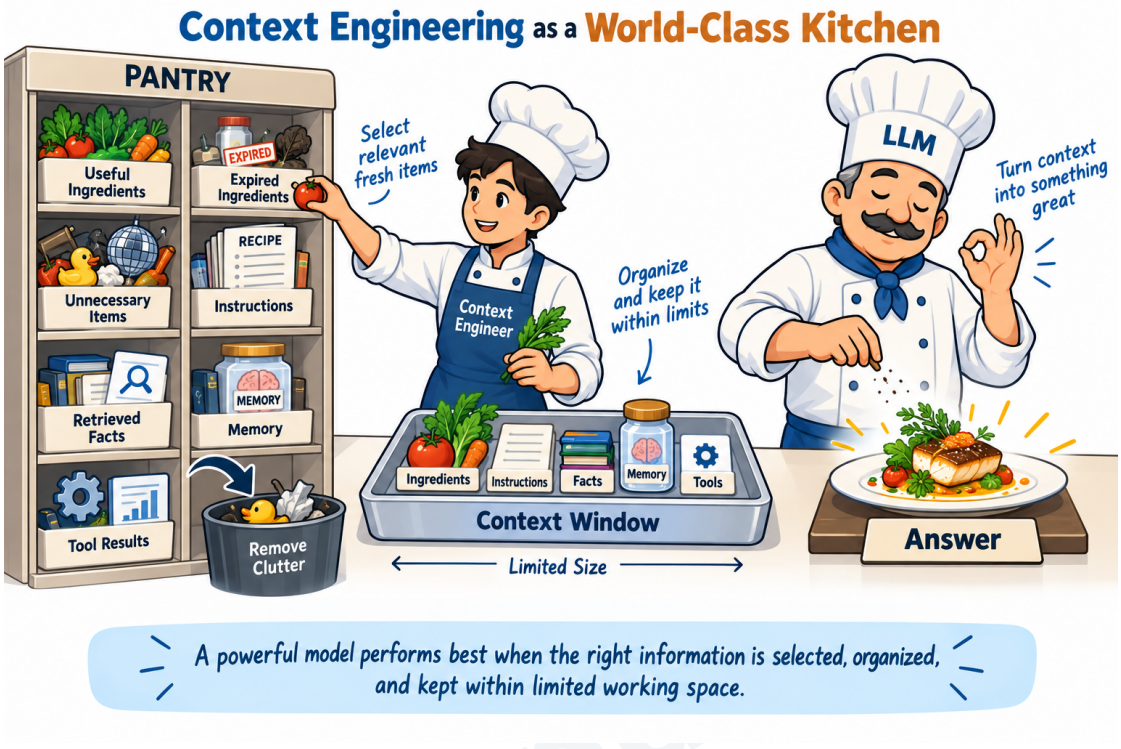

Figure 8. Context engineering selects useful information, removes clutter, and organizes the remaining context within the model's limited working space.

3.2 What Is a System Instruction?

A **system instruction** (also called a system prompt or system message) is a foundational set of rules and directives given to an LLM. It acts as the underlying framework that guides how the LLM responds, interprets information, and interacts with users — covering persona, constraints, tools, formatting, and behavior.

> 🧠 **Analogy — The Director and the World-Class Actor**
> Imagine a world-class actor who can play any role — hero, villain, any accent, any emotion. Before shooting a movie, a director tells this actor exactly who their character is, how to behave, and what tone to take. The actor then carries these instructions through the entire performance.
> The actor is the LLM (capable of anything). The movie is your AI application. The director's instructions are the **system instruction** — they shape how the general-purpose LLM behaves for this *specific* application.

Short definition: a system instruction is the **constitution** of your AI application — just as a country's constitution governs its laws and decisions, the system instruction governs everything the LLM does: its behavior, tone, and interaction style. It is effectively a *rule book* for the application.

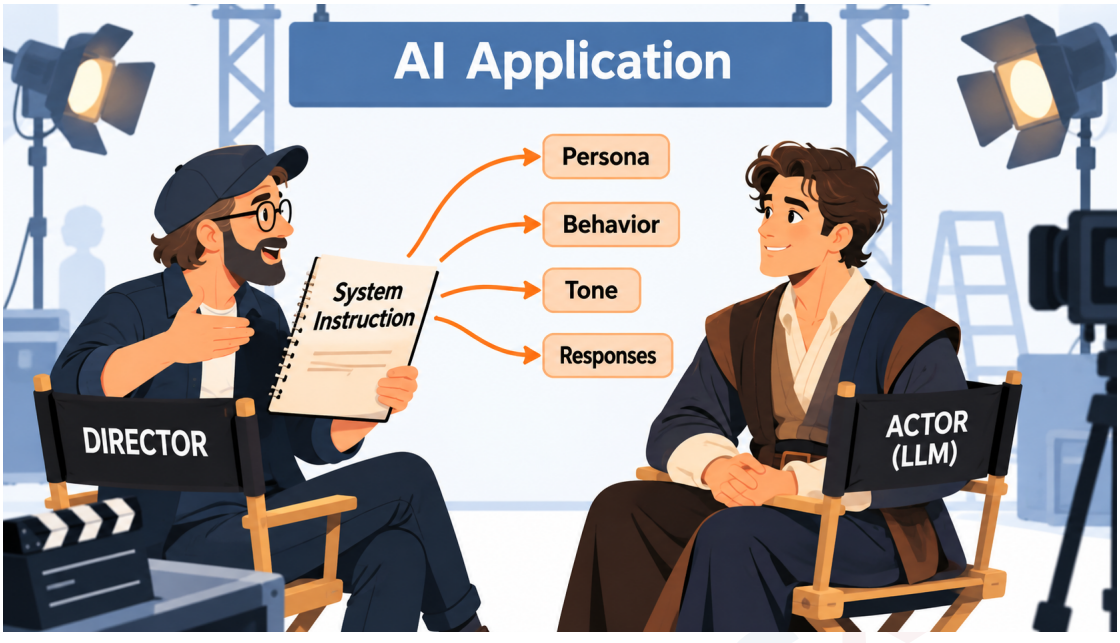

Figure 9. A system instruction acts like a director guiding an adaptable actor by defining the application's persona, behavior, tone, and responses.

Why Effective System Instructions Matter

1. **Direct control** — system instructions are the primary way you tell an LLM application what task to perform, how to perform it, and what to avoid. Write them poorly and you lose your main lever over the model's behavior.
2. **Indirect context management** — since system instructions are one of the components that make up context, writing them well is itself a form of context curation.

3.3 Four Core Elements of an Effective System Instruction

These four elements should be defined clearly, and — as a practical rule of thumb — written **in this order**: persona/task first, then constraints, then style, then audience.

#	Element	What It Covers
1	**Persona & Task**	Treat the LLM as a human entity: job title, role, experience level, specialization, and the core task it must perform
2	**Task Constraints & Boundaries**	What the LLM must *not* do — disallowed actions, topics to avoid, things it shouldn't fabricate

CampusX

#	Element	What It Covers
3	**Communication Style & Format**	Tone (formal/casual/technical), vocabulary level, and output structure (bullets, JSON, Markdown, etc.)
4	**Audience**	Who the end user is, so the depth and framing of responses match them (e.g., explaining "What is AI?" differs for a 6th grader vs. a researcher)

Example — Persona & Task:

> "You are a Senior Curriculum Architect with 15 years of experience in EdTech. You are knowledgeable and encouraging, specialized in teaching by breaking down complex technical topics into digestible, project-based learning modules. You value learning by doing over pure theory."

Example — Task Constraints & Boundaries:

> "Avoid generating content related to illegal, harmful, or sensitive topics. Do not guess, fabricate facts, or hallucinate. Do not provide legal, medical, or professional advice. Stay strictly within the assigned task."

Example — Communication Style & Format:

> "Use clear, concise, beginner-friendly language. Output in bullet points, short paragraphs, and step-by-step explanations, formatted as JSON."

Example — Audience:

> "Your audience is students and early-career professionals with basic foundational knowledge and limited hands-on experience. Generate explanations in an accessible, practical way."

Beyond these four, a system instruction can also include few-shot examples, output-format samples, and other application-specific guidance.

In practice, all four elements combine into one instruction block — persona and task stated first, followed by constraints, style, and audience — forming a complete "constitution" for the application. Most LLM APIs (e.g., Gemini) expose a dedicated `system_instruction` parameter for passing this in.

Figure 10. The four core elements of an effective system instruction: persona and task, constraints and boundaries, communication style and format, and audience.

3.4 Writing at the "Right Altitude"

Anthropic's guidance on effective context engineering (from their blog *"Effective Context Engineering for AI Agents"*) states that system prompts should be **clear** and pitched at the **right altitude** — specific enough for reliable behavior, but not so rigid that the LLM can't use its own reasoning on edge cases.

Right altitude is the **Goldilocks zone** between two failure modes:

Altitude	Problem	Example
Low altitude	Instructions are hard-coded and brittle; edge cases break the system	"If the word 'bug' appears more than 3 times, score negative. If 'fast' appears, score positive." Fails on sarcasm: *"The database is so fast, it makes my old setup look like a joke — but the lack of documentation is a massive bug"* gets misread purely from keyword matching, missing the actual mixed sentiment.
High altitude	Instructions are too vague; too much is left to the LLM's imagination	"Tell us how the community feels about our product and what we should do next." No guidance on *how* to analyze, what to include, or how to weigh signals.
Right altitude	Specific enough to be reliable, flexible enough to reason through edge cases	Defines persona ("techno-managerial data analyst"), explains *how* to reach conclusions (assess tone, detect sarcasm, justify each rating with context), sets guardrails (no hallucination, paraphrase technical issues, no filler phrases), and specifies exact output format — without hard-coding keyword rules.

> 💡 **Insight — Teach the AI to Read the Map**
> Writing at the right altitude means teaching the AI *how to reason its way to a good answer*, rather than giving it turn-by-turn instructions for every scenario. Don't just tell it the route from A to B — teach it to read the map so it can find its own way when the situation doesn't match your exact assumptions.

CampusX

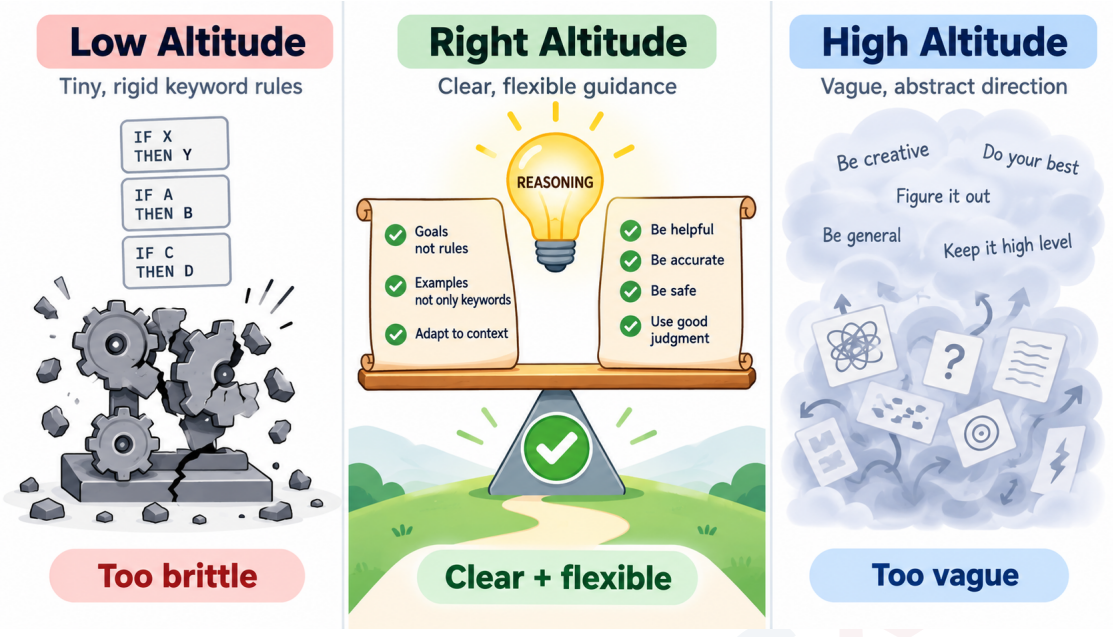

Figure 11. The right altitude balances clear guidance with enough flexibility for the model to reason, avoiding brittle rules and vague directions.

3.5 Writing a Minimal Set of Information

Another practice from Anthropic's guidance: aim for the **minimal set of information that fully outlines expected behavior**.

"Minimal" does **not** mean omitting necessary information — it means avoiding verbose, repetitive prose. The same instruction content can usually be compressed using **delimiter tagging** (labeled fields like `role:`, `objective:`, `instructions:`, `guardrails:`) instead of long paragraphs.

Verbose version (same content, harder to scan):

> "You are going to act like a really helpful data science and AI mentor for my students… When a student asks a question about their code, or if their code is throwing an error, please do not just give them the final answer or write the whole script for them… Instead, use the Socratic method…"

Minimal version (same information, delimiter-tagged):

```
Role: Expert data science mentor
Objective: Guide students to solve their own coding errors and grasp
           the underlying concept, without doing the work for them
Instructions:
  - Always use the Socratic method
  - Ask targeted guiding questions that lead the student to discover
```

CampusX

```
    the error or logic flaw themselves
  - Provide conceptual hints or small syntax examples, but never
    output the fully corrected, copy-pasteable script
  - Maintain an encouraging, academic tone
Guardrails:
  - Only answer questions related to AI/programming
  - Format any code/hints using proper Markdown
```

Both versions carry identical information — the tagged version is simply faster to write, read, and maintain.

3.6 Iterative Approach: Start Minimal, Then Refine

Rather than writing one large "perfect" prompt upfront, build system instructions iteratively:

1. Write a **minimal prompt** covering persona, task, and output schema.
2. **Test** it against real inputs and note failure patterns.
3. Add **targeted instructions** to fix each observed weakness.
4. Repeat until the output is reliable.

Worked example — an agent that extracts structured data (`model`, `dataset`, `key_finding`) from AI research paper abstracts:

- **v1 (minimal):** persona + task + a 3-key JSON output schema. No further constraints.
- **Observed failures after testing:**
  - `key_finding` summaries were too long.
  - Missing fields were inconsistently rendered as `"N/A"` or `null`.
  - Output sometimes came back as plain text instead of JSON.
- **v2 (refined):** same objective, plus targeted fixes —
  - "Key finding must always be under 50 words."
  - "If information is missing, output `null` — never `N/A` or other placeholders."
  - Added a worked input/output example to lock in consistent JSON formatting.

This loop — minimal prompt → test → patch specific failures → retest — converges on a system instruction that is both minimal and reliable, without over-engineering upfront.

CampusX

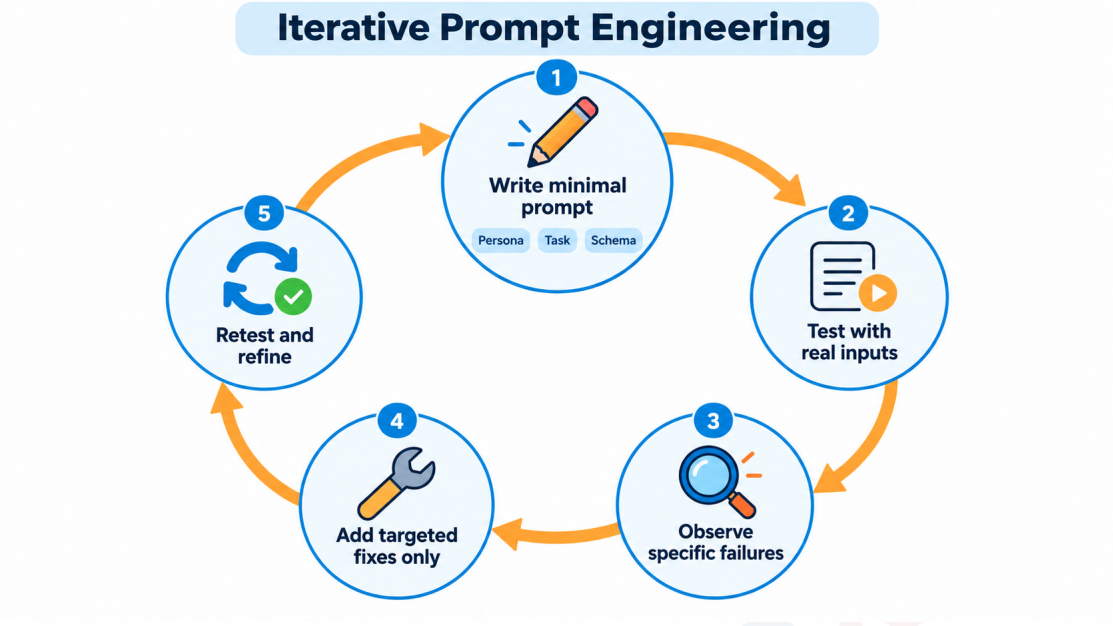

Figure 12. An iterative prompt-engineering loop starts minimal, tests real inputs, observes failures, adds targeted fixes, and retests.

3.7 Benefits of Writing Effective System Instructions

- The LLM reliably knows what to do, how to do it, and what to avoid.
- Output can be shaped precisely to your desired format and style.
- **Lower cost and compute** — a crisp, non-verbose system instruction uses fewer tokens.
- **Better security** — clearly defined "no-go" areas reduce misuse and off-scope behavior.
- Indirectly, it's a form of context engineering: since system instructions are one component of the overall context, curating them well means the broader context is better managed too.

4 Structuring the User Prompt Section

What Actually Goes Into the User Prompt Section

Only two placeholders exist for passing input to an LLM: the **system instruction** (covered in Part 1) and the **user prompt**. A common misconception is that the user prompt only carries the user's current query — in reality it is a composite of almost everything in the context window *except* the system instruction:

- **Message/conversation history** — prior turns between user and assistant
- **Retrieved documents** — content pulled in via RAG
- **Tool call outputs** — results returned from any tools/functions invoked

- **Long-term memory** — stored user preferences, likes/dislikes, profile facts
- **The user's current query** — the immediate prompt itself

Because so many different, dense types of information converge in one section, how you assemble them matters as much as what you include.

The "Wall of Text" Problem

If all of the above is concatenated as plain, undifferentiated text — assistant replies, tool outputs, retrieved chunks, and the query all running together with no visual or structural boundary — the result is a **wall of text**: one continuous block with no demarcation, no sub-sections, no distinction between what's an instruction and what's data.

This creates five distinct failure modes:

Problem	What Happens	Why It Hurts
Semantic contamination	Content from unrelated sources sits adjacent with no boundary, so their meanings bleed into each other	LLMs compute attention scores based on surrounding words; two unrelated chunks (e.g., from different RAG sources) can get merged into a meaning neither originally had
Conflation of instruction and data	The model can't tell which tokens are *instructions* (how to behave) vs. *data* (what to reason over)	Instructions should get more weight than data, but in a flat block everything is treated with roughly equal importance
Impaired target retrieval	You can't tell the LLM "use only this specific section"	Targeted context engineering — pointing the model at one part of the input — requires boundaries that don't exist in plain text
Pattern recognition failure	In few-shot prompting, the model can't tell where one example's input ends and its output begins (or where example 1 ends and example 2 starts)	Breaks the model's ability to learn the input→output pattern you're demonstrating
Attention dilution	Important tokens (instructions, key data) don't get proportionally more attention	The model treats the whole block as one undifferentiated blob rather than concentrating attention on what matters most

CampusX

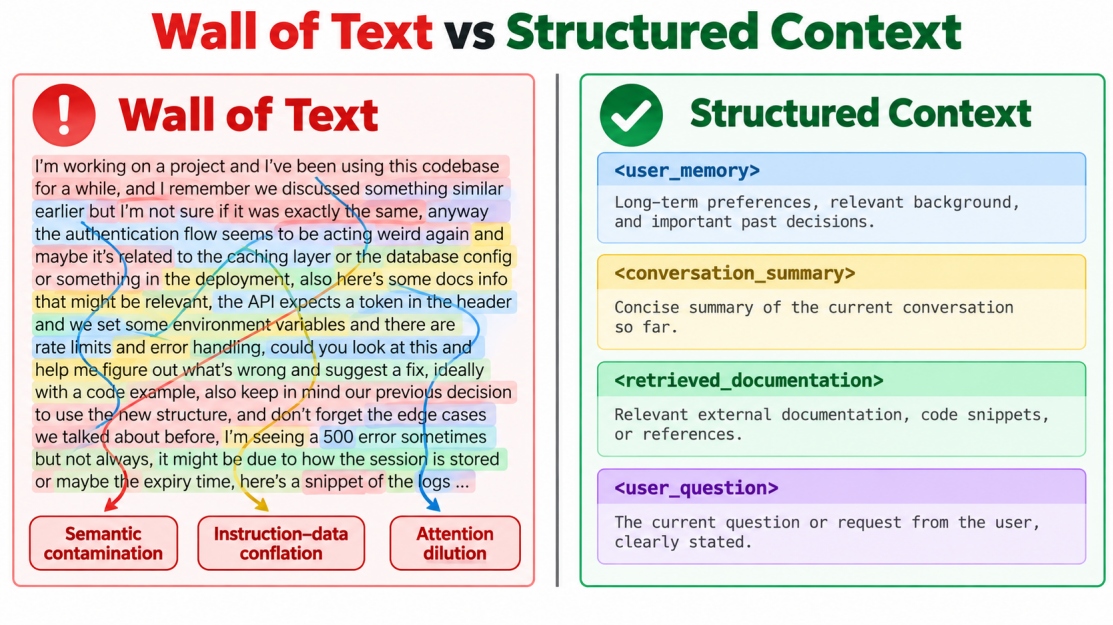

Figure 13. A wall of text causes semantic contamination, instruction-data conflation, and attention dilution, while structured context creates clear boundaries.

The Solution: Delimiters

The fix is to **partition** the content instead of writing it as one continuous stream — the same way a book is broken into chapters, headers, and call-out boxes rather than one unbroken page of text.

> 🧠 **Analogy — A Novel, Not a Single Paragraph**
> A 100-page novel isn't written as one giant paragraph — it's split into chapters, each with headers, bold text, and visual breaks. The content is the same either way, but the partitioning is what makes it navigable. System instructions and user prompts should be structured the same way.

Delimiters are characters or sequences of characters that mark the boundary between independent pieces of text — a "digital fence" telling the model exactly where one section ends and the next begins.

Types of Delimiters

Type	What It Looks Like	Usage Share
Markdown delimiters	`#`, `##`, `###` (headers), `**bold**`, etc.	~8%

Type	What It Looks Like	Usage Share
XML delimiters (tags)	`<objective> ... </objective>` — a named container with a matching opening and closing tag	~90% (industry standard)

XML tagging is the dominant approach: you open a named container (e.g., `<instructions>`), place only relevant content inside it, and close it with a matching `</instructions>` tag. The name should describe the content it wraps (`<objective>`, `<constraints>`, `<retrieved_documents>`, etc.).

> 💡 **Insight — Why Tags Actually Work on LLMs**
> LLMs are trained on massive amounts of code and markup, so they've learned that tags and structural symbols (`<...>`, `#`, etc.) carry special significance — unlike ordinary prose tokens. When a model encounters a tag, it allocates disproportionate attention to it. This is *why* delimiter prompting measurably reduces attention dilution rather than just looking tidier.

Using delimiters when writing prompts is called **delimiter prompting**. Anthropic, OpenAI, and Google's Gemini documentation all explicitly recommend XML-style tagging for long-context and multi-document prompts — e.g., wrapping multiple source documents in a shared `<documents>` tag, with each individual source nested in its own `<document index="...">` sub-tag along with metadata.

How Delimiters Solve Each Problem

Problem	Fix via Delimiters
Semantic contamination	Each piece of information lives in its own tagged container — content never bleeds into a neighboring section
Instruction/data conflation	Rules go inside an `<instructions>` tag; retrieved or reference text goes inside a `<content>`/`<context>` tag — roles become explicit
Impaired target retrieval	You can instruct the model directly: *"Answer using only the data inside the `<context>` block"*
Pattern recognition failure	Few-shot examples become unambiguous with tags like `<input_1>`, `<output_1>`, `<input_2>`, `<output_2>`
Attention dilution	Tags are recognized (from training on code/markup) as structurally significant, pulling focused attention — this doesn't eliminate dilution entirely, but meaningfully mitigates it

CampusX

Worked Example: Structuring a RAG Chatbot's User Prompt

Scenario: a RAG chatbot for ACME Corporation's HR policies. For a single user turn, four different sources of information need to go into the user prompt: long-term memory, conversation history, retrieved document chunks, and the current query.

1. Long-term memory — pulled from a user-memory dict and wrapped in a `<user_memory>` container, sub-divided by field:

```
memory_block = (
    "<user_memory>\n"
    f"<core_identity>\n"
    f"{user_memory.get('core_identity')}\n"
    f"</core_identity>\n"
    f"<user_preferences>\n"
    f"{user_memory.get('user_preferences')}\n"
    f"</user_preferences>\n"
    f"<past_interactions>\n"
    f"{user_memory.get('past_interactions')}\n"
    f"</past_interactions>\n"
    f"<current_goals>\n"
    f"{user_memory.get('current_goals')}\n"
    f"</current_goals>\n"
    "</user_memory>"
)
```

2. Conversation history — wrapped once as a block; each turn already carries a clear `role` field, so no further nested tagging is needed:

```
history_block =
 ↪  f"<conversation_summary>\n{chat_history}\n</conversation_summary>"
```

3. Retrieved documents — each RAG chunk gets its own numbered, titled container, built by looping over the retrieved `found_chunks` and pulling `id`/`title` from metadata and text from `page_content`:

```
doc_tags = []
for i, doc in enumerate(found_chunks, start=1):
    title = doc.metadata.get("title", "Unknown Title")
    content = doc.page_content
    doc_tags.append(f'<document id="{i}"
 ↪  title="{title}">\n{content}\n</document>')
```

```
docs_block = "<retrieved_documentation>\n" + "\n\n".join(doc_tags) +
 ↪  "\n</retrieved_documentation>"
```

Tagging each chunk with its own `id`/`title` is what directly prevents the semantic-contamination failure described earlier — chunk boundaries stay unambiguous instead of bleeding into each other.

4. Current query — wrapped in its own tag, appended last. All four blocks are joined into the final user prompt:

```
Final_prompt = "\n\n".join([memory_block, history_block, docs_block,
 ↪  f"<user_question>{user_query}</user_question>"])
```

The system instruction then explicitly tells the model what each tag means and how to use it:

> "You will receive four context blocks. `<user_memory>`: employee facts — use to personalize the response. `<conversation_summary>`: recent history — use to maintain continuity. `<retrieved_documentation>`: official ACME policies — **this is your single source of truth**, base all facts entirely on this block. `<user_question>`: the immediate query to answer. If the retrieved documentation doesn't contain the answer, state that plainly rather than guessing."

This makes **targeted retrieval trivial**: the system instruction tells the model exactly which tag to consult for which type of information, instead of relying on it to parse an undifferentiated block. Run against the real HR policy PDF, this structuring produced an accurate, grounded answer — correctly citing the 12-week paid leave entitlement, its STD/bonding-phase breakdown, and FMLA overlap — with no hallucination, because the model was pointed directly at the relevant, isolated content instead of having to infer boundaries itself.

Takeaway

Writing the user prompt section with proper delimiter structuring — rather than as a wall of text — directly prevents context rot, hallucination, and generic/degraded output. This is now standard guidance from major LLM providers (Anthropic, OpenAI, Google) for any prompt involving multiple documents, memory, history, or long context.

4.1 Golden Rules for This Session

- **Write system instructions at the right altitude.** Specific enough to be reliable, flexible enough for the model to reason through edge cases — don't hard-code keyword-level rules, and don't leave everything vague.
- **Never send an LLM a wall of text.** Partition every distinct type of information (instructions, data, memory, history, retrieved chunks, query) into its own clearly labeled container, preferably

with XML tags — this is what prevents semantic contamination, instruction/data conflation, impaired retrieval, and attention dilution.

- **Build prompts iteratively.** Start minimal, test, patch only the specific failures observed, and retest — don't try to anticipate every edge case upfront.

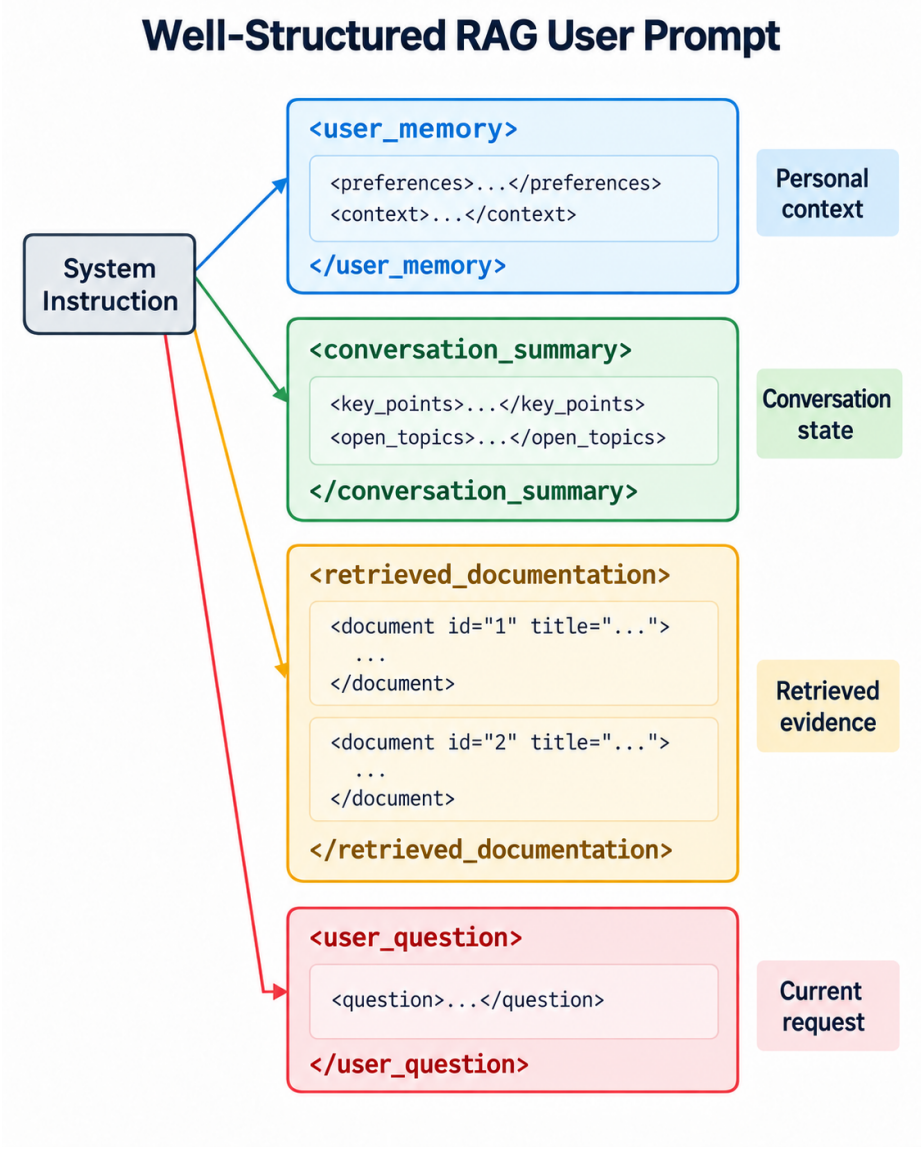

Figure 14. A well-structured RAG user prompt separates memory, conversation state, retrieved evidence, and the current question into explicit tagged blocks.

CampusX

5 KV Cache Part 1

This lecture builds the theoretical foundation for KV caching by first re-deriving how self-attention works internally. A follow-up lecture covers the practical side — writing and managing context so an application actually benefits from KV caching.

> **Definition:** KV cache is a memory optimization technique used in transformer models that dramatically speeds up text generation by reusing previously calculated Key and Value information instead of recomputing it.

This definition will make more sense after walking through self-attention — that's the mechanism KV caching is optimizing.

5.1 Self-Attention: The Foundation

LLMs are built on the Transformer architecture, whose core component is the **self-attention mechanism**.

Definition: Self-attention lets each token (roughly, a word) look at every other token in a sentence and decide which ones matter most for understanding its own meaning.

> 🧠 **Analogy — Tokens Making Friends**
> Think of every token as a person looking to make friends. To do that well, you need to be clear on three things: what kind of friend you're looking for, what you yourself are like, and what you're willing to share about yourself. Tokens go through the same process — each one checks its compatibility with every other token before deciding who matters most to it.

From a Token to Q, K, V

Before a token can "check compatibility," it needs those three pieces of information extracted from it. The extraction pipeline is:

1. **Embedding** — the token (word) is converted into a vector, a mathematical representation, since LLMs operate on numbers, not text.
2. The embedding is passed through **three weight matrices**, learned during training, each producing a different vector:

Matrix	Learns to extract	Output vector	Conceptual role
Query weight matrix	What is this token looking for?	Query vector	What you type into a search bar
Key weight matrix	What are this token's characteristics?	Key vector	The SEO tag / title that answers a search

Matrix	Learns to extract	Output vector	Conceptual role
Value weight matrix	What is this token's actual factual meaning?	Value vector	The real content behind the tag

Example — "it" and "apple":

- The pronoun *it* produces a Query vector effectively asking: *"Where is the singular noun I'm referring to?"*
- The word *apple* produces a Key vector broadcasting: *"I am a noun, I am singular."*
- Query and Key can match on grammar alone, but neither tells you what *apple* actually means. That's the job of the **Value vector**: it ignores grammar and captures the real meaning — e.g., "apple" as a fruit vs. "Apple" (as in Apple Watch) as an electronic device. The same word can produce different Key/Value content depending on context (compare *I ate an apple* vs. *I bought an Apple Watch*).

How Tokens Actually Connect

For a given token, its **Query vector** is compared against the **Key vector** of every other token using a **dot product**. This produces a compatibility score per pair:

$$\text{score} = Q \cdot K$$

- A high score (e.g., 0.9) means strong compatibility — e.g., *it* scores highest against *apple* because *apple* is the singular noun *it* is searching for.
- This score is then used to weight (multiply against) the matching token's **Value vector**, which is how information actually gets shared between the two tokens.

$$\text{output} = \text{score} \times V$$

Once this is done, *it* doesn't just know it's grammatically linked to *apple* — it now carries the actual meaning: *apple is a fruit, it is red, it is nutritious*, etc. This full cycle (Q·K → score → score×V) is self-attention.

Every token in the sentence runs through this same process. So a sentence with 1,000 tokens triggers this full compatibility check 1,000 times over.

CampusX

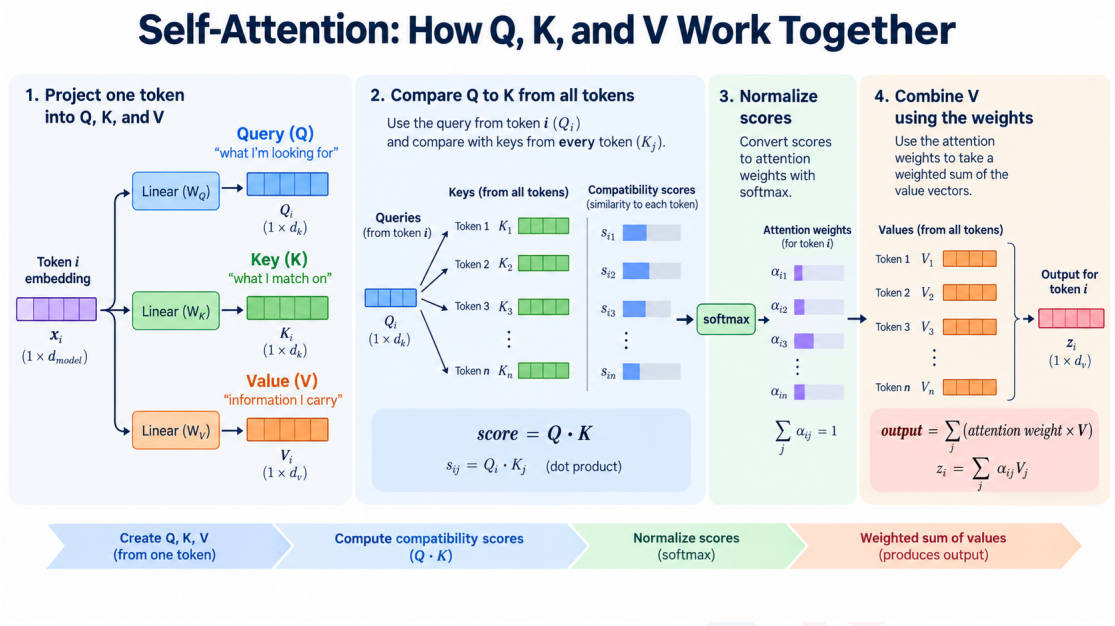

Figure 15. Self-attention projects a token into query, key, and value vectors, compares queries with keys, normalizes the scores, and combines the values.

5.2 Token-by-Token Generation and the Inefficiency Problem

LLMs generate output one token at a time. To predict the next token, the model must first build an "understanding" of everything generated so far — which means running self-attention (embedding → Q, K, V → dot product → weighted value) across *all* prior tokens, every single time.

Example: Generating an essay on photosynthesis token by token — *"Photosynthesis is a process in which…"* — at each new token, the model recomputes Q, K, and V for **every previous token**, not just the new one.

The problem: LLMs don't retain any memory of past calculations between prediction steps — they forget everything after each token is generated. So predicting the 100th token means recalculating Q, K, and V for tokens 1 through 99 all over again, even though the relationship between those earlier tokens (e.g., between "photosynthesis" and "process") hasn't changed at all. This is pure, avoidable repetition.

CampusX

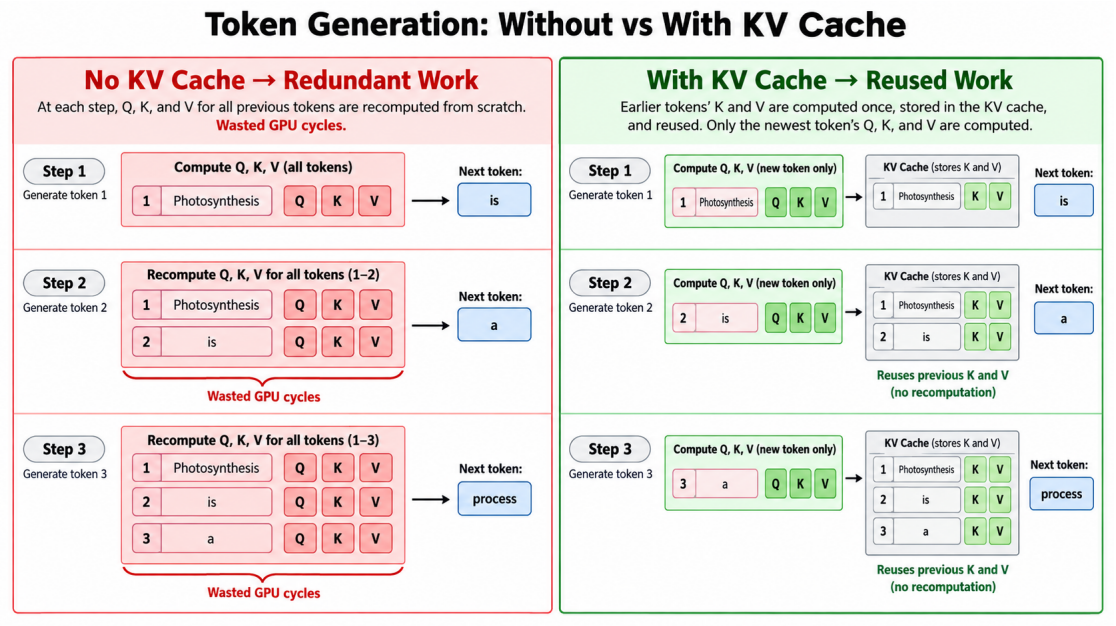

Figure 16. Without a KV cache, earlier token calculations are repeated at every generation step; with a cache, previous keys and values are reused.

5.3 KV Caching: The Solution

Idea: Since the Key and Value vectors of already-processed tokens never change, calculate them once and **store (cache) them**. When predicting a new token, only compute Q, K, V for the *new* token — retrieve everything else from the cache instead of recalculating it.

> 💡 **Insight — What KV Caching Actually Skips**
> KV caching does not skip attention itself — it skips *redundant* attention computation. The Query vector is still computed fresh for the new token (since it depends on what's currently being predicted), but Key and Value vectors for all prior tokens are reused from memory instead of being recomputed.

CampusX

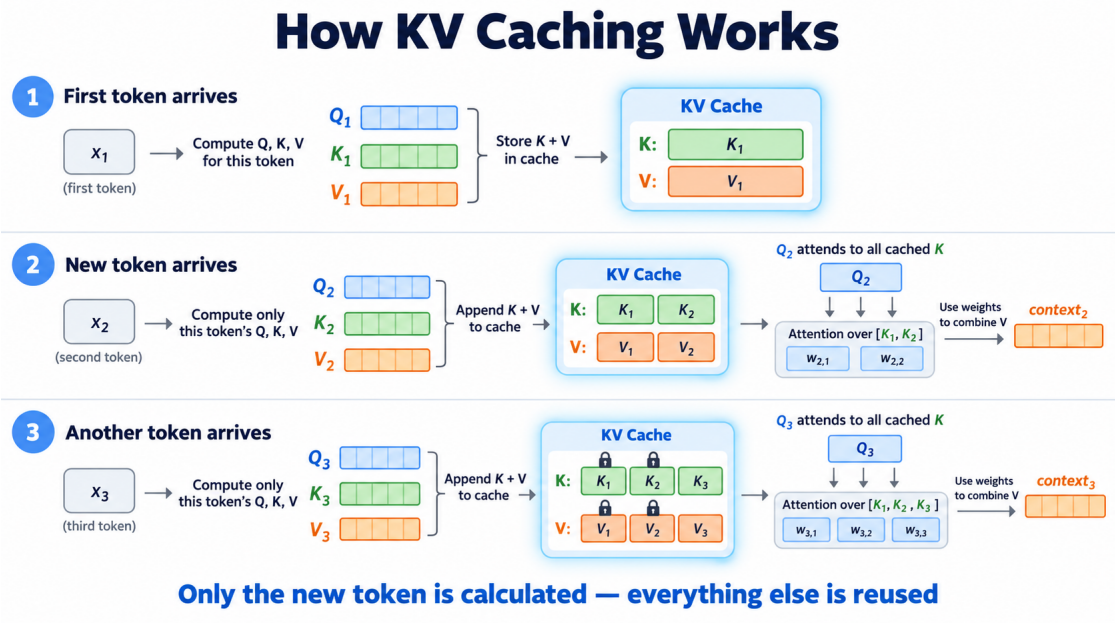

Figure 17. KV caching stores each processed token's key and value vectors, then appends only the new token's vectors while reusing everything already cached.

5.4 Why KV Caching Matters

Benefit	What's happening
Lightning-fast generation	Without caching, generation gets progressively slower as output length grows, since every new token re-triggers full recomputation over all prior tokens. With caching, only the new token's Q/K/V is computed — output speed stays consistently fast.
Massive compute savings	GPUs do the expensive "heavy lifting" of computing K and V once per token, then reuse it, instead of repeating the same matrix operations for every subsequent token.
Lower cost and electricity	Fewer GPU computations means less power draw and lower infrastructure cost — a major reason many AI tools can be offered cheaply or for free.
Scalability — serving many users at once	Reducing per-request computation frees up GPU capacity to serve more concurrent users at the same hardware cost.

CampusX

Reading long documents fast	The delay you notice on a *first* query against a large PDF (all tokens processed fresh) versus the near-instant response on *follow-up* queries is KV caching at work — the document's Key/Value vectors are cached after the first pass.

> 🧠 **Analogy — Building vs. Parking a Car**
> Recomputing K and V for old tokens every time is like building a brand-new car every time you want to drive to the store. KV caching is keeping the car parked in the garage (the cache), ready to drive whenever you need it.

> ⚠ **Mentor's Rule — Design Applications to Be KV-Cache-Friendly**
> There's strong industry expectation that applications manage their context in a way that enables KV caching — either automatically or through deliberate context engineering. The performance and cost benefits are significant enough that this shouldn't be left to chance.

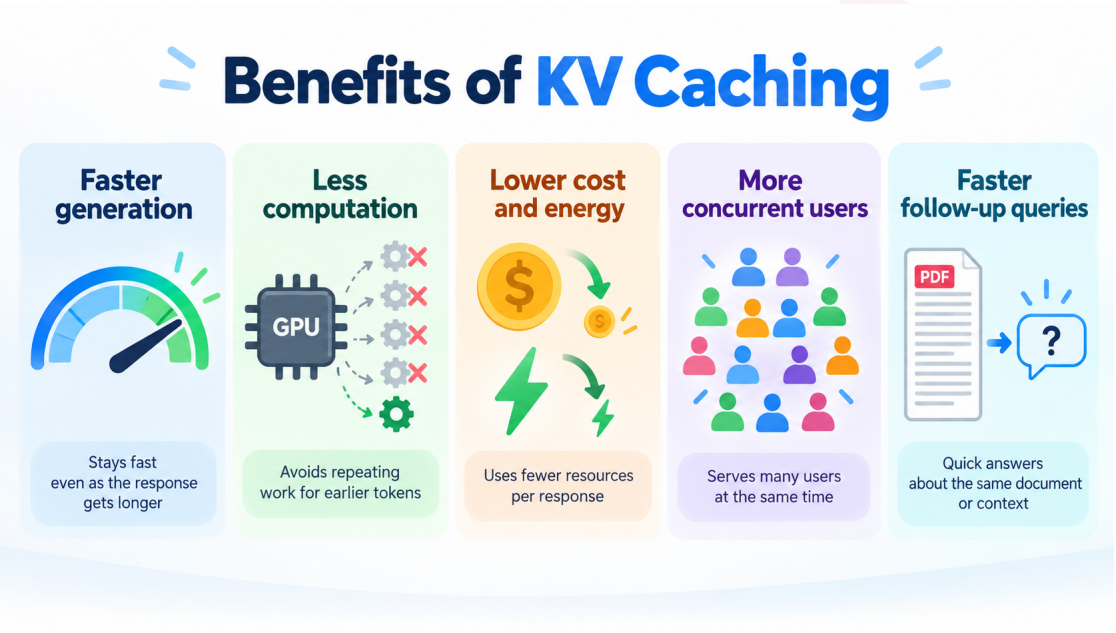

Figure 18. KV caching improves generation speed, reduces computation, cost, and energy use, supports more concurrent users, and accelerates follow-up queries.

6 KV Cache Part 2

> 🧠 **Analogy — Reading Chapter 11 of a Novel**
> When you're reading chapter 11 of a novel, you don't reread chapters 1–10 to follow the plot — those are already memorized. KV caching works the same way: the relationships between earlier words are calculated once and stored, not recomputed for every new word.

The raw Transformer architecture, by itself, has no memory — it only calculates Q, K, V vectors and attention scores; it doesn't store anything. But engineers build system-level infrastructure *around* the transformer that captures and stores these calculations. This is why **most modern LLMs come with KV caching automatically enabled** — you don't have to do anything special to get the basic benefit.

6.1 How a Prompt Actually Reaches the LLM

Before self-attention even runs, your request goes through a pipeline:

1. **You write code** (using a library like Ollama, Gemini, Anthropic, LangChain) defining the system instruction, tool descriptions, and user prompt.
2. **The request passes through a chat template** — a set of formatting rules specific to each LLM (e.g., Qwen, LLaMA). The template wraps your content in special tags (like `<|im_start|>`) and arranges it in a fixed order: system instruction → tool schema/parameters → conversation.
3. **The formatted request reaches the LLM**, which runs self-attention (Q, K, V vectors, attention scores) to process it.

This computation happens on **GPUs** for locally hosted models, or on **High Bandwidth Memory (HBM)** for cloud-hosted models (OpenAI, Gemini, Anthropic) — HBM functions similarly to a very fast GPU for this purpose.

6.2 Why Caching Happens Automatically

Whatever gets calculated on the GPU/HBM is temporarily stored in memory and reused for subsequent generation steps:

- When generating the **first** output token, the LLM calculates Q, K, V for the entire input prompt.
- For the **next** token, it does **not** recompute the whole prompt — it reuses everything already calculated and only computes the new token's vectors.
- In a **multi-turn conversation**, the same principle applies across turns: the calculations for all prior messages are reused, and only the newest user message is freshly processed.

This is exactly the KV cache mechanism from the theory lecture, just happening transparently inside the LLM's infrastructure.

CampusX

6.3 Cache Breaking

A KV cache is **strictly sequential and positional** — it depends on the exact content up to a given point in the input.

Rule: If even a single character changes anywhere in the previously cached sequence (e.g., a comma becomes a full stop in the system instruction), the cache breaks at that exact point. Everything from that point onward is discarded and recalculated from scratch — the cache built for the part before the change stays valid, but nothing after it survives.

This single fact drives all the context-engineering practices below.

6.4 Context Engineering for Implicit (Automatic) Caching

As a context engineer, two structural decisions determine whether automatic caching actually helps you:

Order your user prompt section deliberately

Position	Content type	Examples
Top	Persistent / static information — never changes turn to turn	Long-term memory, company foundational knowledge, core instructions
Bottom	Dynamic information — changes every turn	Newly retrieved RAG context, conversation history, the current user query

Why this order matters: the LLM caches sequentially from the top. If dynamic content (e.g., a freshly retrieved document) sits at the top, the cache breaks at the very start of the prompt on *every single turn* — none of the persistent information downstream gets to benefit from caching either, even though it never changed.

Keep the system instruction strictly static

Most chat templates place the system instruction at the very top of the request — ahead of everything else — specifically so the model pays strong attention to it (related to the "lost in the middle" phenomenon in LLMs). Because it sits first, **any change inside it breaks the cache for the entire request**, no matter how well the rest of the prompt is structured.

CampusX

⚠ Mentor's Rule — Two Things to Get Right as a Context Engineer

1. Persistent information always goes at the top of the user prompt; dynamic information always goes at the bottom.
2. Never inject dynamic variables into the system instruction. If a variable there must change, it should change once per conversation (e.g., at the start), never per turn — and tool schemas should likewise stay stable per conversation rather than changing per request.

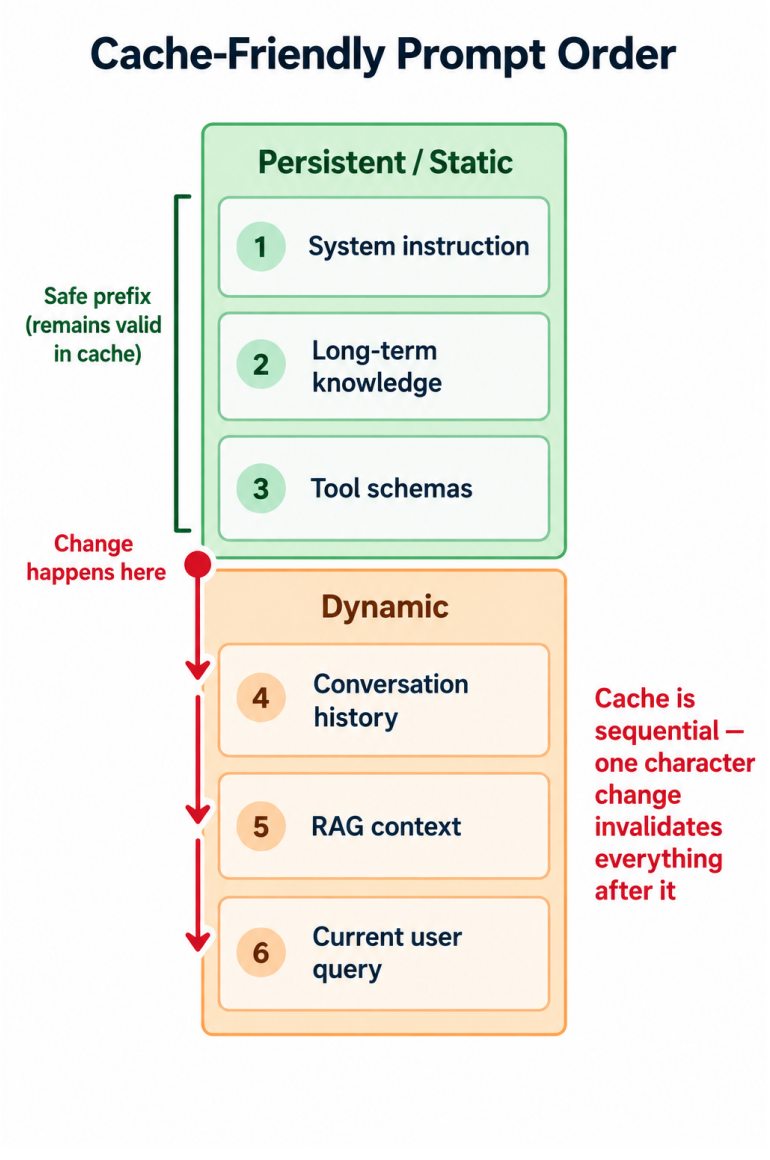

Figure 19. A cache-friendly prompt keeps the stable prefix first and places changing conversation, retrieval, and query content afterward.

You can inspect any open-source model's exact chat template (e.g., via Ollama's `/show template` command, or on Hugging Face) to see precisely where system instructions, tools, and parameters sit relative to the dynamic conversation.

6.5 Implicit vs. Explicit Caching

Every provider implements automatic ("implicit") caching differently, so **always check the specific documentation** for the model you're using.

Provider	Implicit caching behavior
OpenAI	Automatic once the input window exceeds ~1024 tokens
Gemini	Automatic once a model-specific token threshold is hit (e.g., ~2048 tokens on 2.5 Flash, ~4096 on some other models)
Anthropic	Not fully automatic — requires setting a `cache_control` parameter in the API call to enable caching

> 💡 **Insight — Implicit Caching Isn't Guaranteed**
> Gemini's own documentation states it cannot guarantee caching will trigger every time. A live multi-turn test confirmed this: caching worked on turn 3, but failed to trigger on turns 4, 5, and 6 — even with over 11,000 tokens of context already accumulated. If you need to *guarantee* cost and latency savings, don't rely on implicit caching alone.

Explicit (Manual) Caching

Explicit caching means you deliberately tell the LLM provider what to cache, rather than hoping the automatic threshold triggers reliably. The workflow:

1. **Create an LLM client instance.**
2. **Identify the piece of information to cache** — this can be a document, image, video, or system instruction; anything static that will be reused across many calls.
3. **Upload that content to the provider's server**, then explicitly instruct the LLM to process and cache it (e.g., via a `cache.create` call), specifying:
   - the **model** to use for the caching computation,
   - the **content** (system instruction, document, etc.) to cache,
   - a **TTL (Time-To-Live)** — how long the cache should remain stored (e.g., 360 seconds).
4. **Reference the cache** (by name/ID) in subsequent requests instead of resending the full content.
5. **Delete the cache once you're done with it**, since providers charge for cache storage — leaving unused caches around incurs unnecessary cost.

Caching itself is billed, but typically at a much lower per-token rate than fresh input tokens (in one example, cached-token cost was roughly 10× cheaper than input-token cost on the same model) — plus it avoids the compute cost of processing those tokens again.

Measured Impact of Explicit Caching (120-page PDF example)

Metric	Without caching	With explicit caching
Response time	~48 seconds	~14 seconds (first cached call), ~9 seconds (subsequent calls)
Input tokens billed at full input rate	~31,481	~10 (only the new prompt tokens)
Cached tokens	0	~31,486 (reused, billed at the much lower cache rate)
New Q/K/V computation required	Entire document	Only the new prompt tokens

Note: the no-cache run used a slightly different model variant than the cached run, so treat these numbers as directionally illustrative rather than a strictly controlled benchmark.

The same document, reused across multiple questions, only needed fresh computation for each new question — the document itself was processed and cached exactly once.

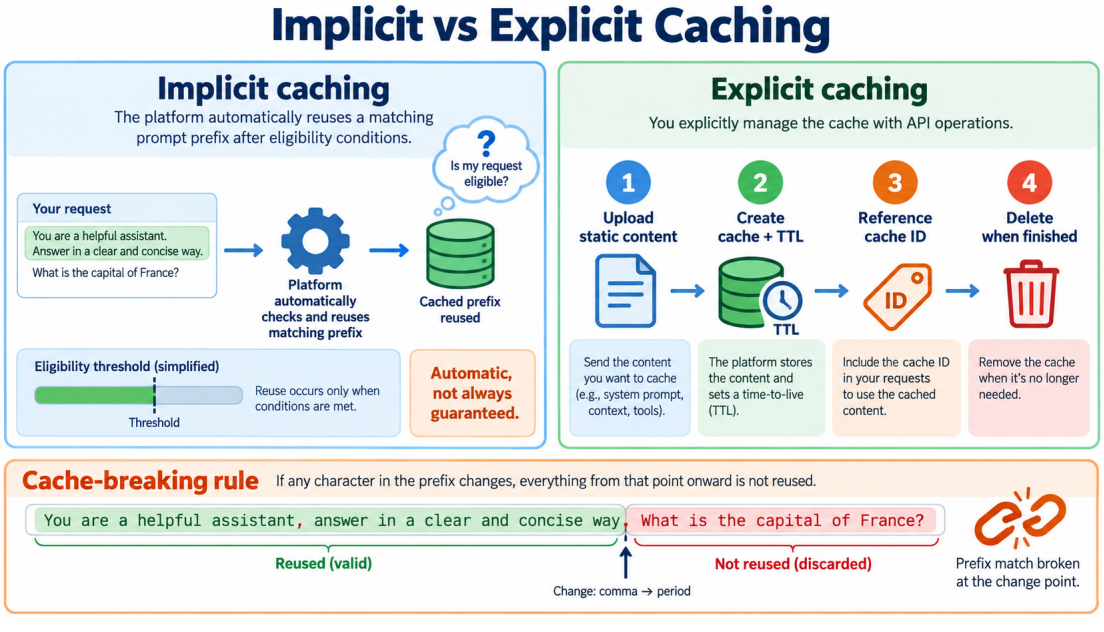

Figure 20. Implicit caching reuses eligible matching prefixes automatically, while explicit caching uploads static content, creates a cache with a TTL, references its ID, and deletes it when finished.

6.6 Recap

KV cache is a memory optimization technique used in transformer models that speeds up text generation by storing previously calculated Key and Value vectors instead of recomputing them for every new token — cutting redundant computation, cost, and latency, while improving scalability and long-document handling.

KV caching is enabled automatically inside most LLMs because engineers built storage infrastructure around the otherwise stateless transformer — but the cache only helps you if your prompt structure protects it. Keep persistent content at the top of the prompt and system instruction fully static, keep dynamic content at the bottom, and check your provider's documentation to know whether you're relying on implicit caching or should switch to explicit caching for guaranteed savings. Done well, this reduces cost, cuts latency, and lowers compute load — the practical payoff of the KV cache theory covered in the previous lecture.

6.7 Golden Rules — KV Caching & Context Engineering

- **Cache reuse only works on unchanged content, in order.** A cache is a sequential record — one character changed mid-sequence (even a comma) invalidates everything after that point, so structure prompts with this fragility in mind.
- **Persistent information goes first, dynamic information goes last.** This single ordering decision determines whether implicit caching helps or effectively does nothing.
- **System instructions must be static per conversation.** They sit at the very top of nearly every chat template, so any change there breaks the cache for the whole request.
- **Don't assume automatic caching is guaranteed.** Providers differ, and even generous context windows can silently fail to cache on a given turn — check documentation, and use explicit/manual caching when savings must be guaranteed.
- **Clean up what you cache.** Explicit caches are billed for as long as they're stored — delete them once no longer needed.
- **Understand *why* caching exists (redundant attention recomputation), not just *how* to call a caching API.** Providers and APIs will keep changing; the underlying inefficiency they're solving won't.

CampusX

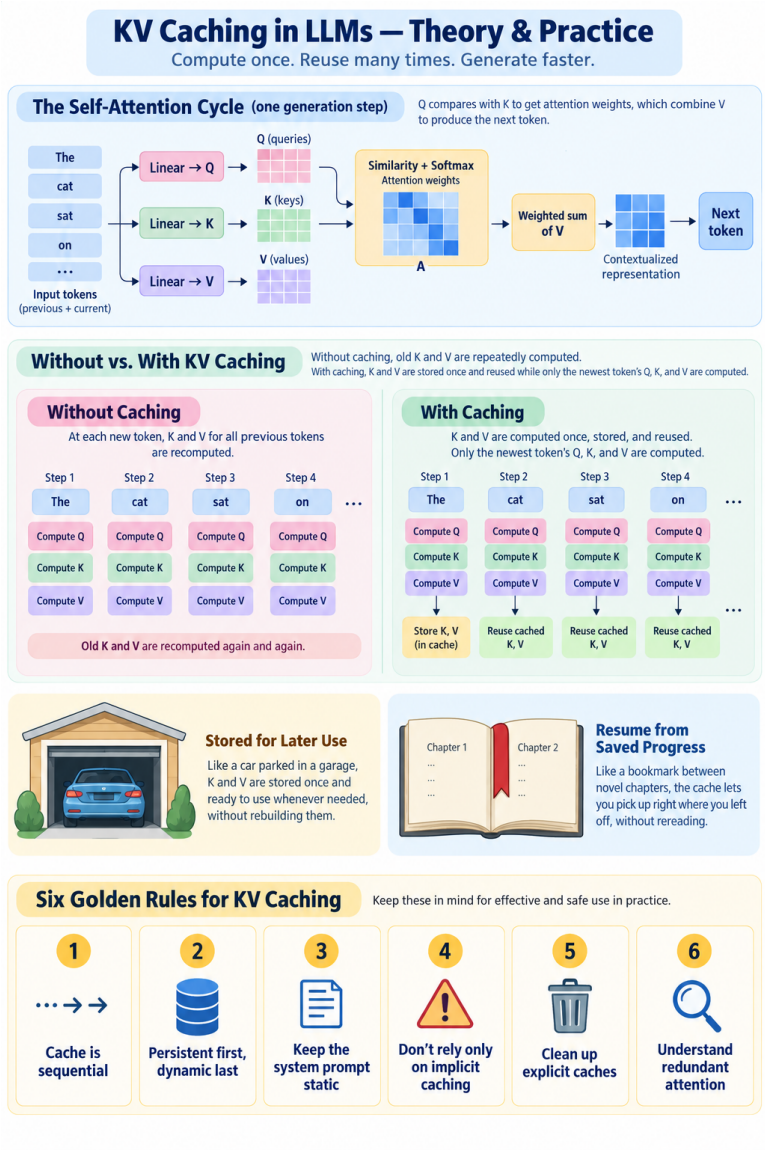

Figure 21. Summary of KV caching theory and practice, from self-attention and reuse of key-value vectors to cache-friendly prompt design and operational rules.

COURSE MODULE

Retrieval-Augmented Generation (RAG)

7 RAG Intro

7.1 From Static Context to Dynamic Context

So far, context engineering has dealt with **static context** — elements that stay constant across every query in an application:

- **System instructions** — fixed persona/behaviour rules for the app.
- **KV caching** — an efficiency technique that only works *because* the cached portion of the prompt doesn't change between calls.

Static context is good at giving a model a consistent personality and operating rules, but it cannot help when a query depends on information the model was never trained on. That's where **dynamic context** comes in: instead of keeping the entire context window fixed, you construct part of it *on the fly*, differently for every query.

Aspect	Static Context	Dynamic Context
Definition	Constant across all queries/sessions	Injected into the context window specifically for a given query
Primary tools	System instructions, KV caching	RAG, vector databases
Role	Defines persona, boundaries, baseline behaviour	Supplies on-demand knowledge for a specific question
Limitation	Can't reflect private/updated data	Needs reliable retrieval logic to fetch the *right* data

CampusX

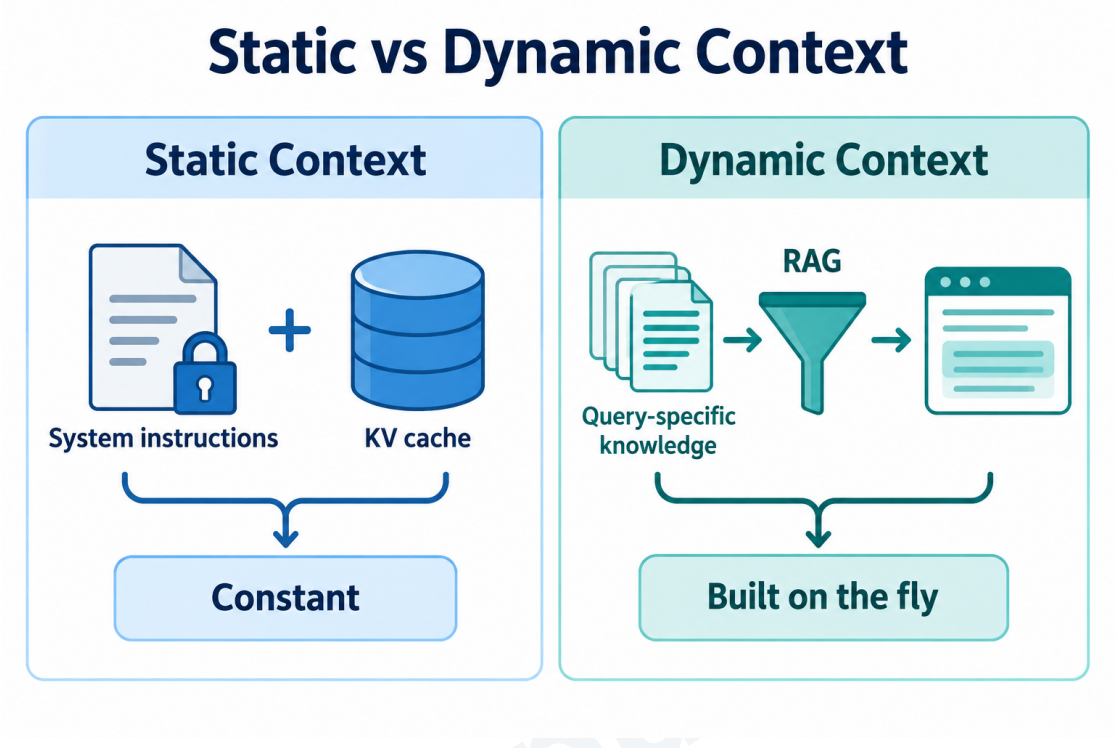

Figure 22. Static context supplies stable instructions, while dynamic context retrieves query-specific knowledge at runtime.

7.2 The Private Data Barrier

LLMs frequently fail on questions like *"How many leave days does my company's HR policy give me?"* or *"Explain this concept from my paid CampusX course."* This isn't a reasoning failure — it's a **data access** failure.

Why this happens:

- The model was trained on a public snapshot of the web, frozen at some cutoff.
- Company HR policies, paid course content, internal Slack threads, etc. are private and were never part of that training data.
- Without an explicit integration/retrieval layer, the model has no bridge to reach this private data at inference time.

The task, then, is to build an application that can answer questions grounded in private data the model has never seen.

7.3 Two Ways to Feed Private Data to an LLM

Strategy 1 — Dump Everything Into the Context Window

Just paste all your private data into the prompt and let the model read it every time.

This breaks down in practice:

- **It often doesn't fit.** A single course can have ~20 hours of content — far more than fits in even a 1M-token window.
- **Even when it fits, quality drops.** If you did manage to squeeze in, say, 950k tokens of a 1M window, you'd still get poor answers, because of the **Lost-in-the-Middle** phenomenon — LLMs pay less attention to information buried in the middle of a very long context, producing generic, low-quality output.

Strategy 2 — Build the Context Window Dynamically (RAG)

Instead of stuffing everything in, you:

1. Store all your data in a database up front.
2. For every user query, search that database for the **relevant** information.
3. Pass only that relevant slice into the LLM's context window.
4. Ask the LLM to generate its answer using that slice as grounding.

Because the injected content changes per query, this is what makes the context window *dynamic*. This is significantly more cost-effective and accurate than brute-force dumping, since the model only ever processes what's actually relevant to the current question.

7.4 What Is RAG?

RAG = Retrieval-Augmented Generation — an AI framework that retrieves relevant information from external knowledge sources and uses it to generate accurate, context-aware responses.

The everyday framing of RAG is "a way to get answers from your private data" — and that's correct, but incomplete.

> 💡 **Insight — RAG Is a Context Management System, Not Just a Q&A Tool**
> Context engineering is about deciding *what* should go into a model's context window. RAG is the operational mechanism that actually *pulls* that content dynamically from an external source. Seen this way, RAG isn't just "chat with your PDF" — it's the system responsible for managing what data enters the LLM's context at request time.

The RAG Flow, Worked Example

Query: *"What is a model file?"* (asked against a 20-hour Ollama course that can't possibly fit in the context window)

1. The course content is broken up and stored in a database beforehand.
2. The user's query is compared against that database to find related content.
3. The database returns the most relevant chunks (e.g., wherever "model file" was discussed across the 20 hours).
4. Those chunks are handed to the LLM with an instruction to generate a good answer using them as context.

That entire loop — search → retrieve → generate — is RAG.

7.5 Building a RAG System: The Six-Step Pipeline

Step 1 — Data Collection

Private data is usually scattered: PDFs, half-filled databases, Slack drafts, internal docs, etc. The first job is consolidating all of it into a single, structured source.

Step 2 — Chunking

Break the consolidated data into smaller pieces (chunk 1, chunk 2, chunk 3, …).

Why chunk instead of feeding the whole source to the LLM directly?

- The context window has a hard token limit — the full source often won't fit.
- Even if it did fit, the Lost-in-the-Middle problem would degrade output quality.

Chunking exists specifically to avoid both problems and to make sure the LLM gets focused, specific information rather than an overwhelming data dump.

Step 3 — Embedding

Embedding converts textual data into numbers (a **vector**), built so that the numbers capture the semantic meaning of the text — not just its literal words.

Example: the sentence *"I have a cat"* becomes a vector like $[0.99, 0.98, 0.97, 0.99, \dots]$.

CampusX

Step 4 — Vector Storage

Store each chunk together with its vector in a **vector store**. These vectors live in an **N-dimensional plane** — think of it as a Cartesian plane, just with far more than 2 or 3 axes (real-world embeddings are commonly 2048 or 4096 dimensions).

- In 2D, a word like "cat" might sit at coordinates $(3.2, 1.5)$.
- In 3D, it might sit at $(1.2, 3.7, 4.9)$.
- A 4096-dimension embedding means the word is represented by 4096 numbers, i.e. a point in a 4096-axis space.

Words with related meaning (e.g. "cat" and "dog") end up positioned close together in this space.

Step 5 — Query Embedding & Semantic Search

1. Convert the user's query into a vector using the same embedding scheme (same dimensionality as the stored chunks).
2. Place that query vector in the same N-D space as the stored chunk vectors.
3. Compute the distance between the query vector and every chunk vector.
4. Retrieve the chunks with the smallest distance (the "nearest neighbours") — these are the most semantically relevant chunks.
5. Map those retrieved vectors back to their original text and return that text.

> 💡 **Insight — Why Embeddings Instead of Direct Text Comparison**
> Comparing the raw query text against raw chunk text word-by-word would be slow, and it breaks on synonyms — e.g. a query using "two-wheeler" would miss a chunk that says "bike." Embeddings encode *meaning*, so semantic search finds relevant chunks even when the wording differs.

Step 6 — LLM Generation

The retrieved chunks are handed to the LLM as context, and the LLM is asked to generate the final answer from them.

Why not just show the raw chunks to the user directly? Retrieved chunks are often fragmented, partially formatted (some in tables, some in plain text, some incomplete) — readability suffers if shown as-is. The LLM's job here is to synthesize these fragments into one coherent, well-written response.

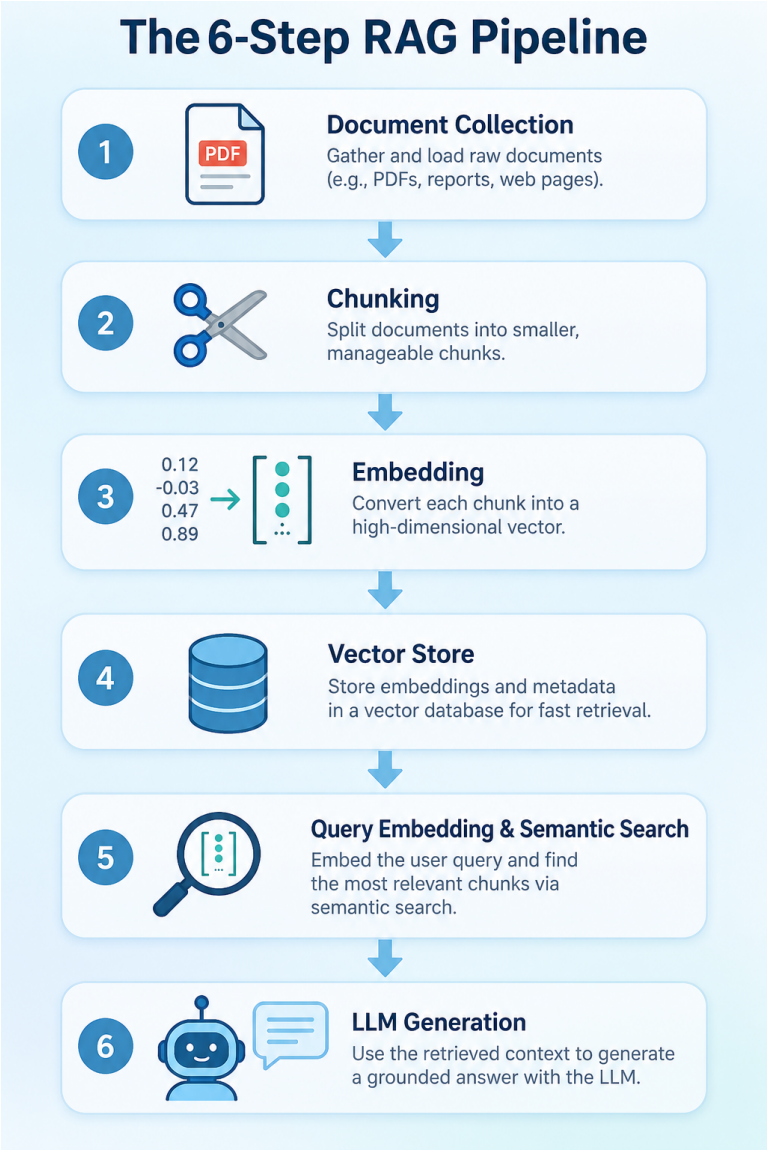

Figure 23. The six-step RAG pipeline loads, chunks, embeds, stores, retrieves, and uses relevant knowledge to generate a grounded answer.

7.6 The Three Pillars of a RAG System

The six steps above collapse into three higher-level pillars:

Pillar	Includes	Role
Database Formulation	Data collection, chunking, embedding, vector storage	Builds the knowledge base

Pillar	Includes	Role
Retrieval	Query embedding, semantic search, distance calculation	Finds the right chunks
LLM Usage	Passing retrieved chunks to the LLM, generating the final answer	Produces the polished output

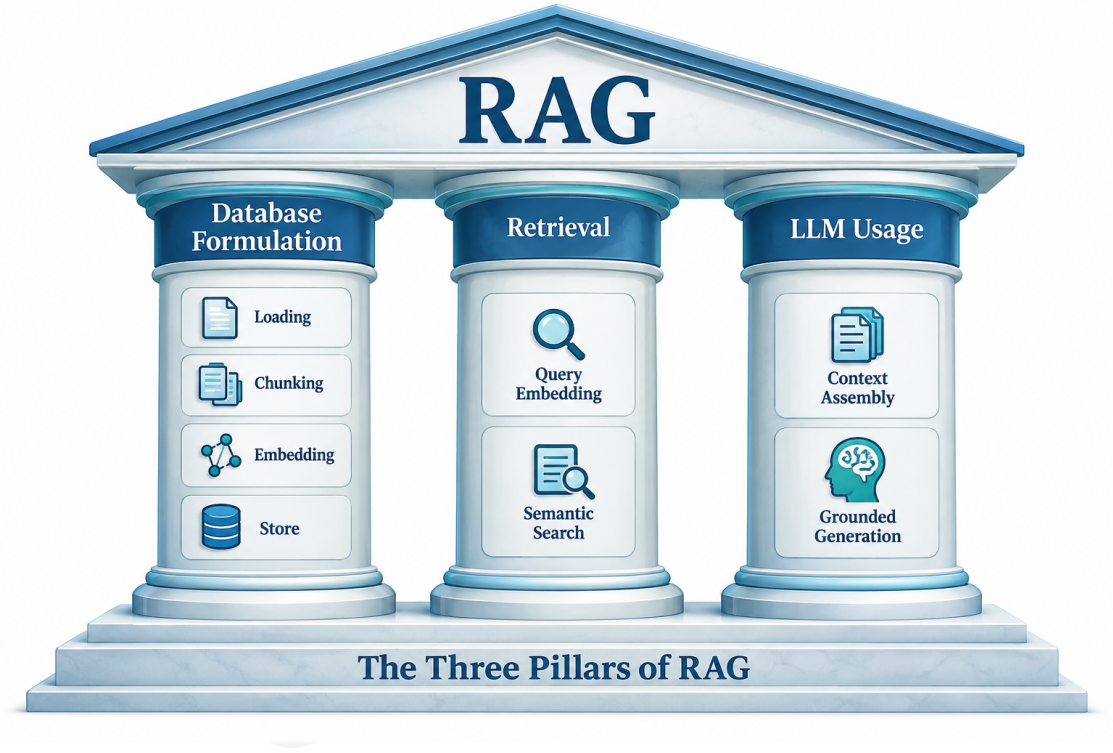

Figure 24. A RAG system rests on three pillars: database formulation, retrieval, and LLM usage.

Why This Course Emphasizes Database Formulation and LLM Usage over Retrieval

- **Database formulation quality caps everything downstream.** A messy, incomplete, unstructured database guarantees poor retrieval, no matter how good the search algorithm is — garbage in, garbage out.
- **Good retrieval can still be wasted.** Even if the right chunks are retrieved, generation quality suffers if those chunks aren't passed to the LLM properly — the model may under-use them or produce a generic answer.
- **Retrieval itself is already widely covered elsewhere** (multiple retrieval strategies, multi-hop retrieval, etc.), whereas database formulation and proper LLM usage rarely get explained in depth. This course will use straightforward **semantic retrieval** and focus its depth on the

other two pillars. Deeper retrieval strategies are covered in a separate Advanced RAG course (CampusX Insider / CampusX One).

7.7 ⚠ Mentor's Rule — Don't Use RAG Everywhere

RAG is not the default choice for every "answer from my data" problem. If your dataset is small relative to the model's context window (e.g., 10k–20k tokens of data against a 200k or 1M-token context window), building a full RAG pipeline adds **unnecessary complexity** — data collection, chunking, embedding, vector storage, query embedding, semantic search, and chunk-passing each introduce their own possible points of failure.

In that situation, the better strategy is:

- Put the entire small dataset directly into the context window.
- Apply **KV caching** to keep repeated calls efficient.

Because the data is small relative to the window, you avoid the Lost-in-the-Middle problem entirely, you never hit context limits, and the LLM can attend fully to every token — often producing *more* accurate output than a RAG pipeline would, since there's no retrieval step that could miss a relevant chunk.

7.8 Anatomy of a Dynamic Context Window

"Dynamic context" doesn't mean the *entire* context window changes per query — only part of it does. Most of an application's context window stays static; only the retrieved information is dynamic.

Element	Static or Dynamic	Notes
System instruction	Static	Fixed persona/behaviour for the whole app
Long-term memory (user history)	Static	Persists across prompts, not query-dependent
Database/schema structure	Static	The underlying knowledge base itself
Retrieved chunks	**Dynamic**	Changes every time, based on the current query

Only the retrieved chunks vary per request — everything else forming the "shell" of the context window remains constant.

7.9 Database Formulation: The Four Steps

Database formulation is the foundational pillar of a RAG system — if the database is poorly built, semantic retrieval will surface the wrong information, and the LLM will generate flawed answers no matter how good the generation step is. Garbage in, garbage out.

Database formulation breaks down into four sequential steps:

1. **Document loading** — ingesting raw external documents into the coding environment.
2. **Chunking** — breaking ingested documents into smaller, manageable pieces.
3. **Embedding** — converting those chunks into vector representations.
4. **Vector store creation** — storing the vectors in a vector database.

This session covers **Step 1: Document Loading**.

8 RAG - Document Loaders

Why It's Needed

Your source data — an HR policy, a course, a report — usually exists *outside* your coding environment: as PDFs, Word files, databases, web pages, HTML, etc. None of that is directly usable by code running in VS Code, PyCharm, or a Jupyter notebook. Document loading is the ingestion step that pulls this external data into your working environment so the rest of the pipeline can process it.

What Are Document Loaders?

Document loaders are the "helping hands" — software components that know how to connect to a specific data source and extract its contents into your application. Because data can live anywhere and in any format, frameworks like LangChain ship a large ecosystem of loaders, each specialized for a source type:

Category	Example Loaders
SaaS / API sources	Airbyte Stripe loader, Airbyte Typeform loader, OneDrive loader
File formats	PyPDF loader, CSV loader, JSON loader
Web sources	Async HTML loader
Specialized	Hugging Face dataset loader, Google Speech-to-Text loader, ChatGPT loader

The workflow is: figure out where your data lives → find the matching loader → use it to pull the data into your workspace.

CampusX

> ⚠ **Mentor's Rule — Always Load Documents in Markdown Format**
> Whichever loader you use, prefer one that loads your file **as Markdown**, not as raw/plain text. This single choice has a major downstream effect on database quality, explained below.

Why Markdown Loading Matters: A Direct Comparison

The same PDF (a book with chapters, sub-chapters, code blocks, and tables) was loaded two ways to see the difference.

Standard parser (e.g. PyPDF) — plain text extraction:

- Only raw characters are extracted; all formatting is discarded.
- **Document structure is destroyed** — chapter/section numbers like "1.1" become plain text with no hierarchy attached; there's no way to programmatically tell what's a heading vs. body text.
- **Code blocks scramble into surrounding text** — no boundary between code and prose.
- **Tables are distorted** — only the raw numbers/characters survive; the row/column structure is gone.
- **Layout awareness is lost entirely** — the output becomes a flat, noisy wall of text.

Markdown-aware parser (e.g. PyMuPDF4LLM) — structured extraction:

- Reads text, tables, images, and layout, then converts the whole document into LLM-friendly Markdown.
- **Hierarchy is preserved**: book title → single `#` heading, chapters → `#`, sub-chapters → `##`, and so on.
- **Code blocks are isolated** in triple-backtick fences, kept distinct from prose.
- **Tables are preserved as proper Markdown tables** — headers and row values stay correctly associated.
- **Full layout awareness is retained**, so the structure of the original document is recoverable from the text alone.

The same contrast holds for web content:

- **Async HTML Loader** alone fetches raw HTML as-is — full of tags, navigation bars, footers, and boilerplate. This is mostly noise that inflates token usage and pollutes the database.
- **HTML-to-Text Transformer** (used after the HTML loader) converts that raw HTML into clean Markdown — headings, bullet points, bold/italic, and code blocks are translated properly, while navigation/footer clutter is stripped out.

CampusX

> 💡 **Insight — A Messy Load Guarantees a Messy Database**
> If chunking and embedding are performed on text where tables are scrambled and code is indistinguishable from prose, no amount of good retrieval logic downstream can recover the lost structure. Document loading quality sets a ceiling on everything that follows.

Why Markdown Specifically Benefits LLM-Based Pipelines

1. **Native LLM alignment** — LLMs are heavily pre-trained on code and Markdown-formatted data, so they're very good at reading it and inferring hierarchy (what's a heading vs. a sub-point vs. body text) once it reaches the context window.
2. **Superior table preservation** — column headers and row values stay correctly linked, so the LLM can answer questions that depend on tabular relationships accurately.
3. **Code block isolation** — fenced code blocks stop the model from confusing code syntax with descriptive text.
4. **Massive noise reduction** — irrelevant layout markup, HTML tags, navigation bars, and footers are filtered out, leaving only high-value semantic content.
5. **Faster parsing** — Markdown files are lighter to parse than heavy PDFs or DOCX files.

Since retrieval quality is only as good as the database it searches, and database quality starts with how well the raw data was ingested, loading documents into Markdown is the single highest-leverage decision in the document loading step.

9 RAG - Contextual Retrieval and Dynamic Splitting

Why Chunking Is Necessary

A **chunk** is simply a small segment of a larger piece of loaded data. Two problems force us to split data into chunks rather than feeding it to the LLM whole:

1. **Context window limits.** A document can easily exceed the model's context window (e.g. a 1M-token PDF against a 200k-token window) — it physically cannot fit.
2. **Lost-in-the-Middle degradation.** Even when a document *does* fit (e.g. 160k tokens inside a 200k window), performance still degrades — the model attends well to the start and end of a long context but loses focus on the middle. Smaller, focused chunks avoid this.

Standard Text Splitters and Why They Fail on Mixed Content

LangChain-style text splitters generally fall into four families:

CampusX

Splitter type	How it splits	Typical use
Length-based	Fixed character/token count	Rarely used alone
Text structure (e.g. Recursive Character)	Sentence/paragraph boundaries, newlines	Plain prose
Document structure (Markdown, HTML, Python, etc.)	Structural markers (headers, tags, class/function boundaries)	Structured single-format documents
Semantic meaning	Embedding-based shifts in meaning	Prose where topic boundaries aren't structurally marked

For plain prose, a **recursive character text splitter** (a text-structure splitter) works well and is the most commonly used default.

The problem: real documents are rarely pure prose. A book with text, Python code, and tables breaks both common approaches:

- **Recursive text splitter** treats everything — including code and tables — as plain text. Code gets cut mid-function (a `def` line in one chunk, its `return` in another), and tables get fragmented, scrambling row/column relationships.
- **Markdown header splitter** correctly preserves the heading hierarchy (chapter/section skeleton), but it *also* has no awareness of code or table boundaries within a section — code still gets split like ordinary text, breaking indentation.

> 💡 **Insight — No Single Splitter Understands Mixed-Content Documents**
> A splitter that's excellent at one data type (prose, code, or tables) is usually the wrong tool for the other types inside the same document. The fix isn't a "smarter" universal splitter — it's routing each data type to the splitter built for it.

Dynamic Chunking: Route Each Data Type to Its Own Splitter

Dynamic chunking means you don't feed an entire mixed document to one splitter. Instead, you identify what *kind* of content each part of the document is, and send each kind to the splitter that specializes in it — the same way a medical camp triages patients to the relevant specialist doctor rather than sending everyone to one generalist.

This is done in two steps:

Step 1 — Partition and label the document. Walk through the document (line by line, in this case a Markdown file) and tag every segment with:

- **Kind**: `prose`, `code`, or `table` (detected via Markdown cues — e.g. triple backticks mark code, a leading `|` marks a table row; anything else defaults to prose).
- **Hierarchy**: which chapter (`#`) and section (`##`) the segment belongs to, tracked by counting hash characters as the parser moves through the document.

- **Text**: the actual raw content of that segment.

Adjacent segments that share the same kind, chapter, and section are then **merged** — this avoids artificially fragmenting a single block of prose just because it was split across lines during parsing.

Step 2 — Build one splitter per data kind, then route.

- **Prose** → a Recursive Character Text Splitter (tuned with an appropriate chunk size, overlap, and separators).
- **Code** → a language-aware code splitter (e.g. a Python-specific splitter) that understands syntax, so functions and indentation stay intact instead of being cut mid-block.
- **Tables** → often left unsplit. If a table is already small (a handful of rows), splitting it further destroys the row/column relationships and loses information; only very large tables need a dedicated table splitter.

Each partitioned segment is routed to its matching splitter, and the resulting chunks retain their original metadata (kind, chapter, section) alongside the split text.

Reference defaults (a reasonable starting point to tune from):

Splitter	Key parameters
Prose (Recursive Character)	`chunk_size=500`, `chunk_overlap=60`, `separators=["\n\n", "\n", ". ", " ", ""]`
Code (Python-aware Recursive Character)	`chunk_size=1500`, `chunk_overlap=0`, language set to Python so splits respect function/class boundaries
Table	left unsplit for small tables (a handful of rows); only split very large tables with a dedicated table splitter

CampusX

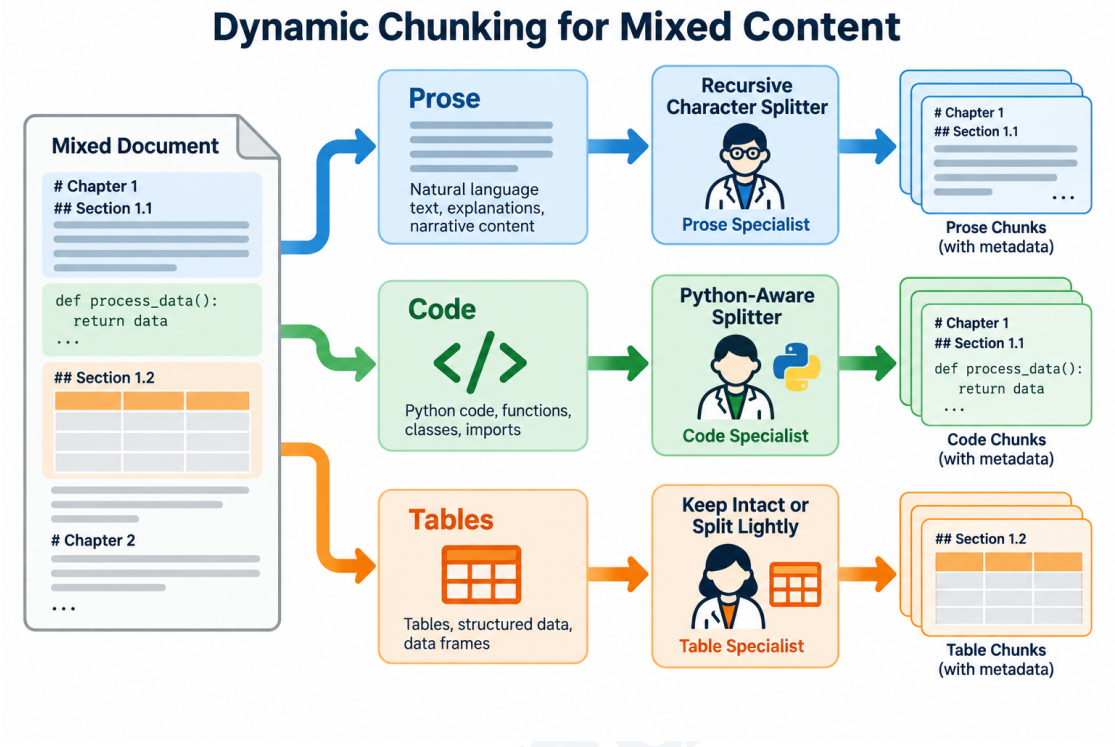

Figure 25. Dynamic chunking partitions mixed-content documents and routes prose, code, and tables to appropriate splitters while preserving metadata.

9.1 The Amnesia Problem

Once a document is broken into chunks, each chunk becomes a **standalone unit** with no awareness of the rest of the document. This causes what's called **amnesia**: a chunk can contain the answer to a query but never get retrieved, because it lacks the keywords needed to match that query.

Example:

- Chunk A: *"Revenue for ABC company in Q1 was 40 Cr."*
- Chunk B: *"Revenue for the same company grew by 30% in the next quarter."*

Chunk B never mentions "ABC" or "Q2" — it only makes sense in light of Chunk A. A query like *"What was ABC's Q2 revenue?"* is unlikely to retrieve Chunk B via semantic search, even though it holds the answer, because the chunk's own text doesn't share vocabulary with the query.

9.2 Contextual Retrieval (Anthropic's Technique)

Contextual retrieval solves the amnesia problem by permanently attaching a short, descriptive summary to each chunk *before* it's embedded and stored — situating the chunk within the document as a whole.

How it works:

1. For each chunk, send the **entire source document** plus that **specific chunk** to a small, cheap LLM.
2. Prompt it to produce a short, succinct context describing what role this chunk plays in the overall document — just enough to "situate" it (e.g. which company, which chapter/topic, which time period it refers to).
3. **Prepend** that generated summary to the original chunk text to form an **enriched chunk**: `summary + original chunk text`. The summary must be *prepended*, not appended.

> 🧠 **Analogy — The Sticky Note**
> An enriched chunk works like a photo captioned with a sticky note. The photo alone might just say "it was a huge success" — meaningless on its own. A sticky note on it saying "this refers to the 2007 iPhone launch" gives the photo enough context to be understood and found again, without needing to attach the entire surrounding article.

Applied to the earlier example, Chunk B's summary might read: *"This chunk refers to ABC company and its Q2 revenue."* The enriched chunk — summary plus original text — now shares vocabulary with queries like "ABC Q2 revenue," making it retrievable.

This is **not** summarizing the whole document — only a short, chunk-specific context is generated and attached, keeping the chunk itself intact.

> ⚠ **Mentor's Rule — Always Prepend the Summary, Never Append It**
> Anthropic's own documentation on contextual retrieval specifies that the generated context must go *before* the chunk, not after. Placement matters for how the enriched chunk reads and embeds.

9.3 Chunking: The Two Things to Get Right

Putting it together, quality chunking for real-world (multi-format) documents requires two techniques:

1. **Dynamic chunking** — partition the document by content type (prose/code/table), preserve hierarchy metadata, and route each part to the splitter built for it.
2. **Contextual retrieval** — enrich every resulting chunk with a short, prepended summary so it remains meaningful and retrievable in isolation.

Together these produce chunks that preserve document structure *and* carry enough standalone context to be found by semantic search — setting up the next steps of embedding and vector store creation.

10 RAG - Complete Flow

What Gets Embedded

An **embedding** is a numerical (vector) representation of text that captures its *semantic* meaning, not just its literal wording. Because of this, two differently worded but semantically similar phrases — e.g. "I drive a car" and "I operate a four-wheeler" — end up with nearly identical embedding vectors, since "car" and "four-wheeler" mean the same thing.

Embeddings are typically **4096-dimensional** — a practical sweet spot. Larger dimensions capture finer semantic nuance, but with diminishing returns against the extra computational cost of distance calculations across a large vector store; 4096 balances semantic fidelity against retrieval speed.

Crucially: embed only the enriched text, never the full chunk object with its metadata. The enriched text (contextual summary + original chunk text, from contextual retrieval) is what should be converted into a vector. Fields like `kind`, `chapter`, or `section` are metadata used for filtering/display, not semantic content — they're stored alongside the embedding, not embedded themselves.

Embedding Models

Embedding models are a distinct category of model — their only job is converting text into vectors, unlike general-purpose LLMs that generate text or code. Every major provider offers one:

Provider	Example embedding models
Ollama (open-source)	Qwen 3 Embedding (8B), Nomic Embed Text v2
Google	Gemini Embedding 001
OpenAI	Text-embedding-3-small, Text-embedding-3-large

CampusX

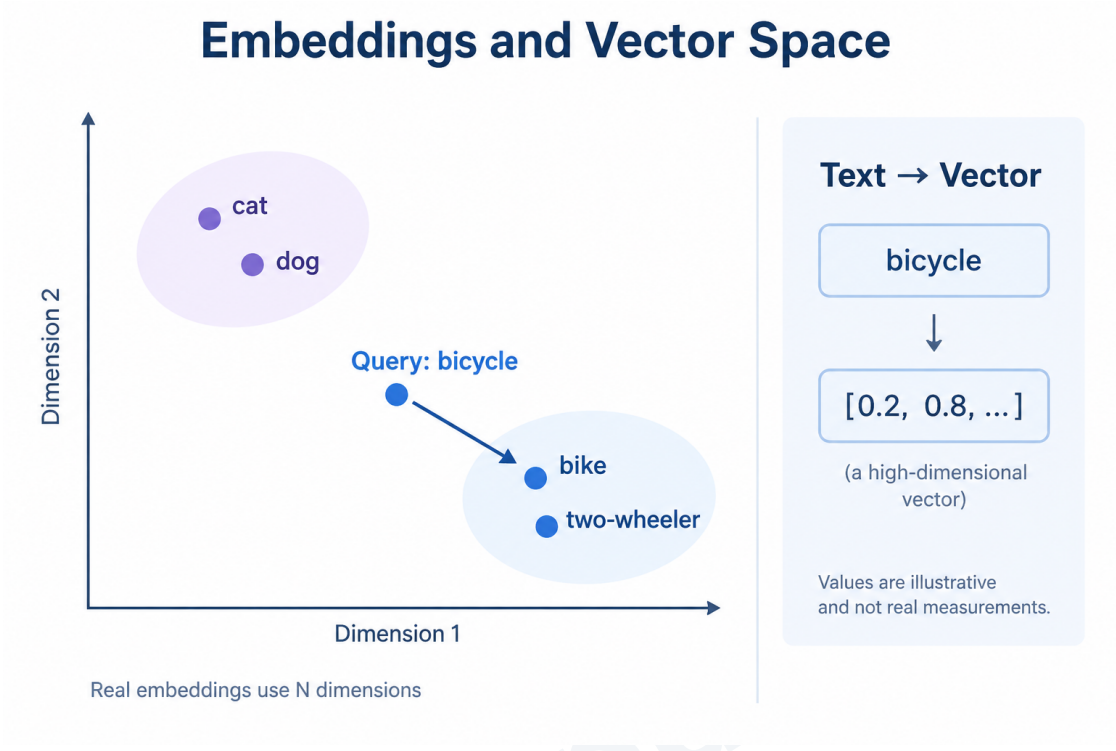

Figure 26. Embedding models map semantically similar text to nearby positions in vector space, enabling meaning-based retrieval.

Finding one is simple: search `"<provider> embedding model"`.

10.1 Step 4: Vector Store Creation

A **vector store** is a database purpose-built to hold embeddings (vectors) alongside their source text and metadata, and to run similarity search over them efficiently. Common options include **ChromaDB**, **FAISS**, and **Pinecone** — each with its own tradeoffs (local vs. managed, scale, speed), covered in more depth in a dedicated Advanced RAG course.

Practical setup: extend each chunk's existing record (which already holds `kind`, `chapter`, `section`, original text, contextual summary, and enriched text) with one more field — its embedding vector — then insert the whole record into the vector store in one step. Each record in the store typically holds:

- A unique **ID**
- The **embedding** (the numerical vector)
- The **metadata** (kind, chapter, section)
- The **original text** (ground truth, *not* the enriched version — kept for later use during generation)

CampusX

10.2 Retrieval

Retrieval is the process of taking a user's query, searching the vector store, and returning the most relevant chunks.

The workflow:

1. Convert the user's query into an embedding, **using the exact same embedding model** used to build the database. (If the models differ, the query vector won't align with the stored vectors, and retrieval breaks entirely.)
2. Place that query vector into the same vector space as the stored chunk vectors.
3. Calculate the distance between the query vector and every stored vector.
4. Return the **top N** nearest vectors (chunks) — the ones with the smallest distance, i.e. the most semantically similar to the query.

> ⚠ **Mentor's Rule — Always Embed the Query With the Same Model Used for the Database**
> A mismatched embedding model between query time and database-build time silently breaks retrieval — the two vector spaces won't correspond, so "nearest neighbor" distances become meaningless.

Choosing N (How Many Chunks to Retrieve)

There's no fixed formula — the right number of chunks to retrieve (commonly 5 as a starting point) is generally settled by hit-and-trial: enough to cover multi-part questions, but not so many that irrelevant chunks dilute the context.

> 💡 **Insight — Retrieval Quality Is a Hard Ceiling**
> If the correct chunk is never retrieved, no amount of prompt engineering at the generation step can recover the answer. This is why database formulation and retrieval tuning deserve more attention than polishing the final prompt — a perfect system prompt can't compensate for the right information never reaching the context window.

Structuring Retrieved Chunks for the LLM

Raw retrieval output (IDs, documents, and metadata in separate structures) needs to be reassembled into one clean, delimited block per chunk before it goes to the LLM — e.g., wrapping each result as:

```
[DOCUMENT ID 1]
Source: Chapter 1, Section 1.1
<chunk text>
[/DOCUMENT ID 1]
```

This gives the LLM clear boundaries between sources, along with the metadata needed to cite them.

> 💡 **Insight — Retrieve With the Enriched Text, Generate With the Original**
> The enriched text (with its contextual summary) exists purely to make a chunk *findable* — it makes the embedding step aware of what the chunk means in the context of the whole document. But that summary is not meant for the LLM's eyes during answer generation: showing it the contextual summary alongside the real content risks distracting or confusing the final output. So: **embed and search using the enriched text, but hand the LLM only the original, un-enriched chunk content** once it's been retrieved.

10.3 Passing Retrieved Chunks to the LLM

Two structural rules govern how retrieved context should be fed into the LLM's prompt:

1. **Delimiter-structure everything.** Wrap the whole retrieval block in a clear tag (e.g. `<retrieved_documentation> ... </retrieved_documentation>`) and wrap the user's query in its own tag (e.g. `<user_query> ... </user_query>`).
2. **Place retrieved context and the user query at the very end of the prompt** — after system instructions, tool definitions, and conversation history. This combats the Lost-in-the-Middle phenomenon, since the most recent, most relevant content sits where the model attends to it most strongly.

System Prompt Design for a RAG Assistant

A robust RAG system prompt should specify:

- The two expected input tags (retrieved documentation, user query).
- **Strict grounding**: answer *only* from the retrieved documentation — never rely on external/pre-trained knowledge.
- **An explicit "ignorance clause"**: if the answer isn't present in the retrieved documentation, the model must say so plainly (e.g. *"I cannot answer the question based on the provided content"*) rather than guessing or hallucinating.
- **A citation requirement**: instruct the model to cite the source (chapter/section) for the claims in its answer, directly in the response — not just log metadata on the side.
- Response guidelines: read the question carefully, scan the retrieved documents thoroughly, and produce a concise, well-formatted answer.

Why Keeping Source Metadata Matters

Passing chapter/section metadata through to the final answer (e.g. citing *"From Chapter 1, Section 1.1: Byte Pair Encoding"*) lets the end user verify the answer against the original source. This

CampusX

traceability is what makes a RAG answer **grounded** and trustworthy rather than an opaque claim — a small amount of extra engineering effort (never dropping hierarchy metadata from loading through to generation) pays off directly in user trust.

10.4 RAG and Context Engineering: How They Fit Together

RAG and context engineering are essentially married to each other:

- Context engineering's core question is: *what should go into the model's context window?*
- RAG is the mechanism that answers that question dynamically, per query — keeping the context window otherwise **empty** and filling it only with the specific chunks relevant to the current question.

Without RAG, the alternative is dumping full documents into the context window for every query — which wastes tokens/cost/compute on irrelevant content, causes context pollution, and triggers the Lost-in-the-Middle problem.

Decision rule, revisited:

- **Large, multi-source, or scattered data** → build a RAG pipeline; it keeps cost and compute down and avoids context pollution.
- **Small data that comfortably fits in the context window** → skip RAG; upload it directly into the context window and KV-cache it for better, simpler results.

10.5 Golden Rules

- **A RAG pipeline is only as good as its database.** Poor document loading, chunking, or embedding quality caps everything downstream — no retrieval or generation step can recover from a badly formed database.
- **Load documents into Markdown before chunking.** Preserving structural hierarchy (headings, code fences, tables) at the loading stage prevents cascading structural loss later in the pipeline.
- **No single splitter handles mixed-content documents.** Route prose, code, and tables to the splitter built for each type rather than forcing one splitter to handle everything.
- **Isolated chunks lose context.** Always situate a chunk within its document (contextual retrieval) before storing it, so it remains findable and meaningful on its own.
- **The query embedding model must exactly match the database's embedding model** — otherwise retrieval silently fails.
- **Separate what you retrieve with from what you generate with.** Use enriched (contextualized) text for search; pass only the original, un-enriched text to the LLM for answer generation.

CampusX

- **Place retrieved context and the user's query at the very end of the prompt**, right before generation, to counter the Lost-in-the-Middle effect.
- **Always give the model an explicit permission to say "I don't know."** Strict grounding plus an ignorance clause is the primary defense against hallucination in a RAG system.
- **Never use RAG by default.** If your entire dataset comfortably fits the model's context window, direct context injection with KV caching is simpler, cheaper, and more accurate than building a full RAG pipeline.

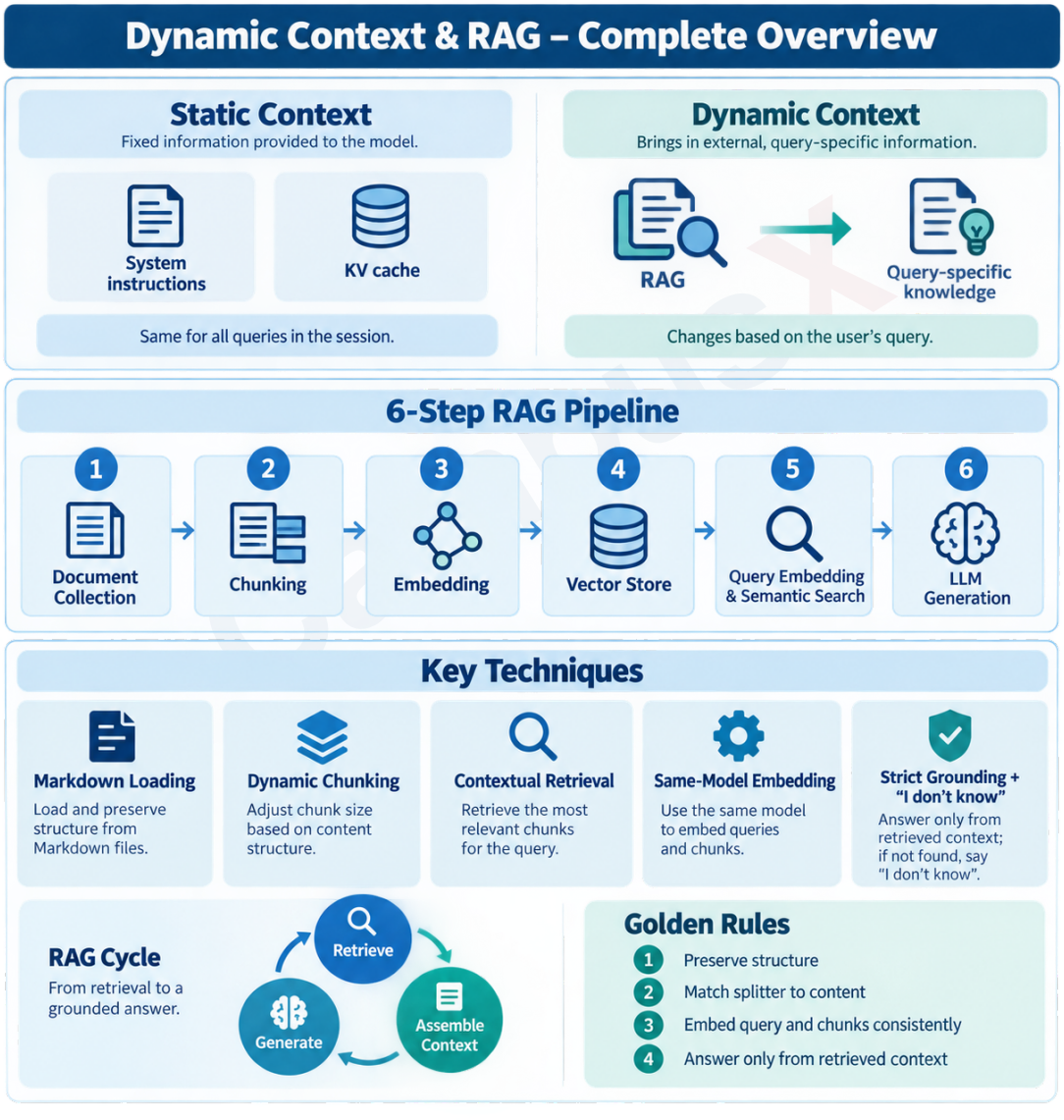

Figure 27. Complete overview of dynamic context and RAG, from source documents and chunking through retrieval and grounded answer generation.

COURSE MODULE

Effective Tool Calling for Proper Context Engineering

11 Tool, Tool Calling and Tool as Context Problem

This module covers tools and tool calling in depth — what they are, how the tool-calling cycle works, and (the central theme of the module) how to use tools *effectively* so they don't pollute the model's context window. This first session builds the foundation: understanding what tools are and why they exist, before diving into the core problem — **tools as a context problem**.

11.1 Why LLMs Need Tools

Brain vs. Hands and Legs: In the AI landscape, the LLM is the **brain** — all reasoning and processing happens there. **Tools** are the **hands and legs** — they let the model interact with and act on the external world.

> 🧠 **Analogy — Brain and Hands/Legs**
> The brain only processes; it doesn't act on the world directly. Hands and legs perform physical tasks and communicate with the environment. In the same way, the LLM processes information, but tools carry out the actual interaction and task execution.

Modern LLMs are extremely capable — they can write code, solve math problems, and generate creative text with high quality. Despite this, they consistently fail at certain categories of tasks:

- Sending an actual email (they'll draft it, but can't send it)
- Reporting the current weather in a city
- Answering questions about events after their training data ends (e.g., "who won the 2026 FIFA World Cup")
- Answering questions about private/internal data (e.g., "how many employees does my company have")

The Two Root Causes

Limitation	Explanation
Isolation	The LLM runs inside a closed environment (e.g., a cloud deployment) with no live connection to the outside world. It has no access to real-world events, private databases, or external systems unless explicitly given a way to reach them.

Limitation	Explanation
Knowledge cut-off date	Every LLM is trained on data only up to a fixed date. It cannot know about anything that happened afterward, because it was never trained on that information.

Demonstrated with Qwen 3 (8B) and Gemini 3.6 Flash: both refused to give a live weather reading for Tokyo, and both said the 2026 FIFA World Cup hadn't concluded yet (or hadn't started) as of their training cut-off, even after the tournament had actually ended. Without any external connection, the model can only work with what it was trained on — leading to generic responses, "I don't know" answers, or outright hallucination.

Tools exist to solve exactly these two problems — isolation and stale knowledge — by giving the model a way to reach outside its own training data.

11.2 What Is a Tool?

A tool is simply a Python function — nothing more exotic than that. The distinguishing feature is that this function gives the model a capability it didn't have on its own: searching the web, hitting an API, querying a database, running code, or interacting with other software.

> Tool: an external function or service that an AI agent can use to perform tasks beyond its inherent capabilities — such as web search, code execution, database access, using an API, or interacting with software.

Example: a plain `get_weather(city)` function that calls a weather API and returns the result. On its own, an LLM can't fetch live weather. Once this function is registered as a tool and passed to the model, the model can now answer weather questions — not because it "learned" weather data, but because it can now delegate that sub-task to a function built for it.

> 🧠 **Analogy — Chaat Masala (Capability Enhancer)**
> A tool is like chaat masala sprinkled on a fruit salad — it doesn't replace the fruit, it enhances what's already there. Tools don't replace the LLM's reasoning; they extend what it's capable of doing. A model that could originally handle 10 kinds of tasks might handle 14 once four well-designed tools are attached to it.

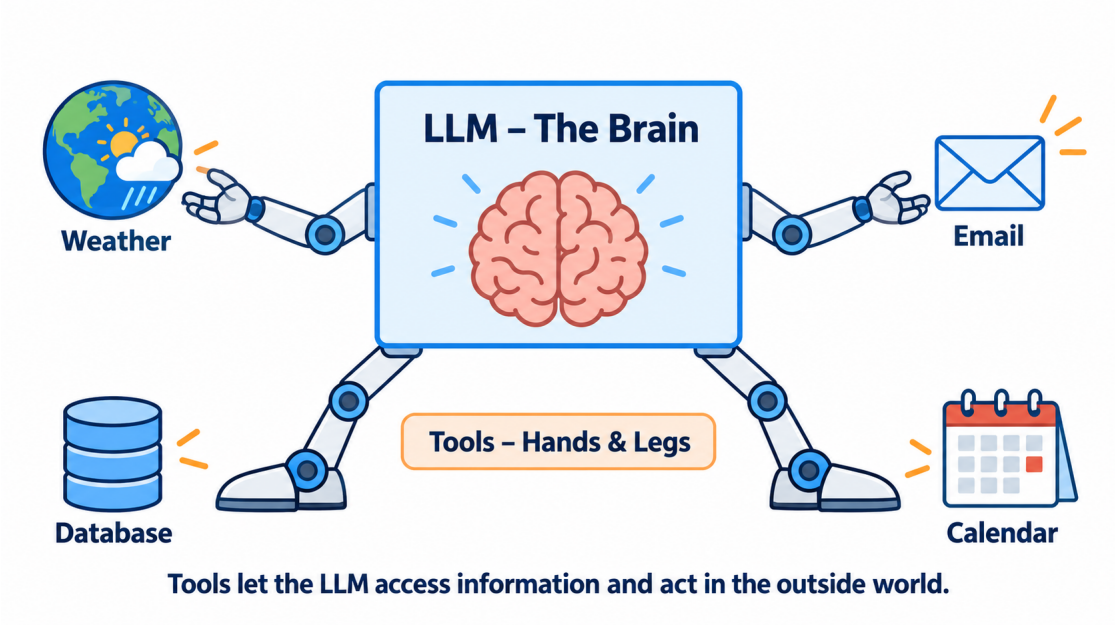

Figure 28. The LLM supplies reasoning while tools act as external hands that search, calculate, retrieve data, and perform actions.

Tool calling is the process by which the LLM actually uses a tool to accomplish part of a task.

11.3 The Tool-Calling Cycle

Five steps describe the full lifecycle of a tool call, from writing the function to getting a grounded final answer.

1. **Create the function.** Define the Python function (e.g., `get_weather(city: str) -> str`), including a clear description and explicit parameter data types. Correct typing matters for reliable tool selection — covered in more depth in a later session.
2. **Pass the function to the LLM.** When calling the model, the function definition is supplied via a `tools` parameter alongside the user's message.
3. **LLM evaluates the query.** It reads the question and checks whether it can answer from its own knowledge. If yes, it answers directly. If no, it scans the available tool definitions to see if one matches the need.
4. **LLM returns a tool call (not an answer).** If a tool matches, the LLM's response contains no direct answer — instead it specifies which function to call and with what arguments (e.g., call `get_weather` with `city="Tokyo"`).
5. **Execute and re-send.** The calling system (not the LLM) actually runs the function, captures its output, appends that output to the conversation's message history, and sends the full history back to the LLM. The LLM then reads this and produces the final, grounded answer.

Worked example (Qwen 3, 8B): asking "What is the current weather condition of Japan's capital?" with `get_weather` registered as a tool produced an empty `content` field but a populated `tool_calls` field naming `get_weather` with `city="Tokyo"`. After executing the function and feeding the result back, the model correctly answered: *"The current weather conditions in Tokyo, Japan are 65°F with clear skies."*

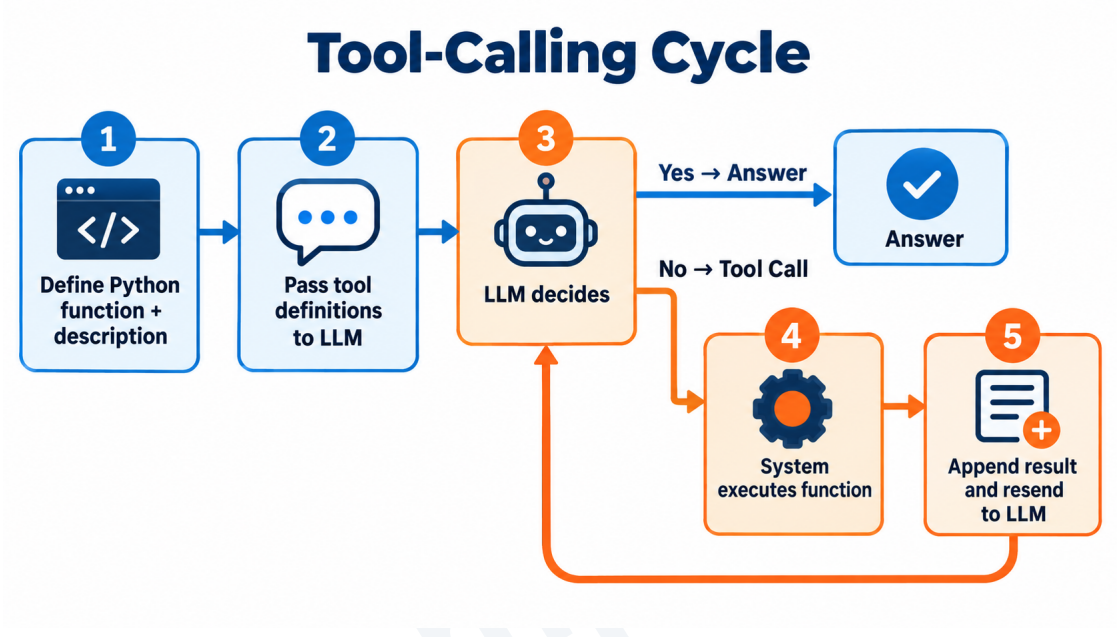

Figure 29. The tool-calling cycle moves from tool definition and model selection to external execution, returned results, and a grounded final response.

> 💡 **Insight — Why Doesn't the LLM Just Run the Tool Itself?**
> An LLM is fundamentally a **next-token predictor** — it can predict text, not execute code. It has no access to your local environment, filesystem, or network; only *you* (or your application/orchestration code) can actually run the function. This is a common interview question: the LLM's job stops at telling you *what* to call and *with what arguments* — running it is always the calling system's responsibility.

11.4 Benefits of Tool Calling

- **Real-time information access** — removes the isolation and knowledge cut-off problems entirely.
- **Action, not just retrieval** — tools let the model *do* things (send an email, change a setting) rather than only answer questions.
- **Higher accuracy, less hallucination** — grounding responses in real, current data reduces made-up answers.

- **End-to-end automation** — chaining multiple tools lets a model complete a whole workflow from a single prompt.
- **Dynamic, up-to-date knowledge** — the model's effective knowledge stays current even though its training data is frozen.

Real-world AI systems (Claude Code, GitHub Copilot, Cursor) are essentially an LLM plus a curated set of tools — one tool to write code, one to run terminal commands, one to debug, one to monitor, and so on. The LLM processes and reasons; the tools perform the actual work. Tool and tool calling are core components of virtually all modern AI applications.

11.5 The Core Problem: Tools as Context Pollution

Tools are necessary — but every tool definition you register also gets injected into the model's context window on **every single call**, whether or not that call actually needs it. This is the central problem the rest of the module addresses.

How the chat template works

A typical API call is structured through a chat template in this order: **system instruction → tool definitions → user message(s)**. All registered tool descriptions are sent on *every* call, regardless of relevance to that specific query.

Case Study: MegaMart

An internal support assistant was built for a fictional company, MegaMart, with 18 tools across four categories:

Category	Example tools
Product	`search_product`, `get_product_by_id`, `get_price_history`, `check_warranty_info`, `get_product_reviews`
Inventory	`get_inventory_count`, `get_low_stock_items`, `get_supplier_info`, `create_restock_order`
Orders	`get_order_details`, `list_recent_orders`, `get_shipping_status`, `process_refund`, `get_return_request`
Customer/Support	`get_customer_profile`, `get_support_ticket`, `get_sales_summary`, `get_payment_status`

With all 18 tool definitions attached, a trivial message — "Hi, how are you?" — consumed **~2,100 tokens** to process. The system instruction plus the greeting itself accounted for only ~100 tokens; the remaining ~2,000 tokens were pure tool-definition overhead that this particular query never

CampusX

needed. That overhead translates directly into unnecessary compute cost and added latency, on every call.

> ⚠ **Mentor's Rule — Tools Are Always in the Prompt, Whether Used or Not**
> Every tool you register gets sent to the model on every single API call as part of the chat template, regardless of whether that call actually needs it. More tools registered means more baseline token cost paid on every request — this scales badly as a system grows toward dozens or hundreds of tools.

Agentic Loops Make It Worse

An **agentic loop** is when a single user query triggers the model to call tools repeatedly and autonomously — reading a result, deciding the next tool to call, and repeating — until it has enough information to answer (this is exactly how tools like Claude Code iterate on fixing an error).

Example query against the MegaMart assistant: *"What are customers saying about AeroBook 14? Summarize the common complaints."* This single query triggered a 3-step autonomous loop:

1. Call `search_product` to find the product and get its ID.
2. Call `get_product_reviews` using the ID returned from step 1.
3. Synthesize the reviews into a final answer (no further tool call needed).

Token cost per turn accumulated fast — the final turn alone required **~6,021 tokens** for a single, fairly simple query, against an 8B Qwen 3 model with a ~40K token context limit. That's roughly **16% of the total context window consumed by one query**. A short multi-turn conversation can fill the context window entirely within just a handful of exchanges.

In real production systems (like coding agents that iterate through many tool calls to fix an error), this compounds much further — token consumption can easily climb into the hundreds of thousands.

Why This Matters

A context window that's saturated with tool definitions and tool-call history:

- costs more compute and adds latency on every call,
- leaves less room for the actual reasoning that matters, and
- risks the **"lost in the middle"** phenomenon, where information buried in a long context gets effectively ignored, degrading response quality.

CampusX

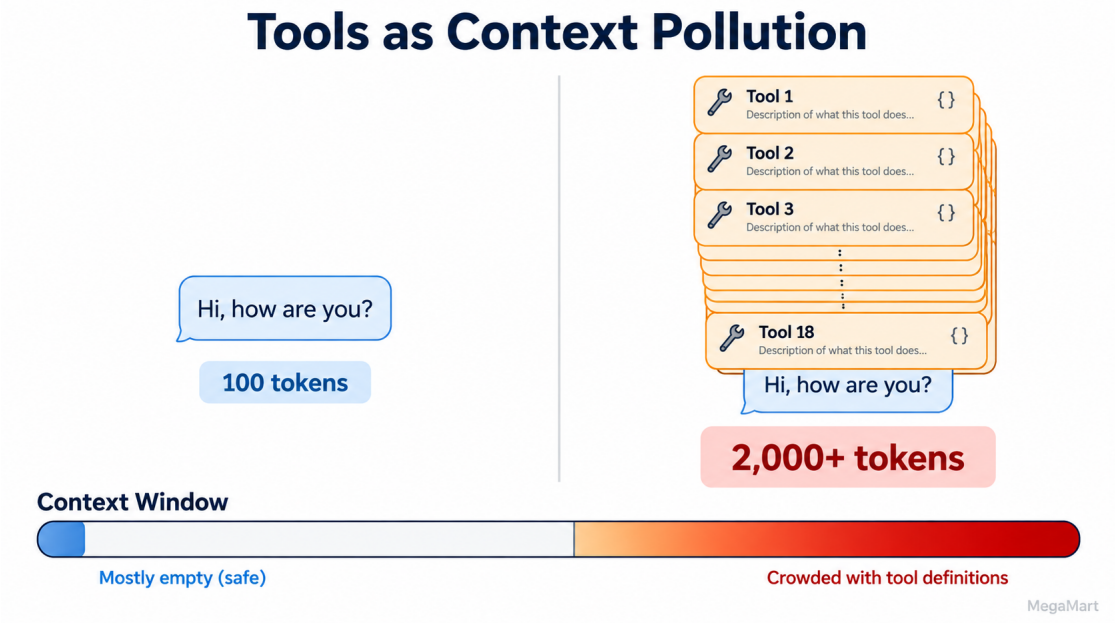

Figure 30. Registering many tools creates fixed context overhead on every request and compounds further during agentic loops.

11.6 Session Takeaways

1. **Tools are necessary** — without them, an LLM cannot search live data, act on the world, or overcome its isolation and knowledge cut-off limitations.
2. **Tools are also a major context problem** — every registered tool adds baseline token cost to every call, and agentic loops can consume large fractions of the context window for a single query.

These two facts together lead to the module's central goal: learning to write and manage tools *effectively*, so that capability is gained without letting the context window become bloated or degraded.

12 Tool Places in Context and Writing Effective Tools

Building on the previous session's point that tools are a context problem, this lecture breaks that problem down into its three concrete mechanisms, then covers the practical rules for writing tools that keep the context window healthy.

A useful framing to start with: nothing related to a tool — writing it, running it, generating its output — happens on the LLM's end. All of it happens on the human/system side. What actually reaches the LLM's context window is only the *information about* and *output of* that work.

Developer folk wisdom ranks context consumers roughly as: **message history first, tools and tool calling second.**

12.1 The Three Ways Tools Occupy the Context Window

Tools take up context space through three distinct channels: **tool definition**, **tool call**, and **tool result**.

Channel	What it is	Relative size	Cost type
Tool definition	The JSON schema describing every registered tool	Large, and repeated on *every* call	Fixed — same every time
Tool call	The LLM's request to invoke a specific function with specific arguments	Small (~30–50 tokens)	Grows with iterations
Tool result	The output returned by the executed function	Can be very large and unpredictable	Grows with iterations

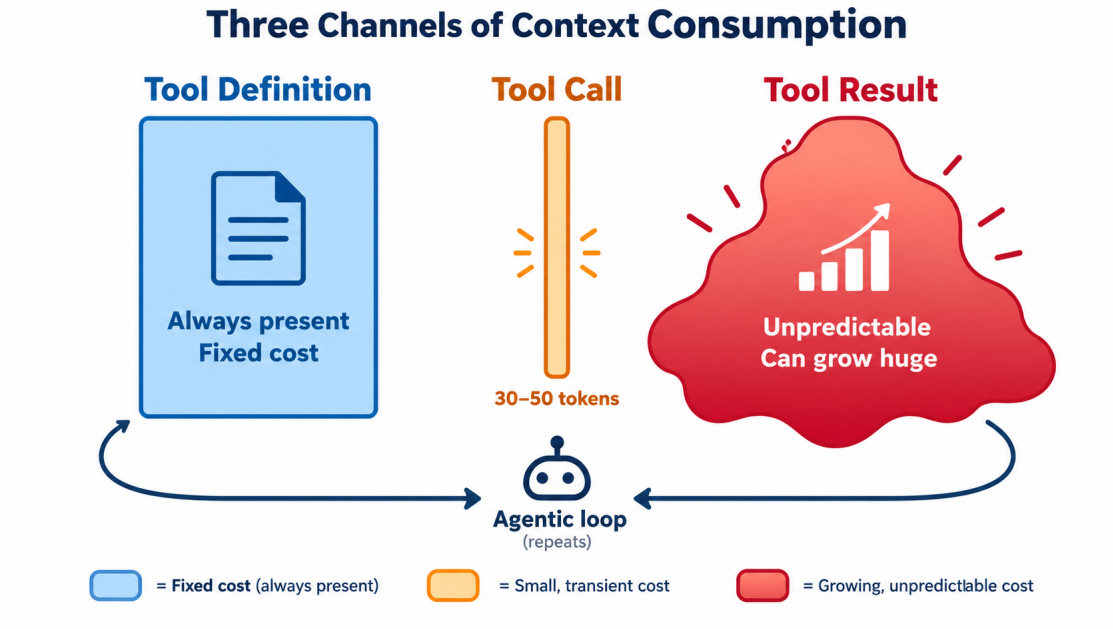

Figure 31. Tools consume context through their definitions, the model's tool calls, and the results returned by execution.

CampusX

Tool Definition — The Fixed Cost

An LLM never sees your raw Python function. Before the request is sent, the SDK/framework (Ollama, LangChain, LangGraph, etc.) automatically converts each Python function into a **JSON schema** — this is the tool's actual "definition" as far as the model is concerned. The JSON captures the function's name, its description, its parameters, each parameter's data type, and what it returns. The LLM reads only this JSON when deciding which tool to call and with what arguments — never your source code.

```python
from ollama._utils import convert_function_to_tool

def get_inventory_level(product_id: str, warehouse: str = "all"):
    """Get current stock levels for a product.

    Use this for any question about availability or stock. Stock numbers
    update every 15 minutes, so treat them as near-real-time.

    Args:
        product_id: The MegaMart product ID.
        warehouse: One of 'delhi', 'mumbai', 'bangalore', or 'all'.
    """
    ...

tool1 = convert_function_to_tool(get_inventory_level)
print(tool1.model_dump_json(indent=2))
```

This produces a schema like:

```
{
  "type": "function",
  "function": {
    "name": "get_inventory_level",
    "description": "Get current stock levels for a product.\n\nUse this for
     ↪  any question about availability or stock...",
    "parameters": {
      "type": "object",
      "required": ["product_id", "warehouse"],
      "properties": {
        "product_id": { "type": "string", "description": "The MegaMart
         ↪  product ID." },
        "warehouse":  { "type": "string", "description": "One of 'delhi',
         ↪  'mumbai', 'bangalore', or 'all'." }
      }
    }
```

```
    }
}
```

Older codebases wrote this JSON schema by hand; modern frameworks generate it automatically from the function's signature, type hints, and docstring — which is exactly why writing the function *well* matters (Section 2).

Why this is a fixed cost, and why it's expensive: almost every model's chat template places tool definitions right after the system instruction and *before* the user's message — so registering a tool guarantees its full JSON schema is processed on **every single API call**, whether or not that call needs it. Asking a MegaMart assistant with 18 registered tools a plain "Hi, how are you?" still costs ~2,100 tokens, almost entirely tool-definition overhead the query never needed. Scale this to a real agent with 40 tools at ~700 tokens each, and you're carrying a fixed ~28K-token tax on every call, regardless of how many tools that specific call actually uses.

Tool Call — The Cheap One

When the LLM decides to invoke a function, its response (name + arguments) also lives in the message history and therefore in the context window — but this is a small, largely unavoidable cost (typically 30–50 tokens). It's necessary: without it the system wouldn't know which function to run. In practice, this segment rarely causes context engineering problems.

Tool Result — The Real Culprit

The output returned by an executed function is appended back into the message history and sent to the LLM on the next turn. Unlike the other two channels, **you cannot bound this size in advance.** A scraper tool pulling social mentions might return 30 results in one run and 5,000 in the next. If each mention costs ~100 tokens, 100 mentions alone is 10,000 tokens — more than the entire tool-definition budget of a 20-tool agent. Error-capturing tools (like a terminal-error catcher in a coding agent) are just as unpredictable — an error can be one line or a hundred.

> ⚠ **Mentor's Rule — Tool Result Is the Real Context Killer**
> Of the three channels, tool result is usually the biggest and least predictable consumer of context. You can size tool definitions and tool calls in advance; you generally can't size tool output. Never dump a tool's raw output into context unmanaged — this is exactly the segment context-engineering techniques (covered later in the module) target hardest.

Fixed Cost vs. Growing Cost

- **Tool definition = fixed cost.** Register 40 tools at ~700 tokens each and you're carrying ~28K tokens from the very first message — today, tomorrow, for every user, forever, regardless of which tools actually get used.

- **Tool call + tool result = growing cost.** These accumulate with every iteration of an agentic loop. A 3-turn agentic loop went from ~2,123 tokens (turn 1) to ~2,438 tokens (turn 2) to ~6,153 tokens by turn 3 for a single, fairly simple query — and this compounds much further in longer-running agents like coding assistants.

Both need active management, and the rest of this module is dedicated to the techniques for doing that.

12.2 Why Writing Effective Tools Matters

Since the LLM only ever sees the JSON schema — never your raw Python — a poorly written function produces a poor schema, and a poor schema is functionally invisible or misleading to the model, even if the underlying function works perfectly.

Compare the same function written two ways:

```python
# No description, no meaningful naming discipline
def get_inventory_level(product_id, warehouse):
    return {...}
```

```json
{
  "type": "function",
  "function": {
    "name": "get_inventory_level",
    "description": "",
    "parameters": {
      "properties": {
        "product_id": { "type": "string", "description": "" },
        "warehouse":  { "type": "string", "description": "" }
      }
    }
  }
}
```

With an empty description and no parameter guidance, the model has no basis for deciding *when* to call this tool or *what* to pass — the JSON is effectively useless for tool selection.

> 💡 **Insight — Bad Schemas Cause Silent Failure Loops**
> When a tool's JSON is vague, the model can't reliably identify the right tool for a query. It may pick the wrong one, get an unhelpful result, and try another — burning multiple failed tool-call round trips (each with its own token cost) before stumbling onto the one that actually answers the query. Every failed round trip you see in a trace is often a tool-description problem, not a model-reasoning problem.

There are two ways to get a good JSON schema: hand-write it for every function (accurate but slow and doesn't scale past a handful of tools), or — the recommended approach — write the Python function itself well enough that the auto-generated schema comes out clean. Only providers that don't offer an auto-conversion utility force you into the manual route.

12.3 Rules for Writing Effective Tools

Rule 1 — Clear Name, and a Consistent Family Name

Avoid vague names like `dev_get`. Name functions for what they do, and give related functions a shared prefix/family so they group logically — e.g. `get_order_details`, `list_recent_orders`, `get_shipping_status` all signal "orders"; `get_inventory_level`, `get_low_stock_items` signal "inventory." This naming discipline becomes important again later in the module when context-management techniques rely on grouping tools by family.

Rule 2 — Write a Proper, Structured Description

Every function's docstring/description must cover three things:

1. **What the function does.**
2. **What its arguments mean.**
3. **What it returns.**

The docstring is copied verbatim into the JSON schema's `description` field, so vague comments become vague — and unhelpful — schemas. It doesn't need to be long, but it must be precise enough that someone reading only the description (not the code) understands the function's role in a few seconds.

Also apply **type hints** to every parameter (`product_id: str`, `units: int`) — this is what tells the auto-conversion utility (and, downstream, the LLM) what data type each argument expects when constructing a tool call.

A well-documented function:

CampusX

```python
def laptop_get_price(product_id: str, region: str = "IN", response_format:
↪ str = "short"):
    """Get the current price and stock status of one laptop.

    Use this when the customer asks about cost, discounts, or availability.
    Do not use this for specifications - use laptop_get_specs for that.

    Args:
        product_id: The exact product ID, for example 'P-1001'. Get this
            from laptop_search first if you don't already have it.
        region: Country to get pricing for. Must be one of 'IN', 'US', 'UK'.
        response_format: 'short' returns just the price and stock status.
            'full' also includes price history and promotion details.
            Use 'short' unless the customer specifically asks about
            discounts or price trends.

    Returns:
        A dict with price_inr (int), stock_status (str), and, only when
        response_format is 'full', mrp_inr (int) and on_promotion (bool).
        Returns an error field instead if the product_id is not found.
    """
    catalog = {"P-1001": {"price": 94990, "mrp": 109990, "stock": "in stock",
↪ "promo": False}}

    if product_id not in catalog:
        # Tell the model exactly what went wrong and what to do next -
        # never leak a raw exception or stack trace into the tool result.
        return {"error": f"No product with id '{product_id}'."}

    item = catalog[product_id]
    result = {"price_inr": item["price"], "stock_status": item["stock"]}
    if response_format == "full":
        result["mrp_inr"] = item["mrp"]
        result["on_promotion"] = item["promo"]
    return result
```

This produces a rich schema — clear function description, and every parameter individually explained — that gives the model everything it needs to select and call the tool correctly on the first attempt.

> ⚠ **Mentor's Rule — Handle Errors Deliberately, Never Leak Stack Traces**
> A common mistake is wrapping tool logic in `try / except` and returning the raw exception (`print(e)`), which can be arbitrarily long and unpredictable — exactly the uncontrolled tool-result problem from Section 1. Instead, catch the failure case explicitly and return a short, specific, structured error message the model can act on (e.g. `{"error": "No product with id 'P-9999'. Call laptop_search first to find the correct id."}`).

A fuller real-world example — a live weather tool — applies the same discipline: validating inputs itself rather than silently defaulting, returning structured `status: "success"` / `status: "error"` results, and giving specific, actionable error strings for each failure mode (timeout, service unavailable, city not found):

```python
def weather_get_current(city: str, unit: str = "celsius") -> dict:
    """Get the current real-time weather for a city.

    Use this whenever the user asks about current temperature, wind, or
    general conditions "right now" or "today". Do not use this for
    multi-day forecasts.

    Args:
        city: The name of the city in English, e.g. 'London', 'Tokyo',
            'Chandigarh'. If the name is ambiguous, use the most
            well-known city with that name.
        unit: Temperature unit. Must be exactly 'celsius' or 'fahrenheit'.

    Returns:
        On success: a dict with status ('success'), city, country,
        temperature, unit, windspeed_kmh.
        On failure: a dict with status ('error') and a message explaining
        what went wrong and what to try instead.
    """
    unit = unit.strip().lower()
    if unit not in ("celsius", "fahrenheit"):
        return {"status": "error",
                "error": f"unit must be 'celsius' or 'fahrenheit', got
                 ↪  '{unit}'"}

    geocode_url = (
        "https://geocoding-api.open-meteo.com/v1/search"
        f"?name={city}&count=1&language=en&format=json"
    )
    try:
        geo_data = requests.get(geocode_url, timeout=10).json()
    except requests.RequestException:
        return {"status": "error", "error": "location lookup service is
         ↪  unavailable right now"}
```

```
    if not geo_data.get("results"):
        return {"status": "error",
                "error": f"no city found matching '{city}'. Check the
                 ↪  spelling or try a larger nearby city."}

    location = geo_data["results"][0]
    weather_url = (
        "https://api.open-meteo.com/v1/forecast"
        f"?latitude={location['latitude']}&longitude={location['longitude']}"
        f"&current_weather=true&temperature_unit={unit}"
    )
    weather_data = requests.get(weather_url, timeout=10).json()
    current = weather_data.get("current_weather", {})

    return {
        "status": "success",
        "city": location["name"],
        "country": location.get("country", "unknown"),
        "temperature": current.get("temperature"),
        "unit": unit,
        "windspeed_kmh": current.get("windspeed"),
    }
```

Rule 3 — Never Write Overlapping Tools

Avoid registering multiple tools that do essentially the same thing under different names (e.g. `get_info`, `fetch_details`, `look_up`, all fetching product data). When the model has to choose between near-duplicate tools, it spends extra reasoning (and output tokens) trying to disambiguate them — with no guarantee it even picks correctly. Each tool should have one clear, non-overlapping purpose.

A quick self-check: if you can't explain what a tool does in about five seconds by reading its description, the model won't be able to either — rewrite it before registering it.

Figure 32. Effective tools use clear names, complete descriptions, explicit types, structured results, and non-overlapping responsibilities.

12.4 Summary

Tools occupy context through three channels — a large, fixed-cost **definition**, a small **call**, and a potentially huge, unpredictable **result** — and the definition and result are the two that actually create context-engineering problems worth managing. The single highest-leverage fix is upstream of any context-management technique: write functions with clear names, family-consistent naming, complete three-part descriptions (what it does / what its args mean / what it returns), full type hints, deliberate structured error handling, and no functional overlap with other registered tools. A clean function produces a clean JSON schema, and a clean schema is what actually lets the model select and call tools correctly — everything downstream in this module builds on that foundation.

13 Deferred Loading

The previous session identified three channels through which tools consume context: **tool definition** (fixed cost, paid on every call), **tool call** (small, unavoidable), and **tool result** (large, unpredictable). This lecture tackles the first of those — tool definition — with a technique called **Deferred Loading**, also known as **Tool RAG**.

CampusX

13.1 Restating the Problem

Whenever an application registers tools for an LLM, every registered tool's JSON schema gets processed on **every** API call — regardless of whether that specific query needs it. This happens because most chat templates place all tool definitions right after the system instruction and before the user's message, so they're baked into every request by construction.

Demonstrated on a MegaMart assistant with 15 registered tools:

Query	Tools actually needed	Tokens consumed
"Hi, how are you?"	None	~2,071
"What is the price of AeroBook 14?"	`search_products` only	~2,078
"Give me sales summary for last 7 days"	`get_sales_summary` only	~2,078

In every case, the model still has to process all 15 tool definitions before it can even decide which single tool (if any) is relevant. This is unnecessary cost, latency, and context pollution — the model never needed 14 of those 15 definitions to answer any of these queries.

13.2 What Is Deferred Loading?

Deferred Loading is a general performance-optimization pattern — borrowed from software engineering, where images, scripts, or videos are loaded only when actually needed rather than all upfront. Applied to tool calling:

> Deferred Loading is the technique of loading only the tool definitions required for the current user query, instead of sending every available tool definition on every call.

> 🧠 **Analogy — A Disciplined Finance Department**
> If a company's finance department released the entire budget upfront with no gatekeeping, it would get used carelessly — nobody has a reason to be efficient. If finance calculates and releases budget only as needed, it gets used effectively because everyone works within a visibly limited allocation. Deferred Loading treats your tool definitions the same way: give the model only what the current task actually requires, nothing more.

This is also called **Tool RAG**, because the flow directly mirrors Retrieval-Augmented Generation: instead of retrieving relevant *documents* for a query, you retrieve relevant *tools*.

13.3 How Deferred Loading Works: The Search Tool

Deferred Loading introduces one new function — commonly called the **Search Tool** — that acts as a search engine over your tool registry, the same way Google returns only relevant pages out of the entire web rather than everything it knows.

> 💡 **Insight — The Search Tool and Its Schema Are the Real Product**
> Once you adopt deferred loading, the search tool's implementation and its JSON schema become the most important artifacts in your system — they're what filters everything else the LLM can ever reach. If the search tool is written poorly, every downstream tool becomes effectively unreachable no matter how well *those* tools are written. This is also why, unlike ordinary tools, the search tool's schema is usually written by hand rather than left to auto-generation — it's too central to leave to chance.

Complete Workflow

1. Create a **search function** that scans the tool registry and returns the names of tools relevant to a query.
2. Write a **JSON schema** for this search function (by hand, for reliability — see the insight above).
3. Call the LLM with the user's query and **only the search tool's schema** — none of the actual domain tools.
4. The LLM decides whether it needs a capability; if so, it emits a tool call for `search_tools` with a query string.
5. The application runs the search function against the registry and gets back matching tool names.
6. The LLM is called a **second time** — same user query, but now with the definitions of only the returned tool(s) attached.
7. The LLM emits a tool call for the actual matched tool.
8. The application executes that tool and returns the result to the LLM.
9. The LLM produces the final answer.

CampusX

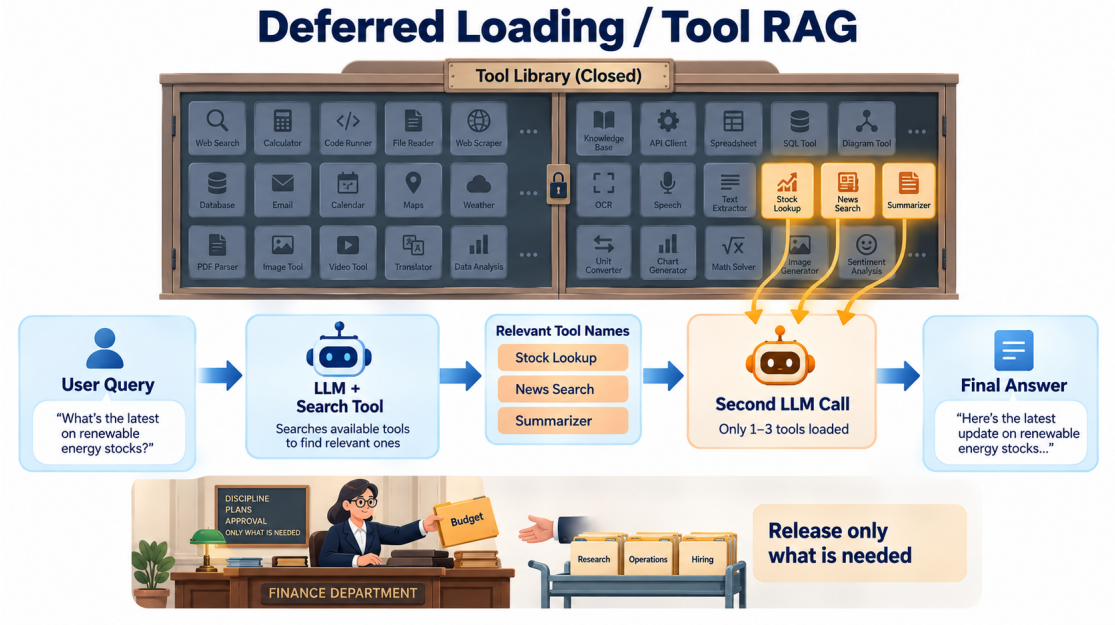

Figure 33. Deferred loading uses a small search tool to discover and load only the tool definitions relevant to the current query.

A Minimal Search Tool (Keyword Matching)

```python
def search_tools(query, max_results=3):
    words = query.lower().split()
    hits = []
    for f in REAL_TOOLS:
        text = (f.__name__ + " " + (f.__doc__ or "")).lower()
        if any(w in text for w in words):
            hits.append(f.__name__)
    return hits[:max_results]
```

This does simple keyword matching against each tool's name and docstring. Its schema (written by hand):

```python
SEARCH_TOOL = {
    "type": "function",
    "function": {
        "name": "search_tools",
        "description": (
```

```
            "Search for tools that are not yet loaded into this conversation.
             ↪  "
            "Right now, this search tool is the only capability you have - "
            "every other tool must be found through this one before it can "
            "be called. Use it the moment you realize you need a capability "
            "you don't currently have.\n\n"
            "Returns a list of matching tool names. Once a tool is returned "
            "here, it becomes directly callable on your next turn - you do "
            "not need to search for it again later in this conversation.\n\n"
            "If nothing matches, try again with broader or different keywords
             ↪  "
            "rather than giving up."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": (
                        "Short keywords describing the capability you need -
                         ↪  "
                        "not a full sentence. Good: 'check inventory stock',
                         ↪  "
                        "'process refund', 'sales report'. Avoid: a complete
                         ↪  "
                        "question or a full description of the task."
                    ),
                },
                "max_results": {
                    "type": "integer",
                    "description": "Maximum number of matching tools to
                     ↪  return. Defaults to 3.",
                },
            },
            "required": ["query"],
        },
    },
}
```

⚠ Mentor's Rule — Write the Search Tool's Description as Usage Instructions, Not Just a Label
Notice the schema above explicitly tells the model *when* to use the search tool, what "good" query keywords look like vs. what to avoid, and that a returned tool stays usable for the rest of the conversation without re-searching. A vague description here (just "searches tools") leaves the model guessing about query style and search behavior — the same JSON-quality principle from the previous lecture applies with even higher stakes to this one function.

Measured Savings

Scenario	Without deferred loading	With deferred loading
"Hi, how are you?"	~2,071 tokens	~443 tokens
"Give me sales summary for last 7 days" (2 LLM calls combined)	~2,078 tokens	~450 + ~613 ≈ 1,100 tokens
"What is the price of AeroBook 14?"	~2,571 tokens, ~30s	~621 tokens, ~2s
Multi-hop: "Current stock level for AeroBook 14?"	~2,792 tokens, ~27s	~1,130–1,150 tokens, ~35s (extra round trip)

Roughly 900–2,000 tokens saved per call in these examples, though the trade-off is one extra LLM round trip for the search step itself — occasionally costing more wall-clock time even while saving tokens.

13.4 Automating It: The Agentic Loop

Rather than manually orchestrating "call LLM → call search tool → call LLM again," the whole cycle can run as a loop: start with only the search tool loaded, and every time the model discovers a new tool via search, add that tool's definition to the active tool list for subsequent turns (never re-removing it, so it stays callable for the rest of the conversation).

```python
def run_with_deferred_loading(question, max_turns=8, verbose=True):
    tools = [SEARCH_TOOL]
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": question},
    ]

    for turn in range(1, max_turns + 1):
        response = ollama.chat(
            model=MODEL,
            messages=messages,
            tools=tools,
            options={"num_ctx": 8000},    # avoid silent context truncation
        )

        if verbose:
            print(f"turn {turn}: {response.prompt_eval_count or 0} tokens,
             ↪  tools loaded = {len(tools)}")

        messages.append(response.message)
```

```
        if not response.message.tool_calls:
            print(f"\nANSWER: {response.message.content}\n")
            return messages

        for call in response.message.tool_calls:
            name, args = call.function.name, call.function.arguments

            if name == "search_tools":
                found = search_tools(**args)
                messages.append({
                    "role": "tool", "tool_name": "search_tools",
                    "content": json.dumps({"found": found}),
                })
                for tool_name in found:
                    fn = REGISTRY[tool_name]
                    if fn not in tools:
                        tools.append(fn)          # newly discovered tool
↪  stays loaded
            else:
                result = REGISTRY[name](**args)
                messages.append({
                    "role": "tool", "tool_name": name,
                    "content": json.dumps(result),
                })

    print("stopped: hit max_turns without a final answer")
    return messages
```

Trace for *"What is the current stock level for the AeroBook 14 laptop?"* — this needs a **multi-hop** lookup (find the product ID first, then check inventory):

```
turn 1: 400 tokens, tools loaded = 1
  [call] search_products({'query': 'AeroBook 14'})
turn 2: 574 tokens, tools loaded = 1
  [search] {'query': 'get current stock level for a product id'}
        -> ['get_inventory_level', 'get_low_stock_items',
↪  'get_product_details']
turn 3: 1016 tokens, tools loaded = 4
  [call] get_inventory_level({'product_id': 'P-1001', 'warehouse': 'all'})
turn 4: 1130 tokens, tools loaded = 4

ANSWER: The current stock level for the AeroBook 14 (P-1001) is 47 total
↪  units...
```

Against the same query without deferred loading: **~2,729 tokens** in 2 turns. A well-written search tool lets the model chain multiple discovered tools together naturally (`search_products`

→ `get_inventory_level`) exactly as it would with all tools pre-loaded — it just discovers them on demand instead of holding them all in context from the start.

> Note: in a production registry, a "glue" tool that most other tools depend on for an ID (like `search_products`, which nearly every product-related tool needs) can be loaded alongside the search tool by default, so the model doesn't have to spend a discovery round-trip finding it every time.

13.5 Better Search Tools: Keyword + Semantic Matching

The keyword-only search shown above is a bare minimum, useful for understanding the concept — but production search tools typically combine two techniques:

1. **Keyword search** — exact word overlap between the query and a tool's name/docstring.
2. **Semantic search** — embedding-based similarity, capturing meaning rather than exact words (the same embeddings/vectors concept from RAG, using cosine similarity directly rather than a full RAG pipeline, to keep things simpler).

```
def get_embedding(text, task_type="search_document"):
    response = ollama.embed(model=EMBED_MODEL, input=f"{task_type}: {text}")
    return response["embeddings"][0]

def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def clean_docstring(doc):
    # Keep only the summary line(s), drop the Args:/Returns: sections -
    # that's the part that actually carries the tool's semantic meaning.
    if not doc:
        return ""
    return re.split(r"\n\s*(Args:|Returns:)", doc)[0].strip()

# Precompute embeddings ONCE (not inside search_tools on every call)
TOOL_EMBEDDINGS = {}
for f in REAL_TOOLS:
    text = f.__name__ + " " + clean_docstring(f.__doc__)
    TOOL_EMBEDDINGS[f.__name__] = get_embedding(text,
 ↪  task_type="search_document")

def search_tools(query, max_results=3):
    query_emb = get_embedding(query, task_type="search_query")
    query_words = set(query.lower().split())

    scored = []
```

```
    for f in REAL_TOOLS:
        name = f.__name__
        text = (name + " " + clean_docstring(f.__doc__)).lower()
        tool_words = set(text.split())

        keyword_score = len(query_words & tool_words) / max(len(query_words),
↪   1)
        embedding_score = cosine_similarity(query_emb, TOOL_EMBEDDINGS[name])
        combined = (0.6 * keyword_score) + (0.4 * embedding_score)
        scored.append((combined, name))

    scored.sort(reverse=True)
    return [name for score, name in scored[:max_results]]
```

> 💡 **Insight — Cosine Similarity Is Just a Distance Measure**
> Represent the query and every tool's description as vectors (embeddings). Cosine similarity measures how close two vectors point in the same direction — a smaller angular distance means the tool's meaning is closer to what the query is asking for, so it ranks higher. This is the same idea used to rank document relevance in RAG, applied here to ranking tool relevance.

The `0.6` (keyword) / `0.4` (embedding) weighting is a tunable choice — found empirically to work well for this case, not a fixed rule. Precomputing tool embeddings once (rather than on every search call) avoids re-embedding the same static tool descriptions repeatedly.

13.6 When Deferred Loading Helps — and When It Doesn't

Deferred loading trades a small amount of complexity and (usually) one extra LLM round trip for large savings in tokens, cost, and often latency — but it isn't free, so it isn't always the right call.

- **Small tool sets (~5–6 tools):** skip deferred loading. The extra round trip and added implementation complexity aren't worth it when the token savings would be marginal — just pass all tools directly.
- **Large tool sets:** this is where deferred loading earns its keep, and the benefit compounds as the tool count grows.

Accuracy, Not Just Cost

Beyond token savings, research (cited from a Digital Webcraft summary) found that tool-selection **accuracy degrades as the number of directly-loaded tools grows** — more candidate tools make it harder for the model to pick correctly, even before running out of context:

Tools available	Accuracy without deferred loading	Accuracy with deferred loading
~10	~78%	~90%

CampusX

Tools available	Accuracy without deferred loading	Accuracy with deferred loading
~100	41–83% (fluctuates)	~85%
~100+	~13%	~43%
~700+	0–20%	~60%

Deferred loading consistently outperformed the direct approach, and the gap widened sharply as tool count increased.

A Hard Platform Limit

Deferred loading isn't only about context management — it also works around hard platform ceilings. OpenAI's API, for example, officially supports passing a maximum of **128 tools** in a single call. Beyond that limit, some form of tool retrieval (deferred loading / Tool RAG) is required, not just recommended.

Industry adoption is real and growing: Red Hat's emerging-technologies team has published on building "Tool RAG" systems using essentially this same pattern.

13.7 Summary

Deferred loading solves the tool-definition portion of the context problem by replacing "load every tool on every call" with "load a search tool, discover what's actually needed for this query, then load only that." It costs a small amount of extra complexity and typically one additional LLM call, but the savings are large and grow with tool count — both in tokens/cost/latency and, notably, in tool-selection accuracy. It's not worth adopting for a handful of tools, but becomes close to mandatory once an application's tool count climbs into the dozens, hundreds, or beyond a platform's hard tool-count limit.

13.8 Golden Rules

1. **The LLM only ever sees a tool's JSON schema, never its code.** A tool is only as good as its schema — clear name, a complete description (what it does, what each argument means, what it returns), and full type hints. A poorly described tool might as well not exist to the model.
2. **Every registered tool is processed on every call, whether or not that call needs it.** Only give the model tools relevant to the task at hand — unused tool definitions are pure, avoidable cost paid on every single request.
3. **Tool output size is never fully in your control — so bound it deliberately.** Return short, structured, actionable results and errors instead of dumping raw or unbounded output into context; this is usually the single biggest source of context bloat.

CampusX

4. **Never register tools that overlap in purpose.** If two tools could plausibly do the same job, the model has to spend extra reasoning to pick between them, with no guarantee it picks right — one clear tool beats several ambiguous ones.

Figure 34. Summary of tool calling as a context-engineering problem and the practices that keep tool definitions and outputs efficient.

CampusX

14 Tool Results

14.1 Tool Results and Why They Become a Problem

What is a tool result?

A tool is just a function you wrote. Its **tool result** is the value that function returns when it runs.

The flow in tool calling:

1. The LLM asks for a tool call.
2. Your **system** runs the function.
3. The result arrives at the **system side first**.
4. The system appends it to the messages and sends it on to the LLM.

Step 3 to step 4 is the gap where all the *prevention* techniques in this session work.

The two problems

Problem	Meaning
Verbose output	The function returns a very long result
Unnecessary information	Alongside the needed data, it returns extra fields the task never uses

Most APIs return **data plus metadata**. Applications often need only the data.

Demo: the GitHub user API. The task only needed four fields: username, followers, public repos, and account-creation date. The raw response also carried dozens of URL fields (followers, following, gists, subscriptions, and so on). The needed values were buried inside a much larger payload. The extra fields are not *wrong*; they are simply unneeded for this requirement.

Why this hurts most in agents

Agents call tools autonomously and repeatedly.

Suppose tool `ABC` returns 100 tokens, of which 30% is unwanted. If the agent calls it three times, junk lands in the context three times. Two things follow:

- **Pollution:** irrelevant information sits in the context.
- **Fullness:** a fuller window brings problems like the *lost-in-the-middle* effect.

CampusX

Results also **do not go away**. They stay in the message history and are re-sent on every later turn. In the weather demo below, the second question began with a context of 6,406 tokens *before* the model made any new call.

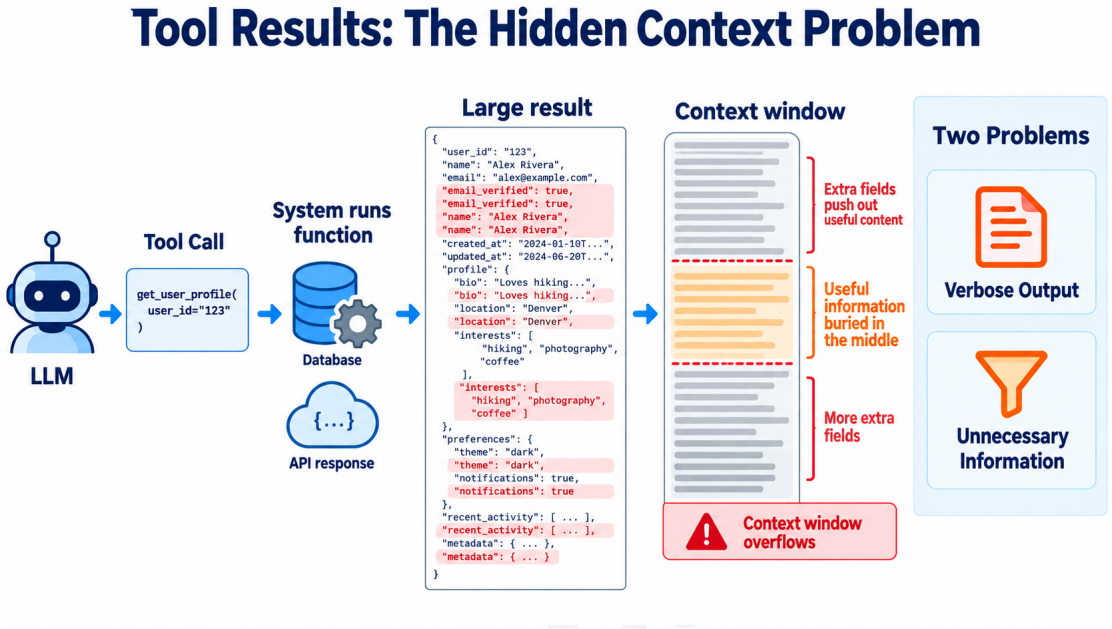

Figure 35. Tool results first reach the application, then enter the model context and remain in message history, causing repeated context growth.

14.2 Two Strategies: Prevention and Cure

Strategy	Idea	Where it acts
Prevention	Stop unnecessary content from ever entering the LLM's context	On the system side, before results are passed to the LLM
Cure	Compress what has already accumulated	After content is already in the context

Prevention comes first. Once content has reached the LLM's context, cure is the only option left.

> 🧠 **Analogy: The Athlete's Training**
> An athlete trains hard *before* the competition so mistakes are already fixed when the real event starts. Prevention works the same way: clean the tool output *before* it reaches the model, so the model only ever sees good context.

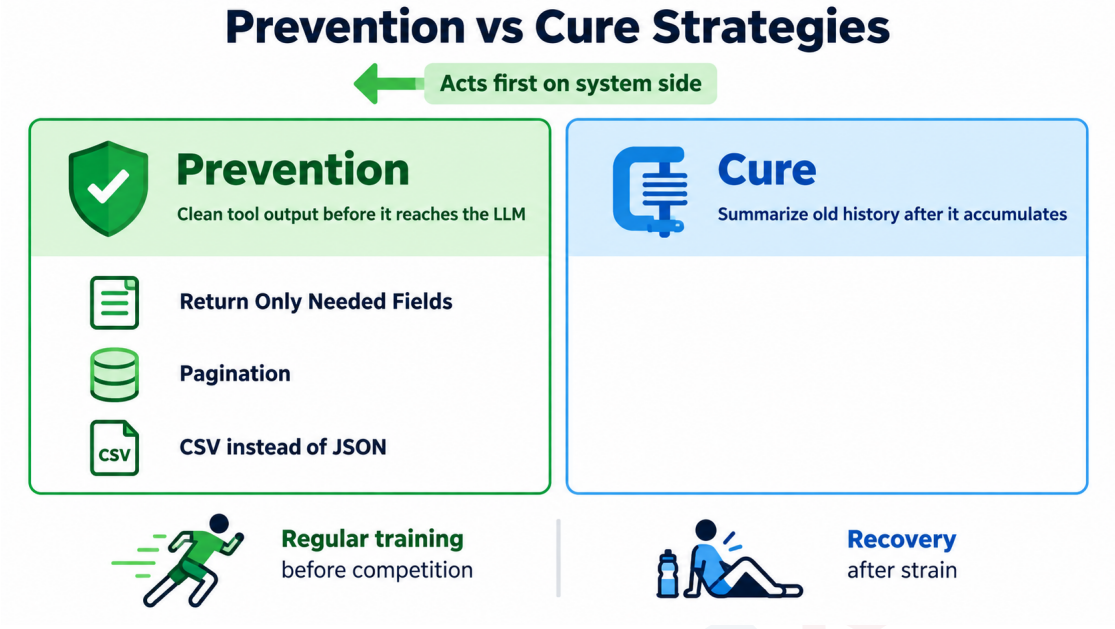

Figure 36. Prevention reduces tool-result size before it enters context, while cure compresses information that has already accumulated.

14.3 Prevention Method 1: Write the Function Properly

Why: Often the fault is in the function itself, not in the agent loop or the model.

What: Design each function so it returns *only* what the LLM needs for the task. Filter the API response inside the function instead of returning it whole.

Shared test harness

The same agent loop is used in every demo. It prints the **real token count** of the context on each turn, using Ollama's `prompt_eval_count`. (In the notebook it appears under the names `run_weather` and `run`.)

```python
import json, requests, ollama

MODEL = "gemma4:31b-cloud"    # Ollama cloud model used in the demos

def run_weather(question, tools, system_prompt, messages=None, max_turns=6):
    """Agent loop with token tracking. Pass `messages` back in to continue a
    ↪ chat."""
```

```
    registry = {f.__name__: f for f in tools}            # tool name ->
↪   function
    if messages is None:
        messages = [{"role": "system", "content": system_prompt}]
    messages.append({"role": "user", "content": question})

    for turn in range(1, max_turns + 1):
        response = ollama.chat(model=MODEL, messages=messages, tools=tools)
        size = response.prompt_eval_count or 0           # real tokens in
↪   context this turn
        print(f"turn {turn}: {size:>6} tokens in context")
        messages.append(response.message)

        if not response.message.tool_calls:              # no tool needed ->
         ↪  final answer
            print(f"\nANSWER: {response.message.content}\n")
            break

        for call in response.message.tool_calls:         # run every
         ↪  requested tool
            name, args = call.function.name, call.function.arguments
            result = registry[name](**args)
            messages.append({"role": "tool", "tool_name": name,
                             "content": json.dumps(result)})    # result
                              ↪  enters the context
    return messages
```

Example 1: Weather

Before. The function forwards the whole API response.

```
def get_weather(latitude: float, longitude: float):
    url = ("https://api.open-meteo.com/v1/forecast"
           f"?latitude={latitude}&longitude={longitude}"
           "&current_weather=true"
           "&hourly=temperature_2m,relative_humidity_2m,windspeed_10m"
           "&daily=temperature_2m_max,temperature_2m_min"
           "&timezone=auto&forecast_days=7")
    return requests.get(url, timeout=10).json()    # Bad: entire response goes
     ↪  to the LLM
```

The user asked only for the *current temperature* (31.2 °C). The response also held 168 hourly temperature values, 168 humidity values, and 168 wind-speed values. Measured with an external token counter, the payload was about 4,600 tokens.

After. Shape the result at the boundary.

CampusX

```python
def get_current_weather(latitude: float, longitude: float):
    """Get the current weather conditions for a location.

    Use this for the weather right now. For multi-day forecasts,
    a separate forecast tool should be used instead.

    Returns:
        A dict with temperature_c, windspeed_kmh and is_day. Nothing else.
    """
    url = ("https://api.open-meteo.com/v1/forecast"
           ↪ f"?latitude={latitude}&longitude={longitude}&current_weather=true")
    raw = requests.get(url, timeout=10).json()
    cw = raw["current_weather"]
    # Translation step: raw API response -> exactly what the assistant needs
    return {"temperature_c": cw["temperature"],
            "windspeed_kmh": cw["windspeed"],
            "is_day": cw["is_day"] == 1}
```

Example 2: Currency conversion

Before. `get_exchange_rates(base_currency)` returns the rates for *every* currency (162 rates) when the user wants one pair.

After. Take both currencies as arguments and return one rate.

```python
def get_exchange_rate(from_currency: str, to_currency: str):
    """Get the current exchange rate between two currencies.

    Returns:
        A dict with from_currency, to_currency, and rate. Nothing else.
    """
    raw = requests.get(f"https://open.er-api.com/v6/latest/{from_currency}",
                       timeout=10).json()
    return {"from_currency": from_currency,
            "to_currency": to_currency,
            "rate": raw["rates"][to_currency]}     # 162 rates in, 1 rate out
```

CampusX

Results

Demo	Before	After
Delhi temperature, then "What about Mumbai?"	6,373 → **12,719** tokens	280 → **384** tokens
USD → INR rate	**2,728** tokens	**259** tokens

> 💡 **Insight: Same Agent, Only the Function Changed**
> In both demos the agent, prompts, and model were identical. Only the function's return value changed, and context use dropped by roughly 90–97%. When context is bloated, look at the tool output *first*.

Note: Token counts depend on the tokenizer, so treat all figures in these notes as approximate.

14.4 Prevention Method 2: Pagination

Why: the tool returns a lot, but the question needs a slice

The function may be perfectly designed and still return too much, because the *dataset* is large.

Scenario: A support assistant fetches a customer's order history (240 orders, 2020 onward). Whether the full data is needed depends on the question:

User question	Data actually needed
What was my most recent purchase?	Only the newest few orders
When was my first purchase?	Only the oldest few orders
What was the last product I returned?	Possibly a scan through many orders
What is the total amount I have paid?	**All** orders

Why not write a custom function per question? Many similar or overlapping tools confuse the LLM, so this does not scale.

What: pagination

Pagination means giving long data to the LLM in **pages (slices)** instead of all at once. The LLM answers from the first page if it can, and asks for another page only if it cannot.

> 🧠 **Analogy: Chapatis, Not a Pile**
> You do not put ten chapatis on your plate at once. You take two, then ask for two more, then one more, until you are full. Pagination serves the LLM data the same way: as much as it needs, when it needs it.

CampusX

Think of the order history as a book. Instead of handing over the whole book, split it into pages (10 orders each) and hand over one page at a time.

How: the function

```python
def get_customer_orders(customer_id: str, page: int = 1, page_size: int =
 ↪  10):
    """Get one page of a customer's order history, newest orders first.

    Page 1 contains the most recent orders, which answer most support
    questions. Only request further pages if the question is about older
    orders and page 1 was not enough.

    Returns:
        A dict with orders (this page), page, total_orders,
 ↪  more_pages_available.
    """
    start = (page - 1) * page_size            # page 1 -> 0, page 2 -> 10,
 ↪  page 3 -> 20 ...
    chunk = ORDERS[start:start + page_size]   # slice out only this page's
 ↪  orders
    return {"orders": chunk,
            "page": page,
            "total_orders": len(ORDERS),                          # lets the
             ↪  LLM plan navigation
            "more_pages_available": start + page_size < len(ORDERS)}
```

Three design details:

- **Slice arithmetic:** `start = (page - 1) * page_size`. Page 1 gives orders 0–9, page 2 gives 10–19, and so on.
- **Navigation metadata:** `total_orders` and `more_pages_available` tell the LLM how far the data goes.
- **The docstring steers behaviour:** it says page 1 holds the newest orders and tells the LLM when to fetch more.

Results

The naive tool dumped all 240 orders. The paginated tool was used with the same agent.

Question	Tool calls (pages)	Context at final turn
Most recent order delivered? (naive, all 240 orders)	1 call, full dump	**22,387** tokens
Most recent order delivered? (paginated)	Page 1, `page_size=1`	**399** tokens
Date of the very first USB-C Hub order	Page 1, then page 24	~2,250 tokens
Courier for an old PixelView Monitor return	Page 1, then page 2	~2,260 tokens
Date of the customer's first purchase	Page 1, then page 24	~2,240 tokens

> 💡 **Insight: The LLM Navigates the Pages**
> For "first" or "oldest" questions the LLM looked at page 1, saw it held recent (2025) orders, and jumped straight to the **last page (24)** instead of reading 2, 3, 4, … in order. It could do this because the tool returns `total_orders` and sorts newest first. Well-designed pagination metadata lets the model plan its own reading. The LLM even chose `page_size=1` for the simplest question.

When *not* to use pagination

Pagination works only when the answer can be found from slices. If the task needs **all** the data, such as comparing sales of product A and product B or totalling all spending, slicing does not work. Those cases need a different technique, covered in the next lecture.

> ⚠ **Mentor's Rule: Analyse the Problem Before Choosing the Technique**
> Ask: *"If I give this data in slices instead of all at once, can the LLM still answer correctly?"* If yes, use pagination. If not, choose another technique.

14.5 Prevention Method 3: Structured Data as CSV

Why: tabular data wastes tokens as JSON

Many tool outputs are **naturally tabular**: a week of temperatures, stock prices, transaction lists. APIs usually return these as JSON, which repeats every key in every record.

Take a week of temperatures. In JSON, each of the seven records repeats the same key names:

```
[{"time": "2026-07-31T00:00", "temperature_c": 28.6},
 {"time": "2026-07-31T01:00", "temperature_c": 28.4}, ...]
```

In CSV, the key names appear **once**, as the header:

```
time,temperature_c
2026-07-31T00:00,28.6
2026-07-31T01:00,28.4
```

Scale it up: 1,000 records with 10 keys each means 10,000 repeated key names in JSON, against 10 header names once in CSV. That space is pure repetition. **Never pass tabular JSON straight to the LLM; convert it to CSV first.**

What: the two versions

Before (JSON records):

```python
def get_hourly_forecast_json(latitude: float, longitude: float):
    raw = requests.get(URL, timeout=10).json()      # URL requests hourly
 ↪  temperature, 7 days
    times = raw["hourly"]["time"]
    temps = raw["hourly"]["temperature_2m"]
    # the standard "list of records" shape, with keys repeated in every row
    return [{"time": t, "temperature_c": c} for t, c in zip(times, temps)]
```

After (CSV text):

```python
def get_hourly_forecast(latitude: float, longitude: float):
    """Get the 7-day hourly temperature forecast, as CSV (one row per hour).

    Returns:
        A dict with row_count and data: CSV text with columns
 ↪  time,temperature_c.
    """
    raw = requests.get(URL, timeout=10).json()
    times = raw["hourly"]["time"]                    # extract the time
 ↪  values
    temps = raw["hourly"]["temperature_2m"]          # extract the
 ↪  temperature values
    csv_text = "time,temperature_c\n" + "\n".join(   # header once, then one
 ↪  row per hour
        f"{t},{c}" for t, c in zip(times, temps))
    return {"row_count": len(times), "data": csv_text}
```

Test question: "Which days and time windows this week stay below 38 °C?" This needs *every* hourly value, so rows cannot be trimmed. Only the format can be made cheaper.

Format	Context at final turn
JSON records	**5,630** tokens
CSV	**4,149** tokens

That saves about 1,500 tokens (roughly 26%) for the same data and an equally good answer. Note that this function also extracted only the needed fields from the API response instead of returning the raw payload.

14.6 The Common Thread of Prevention

Method	Problem it fixes	Use when
Proper function	Extra fields and metadata returned	Always: return only what the task needs
Pagination	Long list, but the question needs a slice	Slices can answer the question
CSV instead of JSON	Repeated keys in tabular data	Output is naturally tabular and all rows are needed

> 💡 **Insight: One Idea Behind All Three**
> These are the three most common methods, and there are others. They all follow one principle: **design the tool so it never returns more context than the task requires.** Understand the requirement first, then shape the function's output to match it. Learn the principle, not just the three tricks.

Practical habit: measure token counts before and after a change, as these demos do. That is how you catch a bloated tool.

CampusX

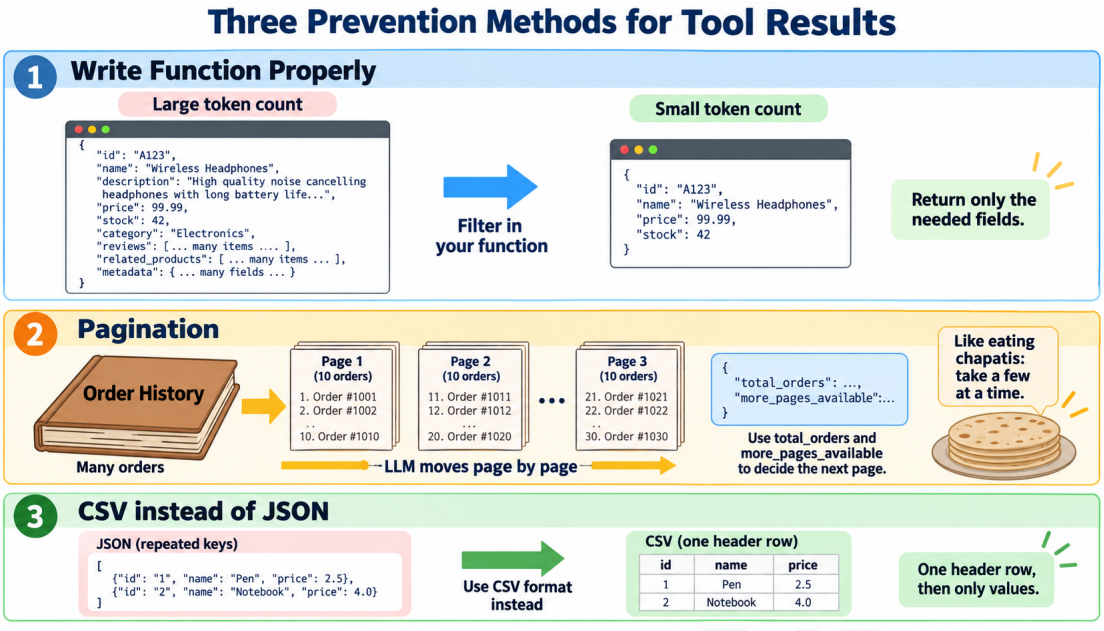

Figure 37. The three main prevention methods are shaping function output, paginating large result sets, and using CSV for naturally tabular data.

14.7 Cure: Summarizing Accumulated Context

Why cure is needed even with perfect tools

Assume every tool is well designed. Two facts remain:

- **Tool output can never be zero.** If it were, the tool would be pointless.
- **LLMs are stateless.** In a multi-turn conversation you must re-send the earlier messages, or the model forgets the topic.

So context grows over time. If each prompt-plus-tool-result exchange costs about 500 tokens, ten turns already cost about 5,000. Here the fault is neither the tool nor the context management. It is the nature of LLMs, and it calls for *cure*.

Why compaction is a necessity, not a nicety:

Situation	Consequence
Context window exceeded	The model behaves badly. It may reject the input, or push out the system instruction
Window nearly full (e.g. ~70%)	Lost-in-the-middle and hallucination problems

CampusX

What cure does

Summarize the old history, delete the original messages, and replace them with the summary.

Context slot	Before	After
System instruction	Yes	Yes
Tool definitions	Yes	Yes
Turns 1–10 (prompt, tool call, result, response)	~10,000 tokens	**One summary (~1,000 tokens)**
Latest prompts (11, 12)	Yes	Yes

Deleting is safe **only after** summarizing. Delete without summarizing and the earlier context is simply lost.

Demo: a "Make My Trip"-style travel assistant

The point of the demo is to show how components combine in a real system. The assistant has 10 tools:

Tool	Tool
`search_flights`	`search_hotels`
`get_forecast`	`search_trains`
`book_cab`	`search_restaurants`
`get_attractions`	`get_travel_insurance_quote`
`get_visa_info`	`convert_currency`

It uses **deferred loading** from the previous session. The agent starts with only a `search_tools` tool, which ranks tools by a hybrid score (0.6 × keyword match + 0.4 × embedding similarity), with tool embeddings computed once up front. Matching tools are then loaded into the conversation. The tools here are small, so deferred loading is not strictly necessary; it is used to show the pieces working together.

Baseline run, with no cure. Five prompts, for example:

1. Find Delhi→Jaipur flights that land before noon (a friend's mother can't do stairs).
2. Book the cheapest non-stop flight and find hotels under ₹6,000.
3. Pick the best-rated hotel with ground-floor rooms, and give the weather and packing advice.
4. Switch to a cheaper hotel?
5. Summarize what we discussed.

Observations:

CampusX

- **Context climbed steadily:** 2,177 → 3,873 → 4,743 → 4,937 → 5,198 tokens.
- **History gives the model memory.** The "ground floor" requirement was stated once, in prompt 1, yet later hotel searches still respected it. This is why history cannot simply be dropped.
- **Deferred loading behaved as expected.** The set of loaded tools grew (1 → 4 → 7 → 9), and an already-loaded tool was reused without another search.

The compaction code

The idea is to check the real token count of each call. When it crosses a threshold, summarize the old messages and rebuild the history.

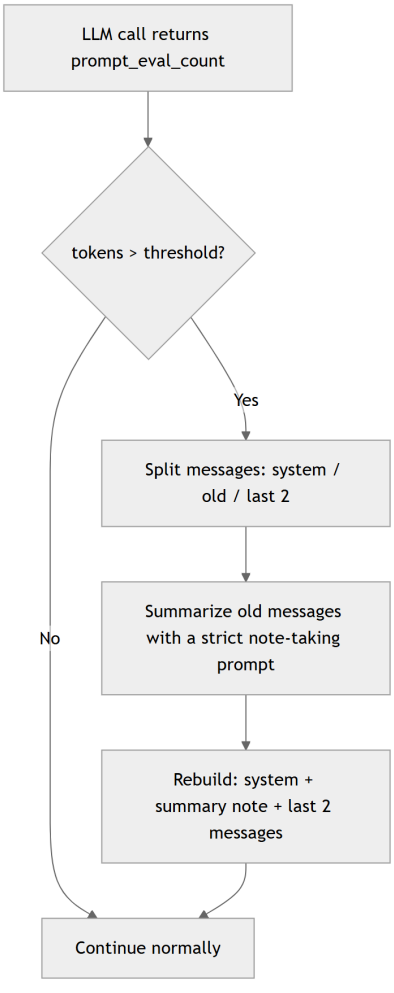

Figure 38. Diagram for 7. Cure: Summarizing Accumulated Context

```
COMPACT_THRESHOLD = 4000    # demo value: compact once the prompt exceeds
 ↪ 4,000 tokens

# ...inside run(), after the tool results for this turn have been appended:
if size > COMPACT_THRESHOLD:                        # size = real
 ↪ prompt_eval_count
    system_msg = messages[0]                        # keep the system prompt
 ↪ untouched
    old, recent = messages[1:-2], messages[-2:]     # keep the latest messages
 ↪ verbatim

    # Flatten the old history into "[role] content" lines for the summarizer
    old_text = "\n".join(
        f"[{m['role'] if isinstance(m, dict) else m.role}] "
        f"{m['content'] if isinstance(m, dict) else (m.content or '')}"
        for m in old)

    summary = ollama.chat(model=MODEL, messages=[
        {"role": "system", "content": COMPACT_PROMPT},    # the crucial
 ↪ instruction, below
        {"role": "user",   "content": old_text},
    ])

    note = {"role": "user",
            "content": f"[conversation so far]: {summary.message.content}"}
    messages = [system_msg, note] + recent          # summary replaces the old
 ↪ history
```

The summarizer's system prompt decides what survives, so it is the most important part:

```
COMPACT_PROMPT = """You are creating a memory note for an assistant whose
 ↪ earlier
conversation is about to be deleted. This note will be the ONLY record of
everything that happened. Anything you leave out is lost permanently.

Write the note in exactly this structure:

DECISIONS MADE:
- every choice finalised so far, with exact ID, name, price, date and time
  (e.g. flight FL-204, IndiGo, 08:15, Rs 4,300, 2026-12-20)

USER CONSTRAINTS AND PREFERENCES:
- every requirement the user stated, WITH THE REASON if given
  (e.g. 'hotel must have ground-floor rooms - mother cannot use stairs').
```

```
  Never keep a decision while dropping the reason behind it.

FACTS AND CONCLUSIONS:
- answers already worked out that may be needed again (weather, packing,
 ↪  timings)

STILL OPEN:
- anything discussed but not yet decided

Rules:
- Copy IDs, numbers, prices, dates and times EXACTLY. Never round or
 ↪  paraphrase.
- DROP entirely: raw tool outputs, and every option the user rejected or
 ↪  ignored.
- If a section has nothing, write 'none'.
- Use short bullet points, not paragraphs."""
```

> ⚠ **Mentor's Rule: You Decide What the Summary Keeps**
> A summarizer left to its own judgement will lose information. Spell out exactly what must be preserved: decisions, constraints and their reasons, conclusions, and open questions. A detailed summarizer prompt is what keeps the summary useful.

Results

Context size at the last turn of each prompt:

Stage	Without compaction	With compaction
Prompt 1 (flights)	2,177	2,185
Prompt 2 (booking + hotels)	3,873	3,842
Prompt 3 (hotel pick + weather)	4,743	4,118 → **1,162** after compaction → 2,296
Later prompts	4,937, then 5,198 after prompt 5	2,516 on the final-summary prompt

The 4,118-token context crossed the threshold, and **13 messages were replaced by one summary**, dropping the next turn to 1,162 tokens. The final summary still contained the exact flight (FL-216, SpiceJet, 06:45, ₹3,203), the hotel and its price, the ground-floor reason, and the packing advice. The detailed summarizer prompt prevented the key information from disappearing.

Side effect to notice: right after compaction the agent called `search_hotels` again, because the raw hotel results had been dropped from the summary. Compaction keeps *decisions*, not raw data, so a tool may need to be re-run.

Trade-offs of summarization

Trade-off	What happens	How to handle it
KV cache breaks	The model can reuse cached computation only while the start of the context is unchanged. Replacing history with a summary changes the context, so the cache is rebuilt from that point	Unavoidable if you want the model to behave well. Compaction happens only when the threshold is hit, so the cache still works between compactions
Lossy	Some information is always lost when compressing	Write a detailed summarizer prompt that lists what must be kept

14.8 Summary

	Prevention	Cure
Goal	Keep unneeded content out of the context	Compress what has accumulated
Where	Tool/function code, before results reach the LLM	Message history, once it grows too large
Main techniques	Return only needed fields; paginate; CSV instead of JSON	Summarize old history, delete the originals, replace with the summary
Watch out for	Pagination only suits slice-able questions	Lossy summaries; KV cache rebuilt at each compaction

Best-practice checklist

- Return only task-relevant fields from every tool.
- Do not forward raw API responses unless truly necessary.
- Filter out metadata before results reach the LLM.
- Paginate large datasets, and return `total` and `more_pages` metadata.
- Send naturally tabular data as CSV.
- Summarize old history in long-running agents, with a carefully written summarizer prompt.
- Track token usage and context growth.

CampusX

15 Programmatic Tool Calling

15.1 The Problem: Multi-Tool Loops Flood the Context

The scenario

An AI assistant answers questions from a company database (think of an e-commerce company). It has three tools:

Tool	Purpose
`get_all_customers()`	Returns the list of customers
`get_order(customer_id)`	Returns a customer's order details
`get_total_spend(customer_id)`	Returns how much a customer has spent so far

User query: *"Which customers have spent more than ₹1 lakh?"*

To answer it with ordinary tool calling:

1. Call `get_all_customers()`. With 500 customers, this returns 500 records.
2. Call `get_total_spend(customer_id)` **once per customer**, so 500 calls and 500 results.
3. Compare each spend with ₹1 lakh and list the matches.

Everything from steps 1 and 2 lands in the context window: one big list, 500 tool calls, and 500 results.

Why the LLM struggles

- The LLM ends up holding raw data (each customer and their spend), but the question requires **filtering and comparing hundreds of numbers**.
- LLMs are next-token predictors. Bulk comparison has to be done through reasoning tokens, which is slow and error-prone.
- With so much context, the model gets **lost in the context**. The result can be a wrong answer, a hallucinated answer, or no answer at all.

The demo (500 customers, 316 of them above ₹1 lakh)

Two tables were generated (customers and spending), with two tools (`get_all_customers` and `get_total_spend`) and the normal tool-calling loop on a Gemma 4 31B cloud model via Ollama. The step-by-step run:

CampusX

Step	What happened	Context size
LLM call 1	Asked to call `get_all_customers`	~171 tokens
LLM call 2	Asked to call `get_total_spend` for many customers at once	~9,600 tokens
LLM call 3	After all the spend results were appended	~31,000 tokens, **empty content**

The model returned neither an answer nor another tool call. The full agentic loop (up to 20 turns) also ended with an empty answer after about two minutes.

Switching to a larger model does not fix it. With GPT-OSS 120B the loop used all 20 iterations (about 40 seconds) and still produced no answer. Simply allowing more iterations is not a solution either: 500 customers might become 1 crore customers, and no one can run 1 crore agent iterations.

15.2 The Key Idea: Keep Intermediate Steps Away from the LLM

Solve the query the way a person would:

1. Get all customers.
2. Get sales for each customer.
3. Compare each customer's sales with ₹1 lakh.
4. Keep the customers above ₹1 lakh.

In the failing demo, **the output of every step was passed to the LLM**, which then had to process it all. The better approach:

- **Execute** steps 1 to 3 without ever showing their output to the LLM.
- Show the LLM **only the output of the final step**.

The LLM then spends its effort understanding a small final result, and the context stays clean.

The obstacle: queries are dynamic

Users can ask anything: customers who spent more than ₹1 lakh, sales between May and July, and so on. Each query has *different* intermediate steps. You cannot build a separate tool for every possible query.

The solution: code

Most such queries can be expressed as **code**: loops, filters, comparisons, date conditions. So:

CampusX

1. The LLM **writes code** for the query.
2. The code runs **outside the LLM**, in a sandbox.
3. Only the code's **output** goes back to the LLM.

15.3 What Is Programmatic Tool Calling?

> **Programmatic tool calling:** the model writes code to solve the problem, and that code calls the tools itself when needed. The code runs in a workspace outside the LLM, and only the final result is returned to the LLM.

Sandbox

A **sandbox** is an isolated, virtual place (a container-like space) where code can run without outside influence. You run the code there and carry the output out. In this lecture's implementation, the sandbox is simply **a tool**: a Python function that accepts code as input, runs it, and returns the output.

The flow

Traditional vs programmatic

	Traditional tool calling	Programmatic tool calling
Who orchestrates the steps	The LLM, one tool call at a time	Code written by the LLM, run outside it
What reaches the LLM	**Every** intermediate result	**Only** the final output
Context growth	Grows with every call	Stays small
Computation (filter, sort, sum)	Done by the LLM through reasoning tokens	Done exactly by code

Benefits: a smaller context, fewer tokens, lower cost, faster responses, less distraction for the model, and better scalability on multi-tool workflows.

CampusX

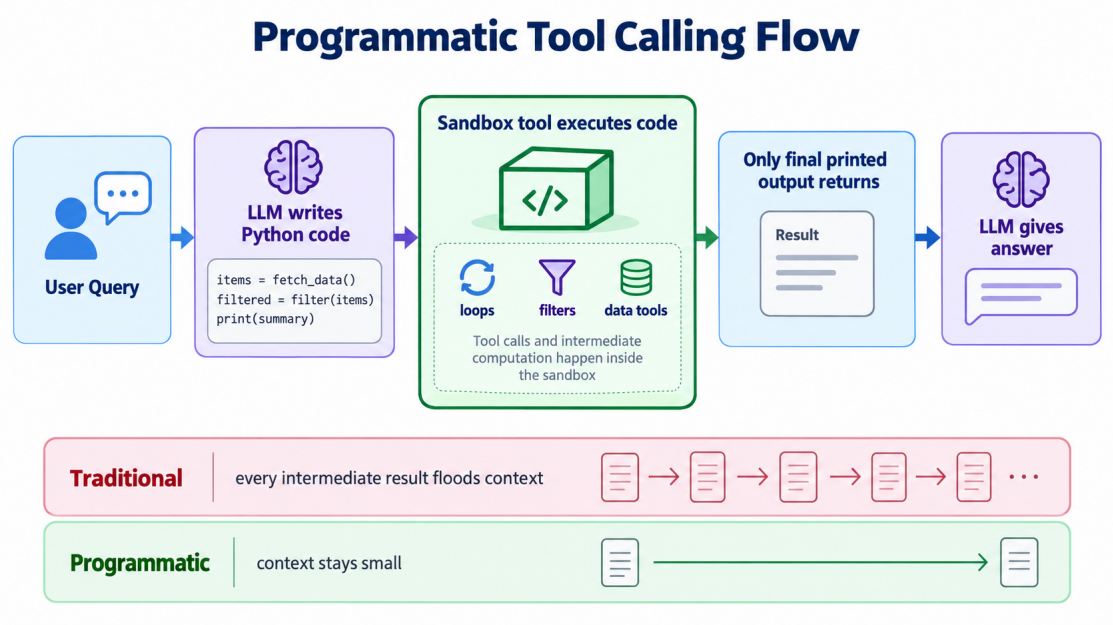

Figure 39. Programmatic tool calling lets the model write code that runs tools inside a sandbox and returns only the final computed result.

15.4 Building It Step by Step

Step 1: Create the sandbox tool

The tool receives a Python string, runs it with the data tools available inside, and returns whatever the code printed.

```python
import io
from contextlib import redirect_stdout

def run_code(code: str) -> str:
    """Run Python code to answer the question and return what it prints.

    Available inside the code: get_all_customers(),
    get_total_spend(customer_id).
    Print ONLY the final answer, never intermediate data.

    Args:
        code: The Python code to execute.

    Returns:
```

CampusX

```
        Everything the code printed.
    """
    workspace = {"get_all_customers": get_all_customers,       # the data
↪   tools live INSIDE
                 "get_total_spend": get_total_spend}           # the sandbox,
                  ↪  not in the LLM's tool list
    buffer = io.StringIO()                                     # temporary
     ↪  in-memory text buffer (fast)
    try:
        with redirect_stdout(buffer):                          # print() now
         ↪  writes into the buffer
            exec(code, workspace)                               # run the
             ↪  LLM-written code
        out = buffer.getvalue().strip()                        # collect
↪   everything it printed
        return out or "Code ran but printed nothing."
    except Exception as e:
        return f"Error: {e}"                                    # report
         ↪  failures as text
```

(Reconstructed from the lecture's walkthrough.)

Key points:

- `io.StringIO()` creates a temporary in-memory buffer, and `redirect_stdout` sends everything the code prints into it.
- **Only what the code prints returns to the LLM.** Intermediate variables stay inside the sandbox, so the code should print only the final result.
- The data tools sit in the sandbox's `workspace`. The LLM's own tool list contains just `run_code`.
- The docstring is the LLM's only description of what it can call inside the code, so it must list the available functions, their inputs, and their outputs.

> **⚠ Caution: `exec` Is Not Real Isolation**
> Redirecting output is not the same as isolating code. Running LLM-written code with `exec` in your own process is fine for a demo, but in production run it in a genuinely isolated environment such as a container or restricted runtime.

Step 2: Give the LLM only the sandbox tool

The tools list contains `run_code`. The LLM is asked the original question and, instead of calling the data tools one by one, replies with a call to `run_code`. Its `code` argument is Python it wrote itself, with no code supplied by us:

```python
# Illustrative shape of the code the LLM generated
customers = get_all_customers()
for c in customers:
    spend = get_total_spend(c["customer_id"])
    if spend > 100000:
        print(c["name"])
```

Step 3: Run the code and return the output

We execute the requested `run_code` call and append the output as an ordinary tool message. The message history is now just four messages: system, user, the assistant's `run_code` call, and the tool result. Calling the LLM again gives the final answer.

Results

Query	Approx. tokens
Which customers spent more than ₹1 lakh? (names)	**~1,850** (versus ~31,000 and no answer before)
Same, plus the count	~1,400 (316 customers)
Same, with each customer's spend and the total count	~5,000

Each time, the LLM rewrote its code to match the new question (adding a counter, then printing each customer's spend).

> 💡 **Insight: Cost Follows the Printed Output**
> The number of tool calls happening inside the sandbox no longer matters to the context. What you pay is the size of what the code *prints*. That is why the third query, which prints every customer's spend, costs more than the first two.

The same behaviour holds inside the agentic loop. The loop is unchanged: call the LLM, and if it requests a tool, run it and append the result. The only difference is that its tool list is `[run_code]`.

CampusX

15.5 When to Use It and When Not To

Use it when

Situation	Why it fits
Tool calls happen in a loop (one call per customer, 500 times)	A `for` loop in code replaces 500 LLM round-trips
Real computation over results (sorting, filtering, calculating)	Code is exact, and the LLM would need reasoning tokens
Large data in, small answer out	500 names go in, one short answer comes out
One tool's output feeds the next tool	e.g. regions → stores per region → sales per store (5 regions × 10 stores = many chained calls)

Avoid it when

Situation	Why not
Simple query, one call, one answer (e.g. today's weather)	Extra code adds complexity for no benefit
The LLM must read and interpret the data (e.g. read 5 customer complaints and find the common theme)	Understanding language cannot be done by ordinary code without an LLM, so the LLM has to read the data itself
Tools return short, simple answers	Nothing to optimise

> ⚠ **Mentor's Rule: Is the Data a Means or the Answer?**
> If the tool's data is only a **means** to an answer, meaning you must compute over it, use programmatic tool calling. If the tool's output **is** the answer and the LLM has to read and reason about it (reviews, complaints), do not use it.

15.6 Combining Deferred Loading and Programmatic Tool Calling

The two techniques solve different problems, so they can be used together:

Technique	Problem it solves
Deferred loading	Too many **tool definitions**
Programmatic tool calling	Too much **tool result** output

CampusX

The project: a hospital AI assistant

The assistant answers queries from a hospital database with four tables:

Table	Holds
Patients	ID, name, ward, age, days since admission, minutes until next medicine, pending bill, and a critical flag
Beds / wards	Total and occupied beds per ward (ICU, emergency, general, and so on)
Staff	Doctors and nurses per ward
Drugs / pharmacy	Medicines and units in stock

There are **22 tools**: patient IDs, patient details, search by name, next-medicine time, patient bill, ward and bed information, plus billing, pharmacy, and appointment tools. All 22 are registered in a registry.

The design: only two tools are exposed to the LLM

Exposed tool	Role
`search_tools`	**Deferred loading.** Finds and loads the tool needed for a single, targeted query. Uses the same hybrid ranking as before (60% keyword score, 40% embedding score), with tool embeddings computed once from each tool's name and docstring
`run_code`	**Programmatic tool calling.** The sandbox's workspace holds the full registry of all 22 tools, so generated code can call any of them

The agent loop handles the two branches differently:

```
# inside the agent loop, for each tool call the LLM makes
if name == "search_tools":
    found = search_tools(**args)      # deferred-loading branch: find and load
 ↪  the needed tool
    ...
elif name == "run_code":
    result = run_code(**args)         # programmatic branch: run the LLM's
 ↪  code in the sandbox
```

(Simplified from the lecture; the rest of the loop is unchanged.)

CampusX

The system prompt does the heavy lifting

The LLM writes code without seeing the data, so it has no way to know the table names, column names, or what a column means. The system prompt therefore has these parts:

Section	Content
Role	What the assistant is for
Data dictionary	Each table, its columns, and the meaning of each column (for example, the column giving minutes until a patient's next medicine)
Tool usage strategy	It has only two tools. A *single targeted query* goes through `search_tools` and a direct call. A *bulk or aggregate query* goes through `run_code` with Python
Coding guidelines	How to write the code
Output constraints and tone	How to present answers

> 💡 **Insight: Code Generation Needs the Schema**
> When the LLM writes code against your data, it can only use what you told it. A good data dictionary and an explicit rule for *which tool to use when* are what make the setup reliable.

Results

Query	Route	Result	Approx. tokens
Pending bill for Vikrant Iyer	Search, then direct call	₹0 (checked against the table)	~1,330
Pending bill for Karan Sharma	Search, then direct call	Patient 2003 (matches the table)	small
Is insulin in stock?	Search, then pharmacy stock tool	Yes, 45 units	~1,640
Patients with pending bill over ₹40,000 (names and count)	`run_code`	99 patients	under ~2,000
Average age of all patients	`run_code`	47.87 years	~1,350
Patients aged 12 to 18	`run_code`	30 (checked manually)	under ~2,000
Patients per ward	`run_code`	Per-ward counts	~1,370
Patients needing medicine within 60 minutes	`run_code`	151	under ~2,000
Patients in critical condition	`run_code`	43 of 1,316	under ~2,000

CampusX

Query	Route	Result	Approx. tokens
Male and female patients per ward	`run_code`	Reported that gender is not recorded and gave per-ward totals instead	under ~2,000

Points to take from the results:

- **Simple lookups and heavy aggregations both work.** Simple queries use the search route, and aggregate queries use the sandbox.
- **No query exceeded about 2,000 tokens**, even with 22 tools and a long system prompt.
- The last query asked for data that does not exist. The model said so instead of inventing an answer, which is the behaviour you want.

15.7 Recap

Idea	Summary
Problem	Repeated, dependent tool calls push every intermediate result into the context and overwhelm the LLM
Fix	The LLM writes code; the code runs in a sandbox and calls the tools; only the final output returns to the LLM
Sandbox	A tool that takes code, runs it, and returns what it printed
Best for	Loops, computation over results, big-data-in/small-answer-out, chained tools
Not for	Simple one-call queries, or tasks where the LLM must read the data itself
Combining	Deferred loading handles tool definitions; programmatic tool calling handles tool results

CampusX

16 Deciding Which Technique to Use

16.1 Module Recap

The module's starting point: **tools are a context problem.** A tool can occupy the context window in three ways: its **definition**, its **call**, and its **result**. Each session added techniques for one of these:

Technique	Problem it addresses	Covered in
Deferred loading	Too many tool definitions in every request	Session 3
Writing tools well (good names, good descriptions)	Model picks the wrong tool or fills its parameters wrongly	Earlier in the module
Shaping tool output (return only needed fields), **pagination**, **JSON → CSV**	Tool results that are verbose or full of unneeded data	Session 4
Compaction (summarization)	Old results piling up over a long session	Session 4
Programmatic tool calling	Many intermediate steps, or computation over tool results	Session 5

That is a lot of techniques, which raises an obvious worry: must all of them be applied everywhere? No. This session gives the method for choosing.

16.2 The Method: Two Diagnostic Questions

You do not memorise a mapping from scenario to technique. You answer two questions about your own system:

1. **What is filling up the context window?** What *kind* of data is it?
2. **Where is that data coming from?** What is its *source*?

Once both are answered, the right technique, and where to apply it, follows almost automatically.

> 💡 **Insight: Diagnose First, Then Pick the Fix**
> Start with a plain, unoptimised version. See where it goes wrong, and improve on the basis of that specific weakness. Reach for deferred loading because you *see* a flood of unused definitions, and for programmatic tool calling because you *see* intermediate steps piling up. No technique is a default.

CampusX

16.3 Diagnosis to Technique

Case 1: Many tools, but each task uses only a few

Diagnosis: The data filling the context is **tool definitions**. You pay for every tool's definition in every API call, even though most are never used. **Fix: Deferred loading.** Load only the tools that are needed, when they are needed.

Case 2: The model picks the wrong tool, or uses the right tool wrongly

Diagnosis: The tool call itself is flawed. Either the wrong tool is chosen, or the right tool is called with bad parameters (a missing value, a wrong data type). For example, a tool with three parameters is called with only two filled in. **Cause:** The tool has not been written well. **Fix: Write the tool properly.** Give it a clear name and a good description. Modern frameworks convert the description into the JSON schema that the LLM reads when deciding which tool to use and how. A good description gives a good schema, and a good schema gives correct calls.

> ⚠ **Mentor's Rule: Write the Schema of Your Key Tools Yourself**
> For the tools doing the heavy lifting, such as the search tool in deferred loading or the sandbox tool in programmatic tool calling, design and write the schema yourself. You can use ChatGPT or another AI to help, but the design is yours. The whole approach depends on these tools being used correctly.

Case 3: A single tool returns far too much data

Diagnosis: The tool definition and the call are both fine. The **tool result** is what floods the context. **Fix: Tool-result handling.**

- Shape the function so it returns only what is needed.
- Use pagination when the question needs only a slice.
- Convert tabular JSON to CSV.

Case 4: A long session where results pile up over time

Diagnosis: No single result is the problem. Over many iterations, the accumulated history is consuming the window. **Fix: Compaction.** Summarize the old context and replace the original messages with the summary, so the context window is largely freed while the important information survives.

Case 5: The task needs computation over the data (filtering, math, logic)

Diagnosis: The tool does not return the final answer. The LLM must work on its output to reach one, and the intermediate steps clutter the context. **Fix: Programmatic tool calling.** The filtering, math, and logic run in the sandbox, so the context is not polluted with intermediate data.

Decision flow

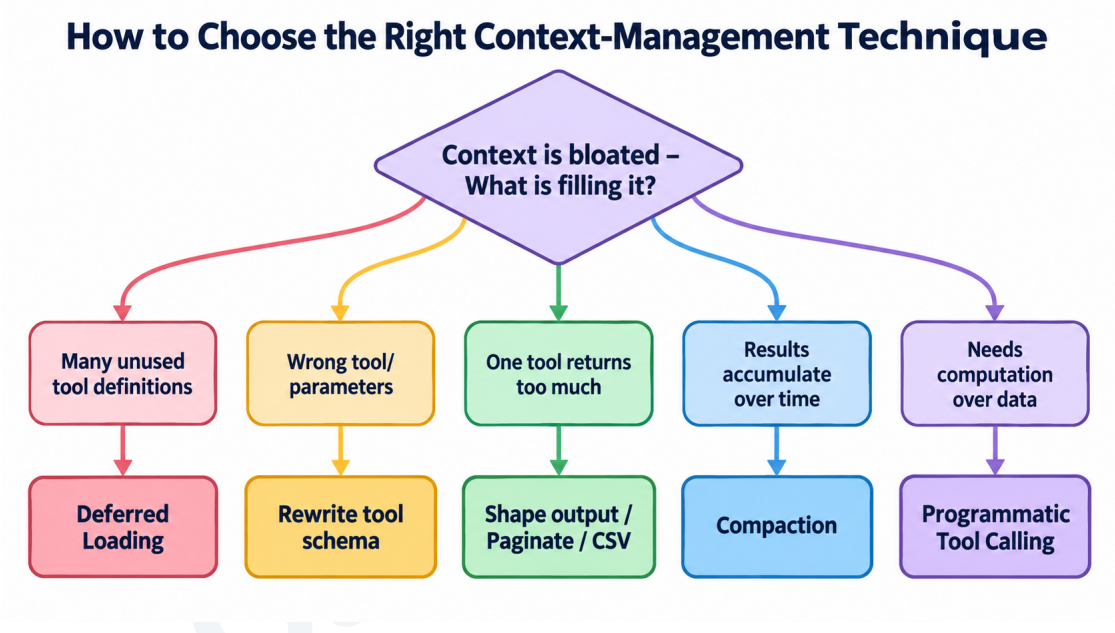

Figure 40. A context-management decision flow maps each source of context growth to the appropriate technique.

16.4 Combining Techniques

Real systems often have more than one problem at once. The same method applies: identify each problem and combine the matching fixes.

Your situation	Combination
Many tools **and** computation over their results	Deferred loading + programmatic tool calling (the hospital assistant from the previous session)
Many tools **and** very verbose results	Deferred loading + tool-result handling (shape output, pagination, CSV)
Very big results **and** a very long session (for example, an application like Claude Code)	Tool-result handling + compaction

CampusX

Your situation	Combination
Many tools, big results, **and** a long session	Deferred loading + tool-result handling + compaction
Many tools, computation over results, and a long agent loop	Deferred loading + programmatic tool calling + well-written tools + compaction in the loop

The last row is a particularly strong combination:

- Deferred loading keeps definitions out of the context until needed.
- Programmatic tool calling keeps intermediate data in the sandbox.
- Well-written tools make sure the model uses them correctly.
- Compaction keeps a long-running loop from filling up.

There is no hard-and-fast rule about what to use. Understand your problem, identify what fills the context and where it comes from, and decide what to use, and what to leave out, on that basis.

16.5 Crux of the Module

Whenever you feel a tool-calling system needs context management:

1. Find **what** is filling the context window (definitions, calls, or results).
2. Find **where** it is coming from.
3. Apply the technique that removes that specific source of bloat.

With those two answers, choosing and combining techniques is not difficult.

16.6 Golden Rules

1. **Diagnose before you optimise.** Identify what is filling the context (definitions, calls, or results) and where it comes from. Choose the technique from the diagnosis, not from habit.
2. **Pay only for the tools you use.** Load tool definitions on demand instead of sending every definition on every request.
3. **Return only what the task needs.** Shape tool output at the function boundary; do not forward raw API responses by default.
4. **Send data in the leanest form that keeps its meaning.** Paginate when a slice answers the question, and send tabular data as CSV rather than repeated-key JSON.
5. **Do computation in code and reasoning in the model.** If data is only a means to an answer (filtering, sums, comparisons), compute it outside the model and return only the result. If the model must read and interpret the data itself, let it.

CampusX

6. **Clear tool names and descriptions are the model's interface.** Wrong tool choices and bad parameters usually trace back to how the tool is written.
7. **Prevent first, compact second.** Content that never enters the context costs nothing. Summaries are lossy, so state exactly what must survive.
8. **Measure.** Compare token counts before and after every change.

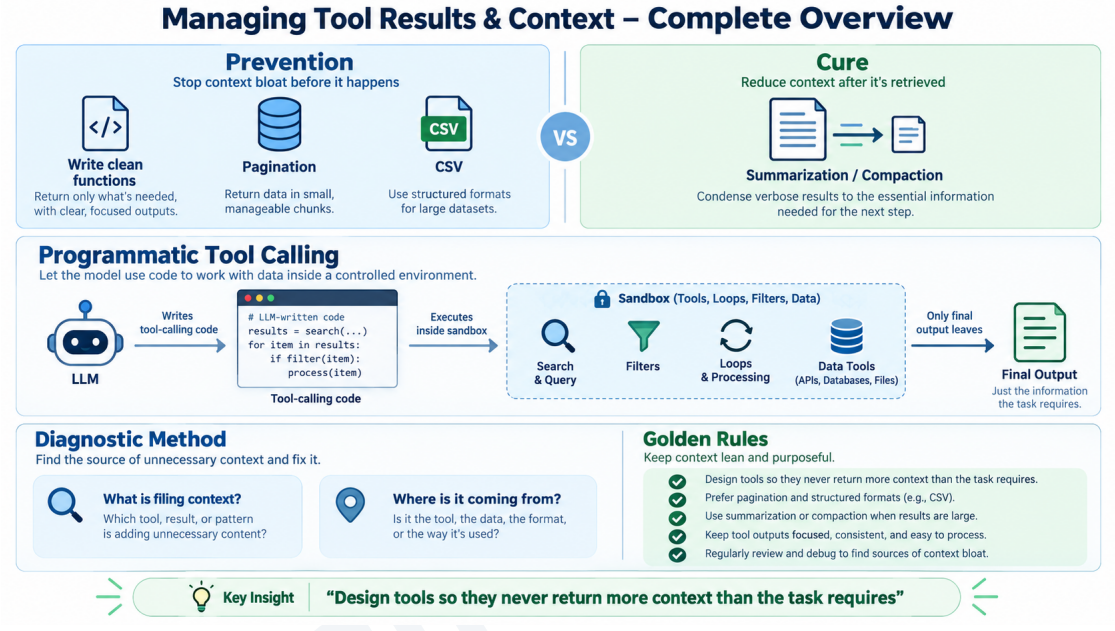

Figure 41. Complete overview of managing tool definitions, tool results, accumulated history, and computation while preserving a focused context window.

CampusX

COURSE MODULE

Model Context Protocol (MCP)

17 What Is MCP and the Context Problem It Can Cause

17.1 Module Overview and Prerequisite

This module covers **MCP (Model Context Protocol)** from a *context-engineering* perspective. It follows the tools module, since MCP is the next thing most people learn after tools.

Across the module:

1. What MCP is and what problem it solves
2. What problem MCP creates for the context window
3. How to prevent or fix that problem

Prerequisite	What You Need
MCP fundamentals	The first five videos of the CampusX *Model Context Protocol* playlist ("MCP Trilogy", Part 1): what MCP is, why it exists, its architecture, and how to use it

17.2 Where Tool Calling Stops Being Enough

Tool calling is the right choice for customised or simple tasks, and the industry recommends it for them. Examples:

- An app where the user pastes text bills and the AI returns the total.
- A blog tool that takes a topic, searches the web, gathers sources, and compiles an article.

It becomes painful when the application must connect to an **external service**: something outside the LLM's world, such as Gmail, Google Drive, Slack, or Notion. Take an app where the user manages Gmail purely through prompts (write, fetch, delete emails). Three problems appear.

Problem 1: A custom function for every action

Each action needs its own function. For Gmail that means separate functions to read, write, fetch a specific email, delete, and move emails between folders. Google Drive then needs its own full set.

The result is a large number of functions to manage and a large amount of **glue code**.

Problem 2: Authentication changes break every tool

Suppose you built 20 Gmail tools and Gmail later changes how you authenticate. Every function now fails to authenticate, so none of them can do any work. You must rewrite all of them for the new method, and the application is down until you finish.

Problem 3: Every LLM wants tools in a different format

Each LLM provider has its own schema for accepting tools. Moving from OpenAI/Ollama-style models to Anthropic models usually means rewriting the tool definitions.

```
# Anthropic style: arguments are described under `input_schema`
{
  "name": "get_weather",
  "description": "Get the current weather for a city",
  "input_schema": {...}
}

# OpenAI / Ollama style: wrapped in type + function, arguments under
 ↪  `parameters`
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get the current weather for a city",
    "parameters": {...}
  }
}
```

Frameworks such as LangChain and LangGraph can hide part of this by converting tools automatically. The underlying format mismatch is still there, and if you change LLMs again a few months later you face the same rewrite.

The root cause

These are only the top three problems. They all come from two differences:

- **Every external service has its own authentication method** (Slack, Drive, and Notion all differ).
- **Every LLM has its own format for accepting tools.**

What is missing is a **standard rule or protocol** for connecting any AI application to any external service.

> 🧠 **Analogy — HTTPS and a Common Language**
> HTTPS is a standard protocol, so you are not tied to one browser: Chrome, Firefox, or any other browser can talk to any website. Similarly, the world has thousands of languages, but a shared standard like English lets most people communicate. The idea was to build the same kind of standard for AI applications and external services.

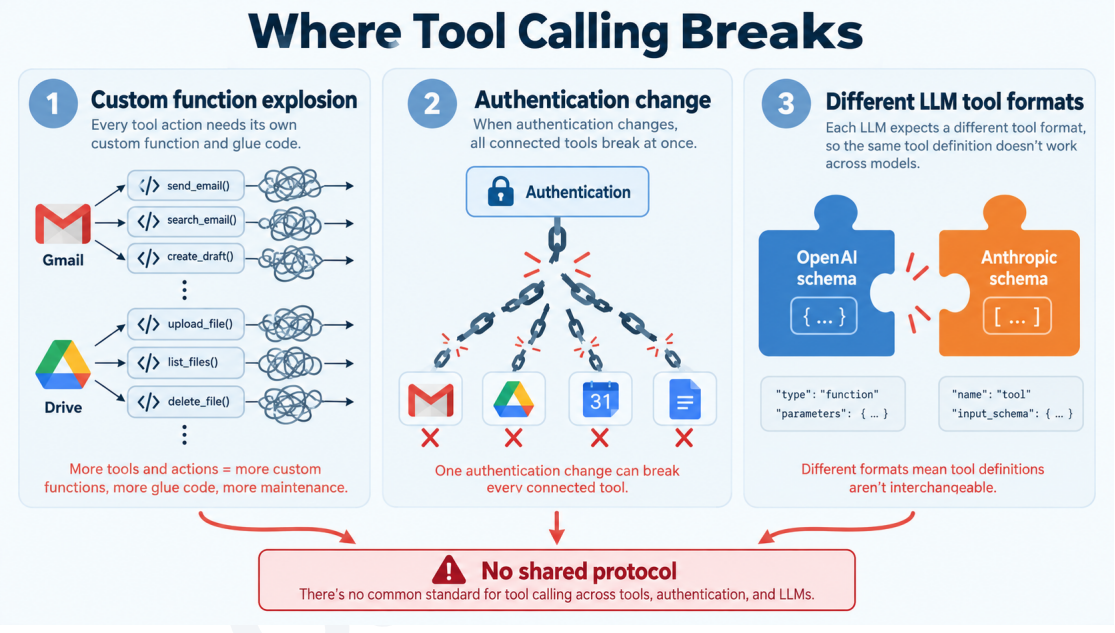

Figure 42. Without a shared protocol, every AI application needs custom integrations for each external service and model-specific tool format.

17.3 What Is MCP?

Protocol first

A **protocol** is an agreed set of rules for communication between two parties.

> 🧠 **Analogy — MOU and Traffic Rules**
> Two companies entering a partnership sign an MOU that fixes what the partnership is, what each side gains, the risks, and how they will work together. That MOU is their protocol.
> Traffic police and drivers are another pair of parties. Both know what a red light means and why a seat belt matters, so they communicate through shared traffic rules. Those rules are the protocol.

Model Context Protocol

MCP is an **open standard** that gives AI applications a common way to connect with external tools, services, and data sources.

Think of it as a **bridge** between the AI application and the external service provider. The two sides speak the same language, so the connection is seamless and no custom integration is needed for each service.

> 💡 **Insight — MCP Connects the Application, Not the Model**
> MCP connects the **AI application** to external services, not the **AI model** itself. The model is Claude; the application is Claude Code. This distinction recurs throughout the module.

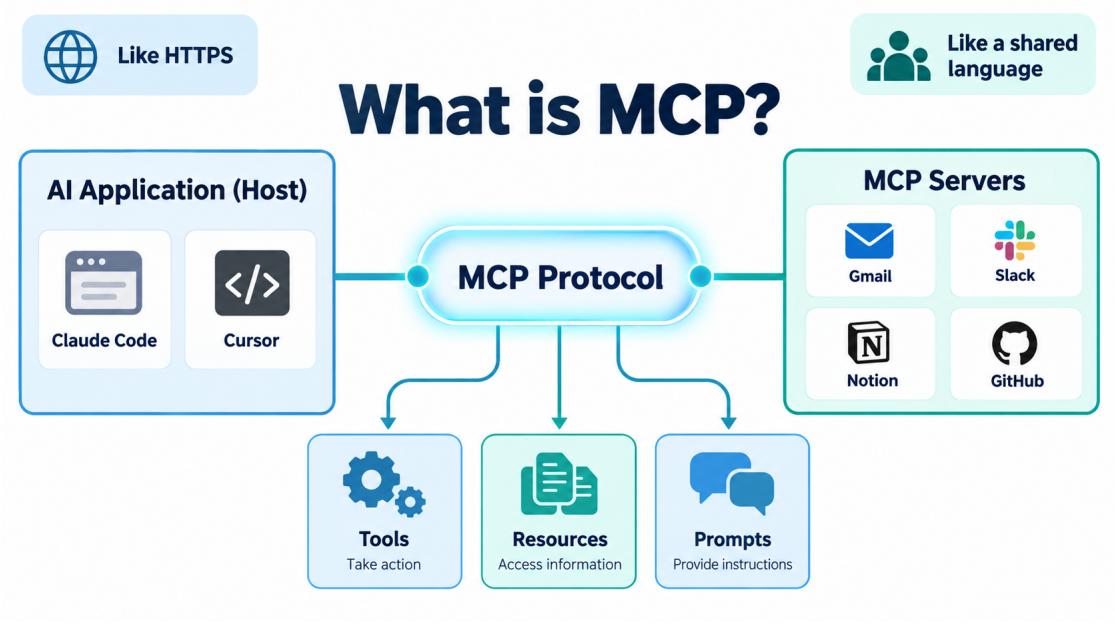

Figure 43. MCP is a shared protocol through which an AI application connects to servers that expose tools, resources, and prompts.

17.4 How MCP Is Implemented: The Two Parties

Party 1: AI applications

Anthropic introduced MCP and then opened it up for everyone. Major AI providers adopted it, so most AI applications are now **MCP-enabled**, meaning they can speak the MCP language. Examples include Claude Code, Cursor, Codex, and the Claude and ChatGPT chatbots.

Party 2: External service providers and MCP servers

Providers such as GitHub, Slack, Notion, Google Drive, and Gmail build their own servers that follow the protocol. These are **MCP servers**.

An MCP server is a repository, built and maintained by the service provider, that exposes everything you need to work with that service through MCP. It can contain:

Element	What It Is	Purpose
Tools	Functions built by the provider, e.g. `create_draft` in Gmail	Perform **actions** in the service
Resources	Static textual information about the service, e.g. its policies or version	Gather **information** only; they cannot perform actions
Prompts	Pre-defined prompts offered by the server	Ready-made prompts the application can use

In practice, most servers expose **only tools**. Very few also provide resources.

Seeing servers in practice

In Claude Desktop, the connectors directory lists MCP servers that real providers have published (Google Drive, Slack, Google Calendar, Gmail, Canva, and others). Connecting one takes a single click on **Connect**.

- **Google Drive server:** 7 tools (create file, download file content, get file metadata, list recent files, get file permissions, read file content, search files). No resources.
- **Slack server:** 11 tools (send message, read channel, read thread, read user profile, and others). Tools only.
- **Gmail server:** tools such as create draft, get thread, list threads, and list labels.

Demo: Gmail through MCP

With the Gmail MCP server connected in Claude Desktop, the prompt asked Claude to draft an email saying the user was unwell.

1. Claude proposed two versions of the email, and the user picked option B.
2. Claude found a suitable tool, `create_draft`, and asked permission to use it.
3. After approval, the draft appeared in the Gmail drafts folder.

The point of the demo is that the user never wrote that tool. **Gmail's MCP server supplied it**, and the user only connected and used it.

CampusX

17.5 What MCP Solves

Returning to the original problem, AI applications and external services could not connect cleanly because there was no shared standard. MCP fixes this in two ways:

- **Seamless integration:** the AI application and the external service both speak MCP, so they connect easily.
- **No manual tool writing:** the provider builds the tools (and resources) in its server. You connect to the server and use them directly instead of writing and maintaining custom functions and glue code.

Integration is solved. The next lecture examines how MCP can still become a problem for the context window.

17.6 AI vs. AI Application

MCP connects **AI applications**, so the difference between the two matters here.

Layer	What It Is	Role	Examples
AI	The LLM: the "brain"	Takes input, processes it, generates output; also decides *which* tool, resource, or prompt to use	Claude, Gemini, and GPT models
AI application	The whole program built around the LLM	Takes user input, passes tools to the LLM, **runs** tools when the LLM asks, and displays the output	ChatGPT and Gemini chatbots, Claude Desktop, Claude Code

You never talk to an LLM directly; you always go through an application.

> 🧠 **Analogy — The Electric Motor**
> An electric motor is like the AI. Engineers build fans, coolers, and water pumps around it. Each finished product is like an AI application: the same core engine, wrapped in different engineering.

In MCP's technical vocabulary:

- The **AI application** is the **host**.
- The **external service provider** is the **server**.
- **MCP** is the protocol that lets a host connect seamlessly to any server and use the server's elements (tools, resources, prompts) directly.

CampusX

17.7 The MCP Connection Flow

Knowing exactly *when* MCP turns into a context problem makes the fixes easy to understand later. The flow has four steps, and **all of them happen automatically** when the host application starts.

1. **Request:** the host sends an MCP request to the server. It carries the host's version, its operating system, and the intent to connect.
2. **Handshake:** the server answers, and the connection between host and server is established.
3. **Discovery:** the host asks the server for everything it offers: all **tools, resources, and prompts**.
4. **Transfer:** the server sends them all back to the host.

Only the first-time connection is manual (for example, clicking **Connect** on a connector in Claude Desktop). After that, every time the app opens it reconnects and re-fetches everything in the background. With the Gmail server connected, Claude Desktop pulls in Gmail's 11 tools each time it starts.

At this point the elements sit with the **host**. The **LLM knows nothing about them yet**.

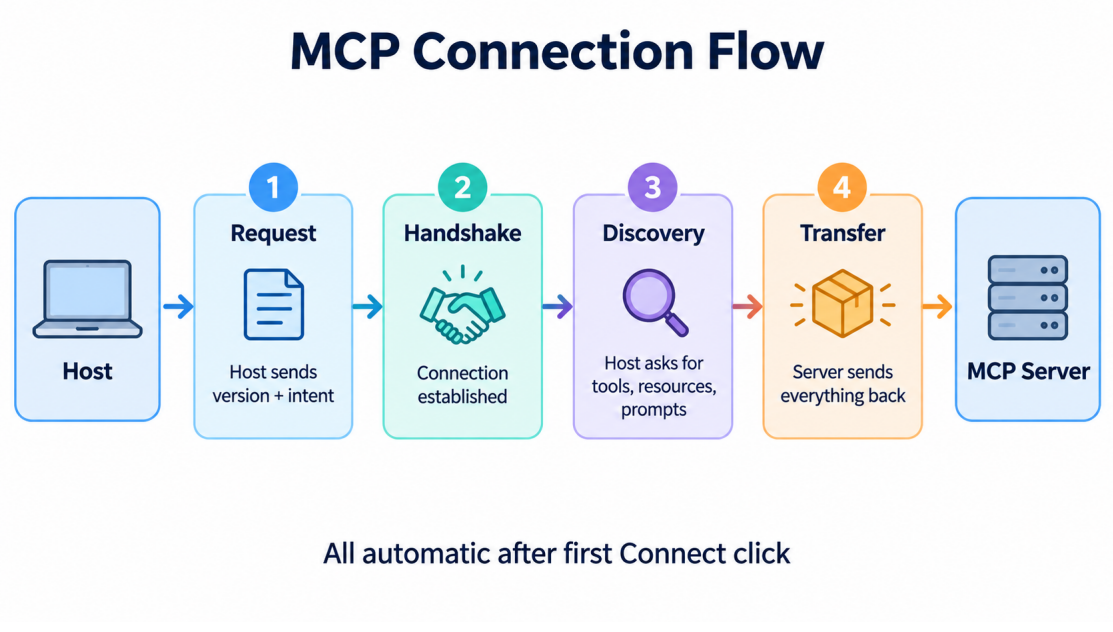

Figure 44. The MCP connection flow establishes a connection, discovers server capabilities, and transfers them to the host before anything reaches the LLM.

18 How MCP Can Become a Context Problem

The LLM is the decision-maker: it chooses which tool to call, which resource to read, and which prompt to use. The host only executes. An LLM cannot choose a tool it has never heard of, so the host must tell it what exists.

The host does this by **dumping the definitions of all tools, resources, and prompts into the LLM's context window** when it calls the LLM. The elements were sitting in the host's "warehouse"; the user's first prompt sends the whole warehouse into the context window.

> 💡 **Insight — The Protocol Isn't the Problem; the Dump Is**
> MCP itself only connects the host to the server. The trouble starts in the hop from host to LLM, where everything gets loaded into the context window. **Context pollution and context dilution begin at this step**, and the techniques in the rest of the module all act on it.

How big is the cost?

Large servers are expensive:

Server	Tools	Approx. Tokens
GitHub (official)	~40	~17,600
Notion (official)	15–19	~17,163
Chrome DevTools, Playwright, Supabase	13–26	~18,000 each
Gmail	11	~2,000

Example: connect GitHub, Chrome DevTools, and Gmail to Claude Desktop, then type only "write a sick-leave email for me." Before that first word is processed, **66 tools and about 37,000 tokens** already occupy the context window, even though the task needed exactly **one** Gmail tool.

Real-world case: when Anthropic launched MCP, its own blog reported that with a 200K context window, about **132K tokens were consumed by tool definitions alone**.

CampusX

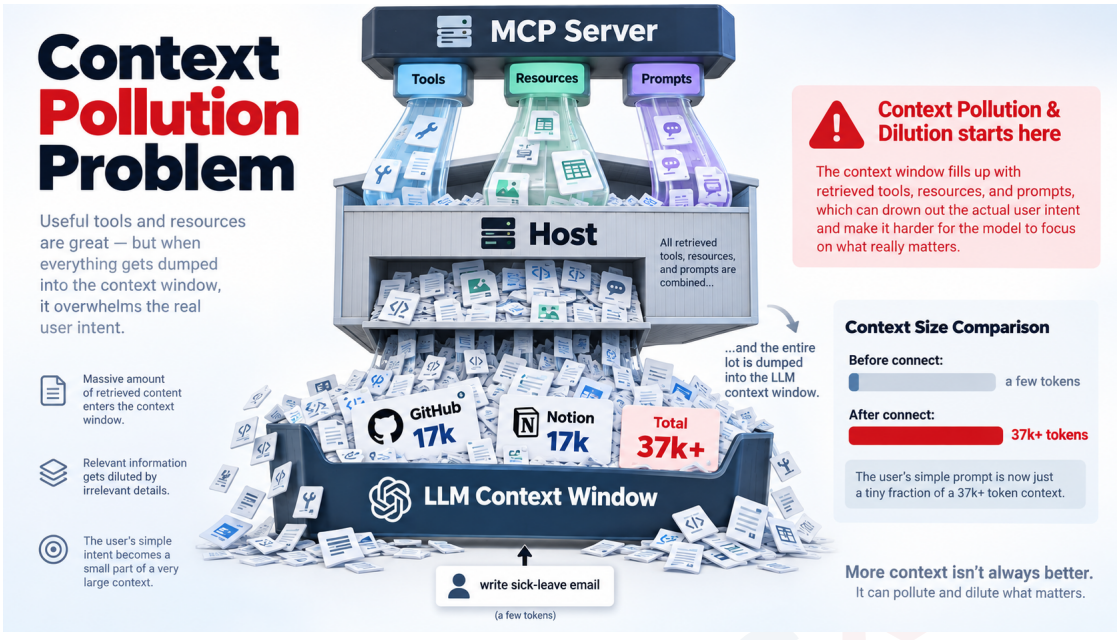

Figure 45. Context pollution begins when the host sends every discovered MCP element to the model, even when the current request needs only a small subset.

18.1 Demo: Watching the Dump Happen in Code

To make the flow visible step by step, the notebook itself plays the role of the **host** and connects to two MCP servers:

- **File system server:** a real server that lets an AI work with files and folders in a directory you authorize.
- **"Everything" server:** a dummy server Anthropic built for testing. It has pre-filled **tools, resources, and prompts**, so it shows how much a full server can add.

Most servers (around 90%) expose **only tools**. The file system server is one of them.

> 🧠 **Analogy — Making an Egg Sandwich**
> The connection involves several round trips (request, response, connect, ask, receive). Waiting idly through each is like standing still while the bread toasts. Better to go and fetch the egg in the meantime. That is why the demo uses Python's `async`/`await`: the program does not sit blocked while the server responds.

CampusX

1 File system server

Its 14 tools: `read_file` (being deprecated in favour of `read_text_file`), `read_media_file`, `read_multiple_files`, `write_file`, `edit_file`, `create_directory`, `list_directory`, `list_directory_with_sizes`, `directory_tree`, `move_file`, `search_files`, `get_file_info`, and `list_allowed_directories`.

Step 1: connect and fetch the tools.

```
async def get_tools():
    # Which server to launch, and which directory it may touch
    params = StdioServerParameters(
        command="npx",
        args=["-y", "@modelcontextprotocol/server-filesystem", "."]  # "." =
 ↪  current directory only
    )
    errlog = open("server_errors.log", "w")  # keep server noise out of the
 ↪  notebook

    # Launch the server and open two channels: read (server → host), write
     ↪  (host → server)
    async with stdio_client(params, errlog=errlog) as (read, write):
        async with ClientSession(read, write) as session:   # session built
         ↪  on those channels
            await session.initialize()                      # the actual
             ↪  handshake
            result = await session.list_tools()             # discovery: ask
 ↪  for all tools
            tools = [{"name": t.name,
                      "description": t.description,
                      "input_schema": t.input_schema} for t in result.tools]
    errlog.close()
    return tools

tools = await get_tools()   # the tools now sit with the HOST, not the LLM
```

`stdio_client` only builds the bridge between host and server. The handshake itself is `session.initialize()`.

Step 2: pass the tools to the LLM, first attempt.

Passing the raw MCP tool definitions straight to the LLM fails. The model claims it cannot see function names and invents a tool called `list_files`, which does not exist. Calling it returns a "tool not found" error. A tiny input token count (204) was the clue that the tool definitions were not being read properly.

The cause is the same format problem from earlier in these notes: MCP tool definitions are not in the format the LLM API expects. **Every host has to convert MCP tools into the LLM's format behind the scenes.**

```
# Convert MCP tools into the format Ollama accepts
ollama_tools = [
    {"type": "function",
     "function": {"name": t["name"],
                  "description": t["description"],
                  "parameters": t["input_schema"]}}    # MCP's input_schema →
                   ↪  Ollama's parameters
    for t in tools
]
```

Step 3: let the LLM choose, let the host execute.

With the converted tools, the LLM answers the question "What files are in the current folder?" with a call to `list_directory` and `path="."`. The LLM only *chooses* the tool. The host runs it through the server:

```
async def call_mcp_tool(tool_name, arguments):
    # ... same connection boilerplate as get_tools() ...
            await session.initialize()
            result = await session.call_tool(tool_name, arguments)   # host
 ↪  executes the LLM's choice
    return result.content[0].text
```

The notebook reconnects inside this function only because each notebook cell runs independently. In an application like Claude Desktop, the connection stays open during the chat.

Step 4: look at the token cost.

- "List files" cost **1,248 input tokens**. All 14 tool definitions were sent although only `list_directory` was needed.
- "Read the contents of a file" with a smaller model (Qwen3 8B) cost **1,562 input tokens**, again with all 14 tools attached.
- This is one server. **Ten servers of similar size would cost roughly 13,000 tokens** before the user says anything.

2 The "Everything" server: tools, resources, and prompts

The server exposes three things, and the notebook fetches each with its own call:

Element	Fetch the list	Fetch the content
Tools	`session.list_tools()`	(definitions come with the list)
Resources	`session.list_resources()`	`session.read_resource(uri)`
Prompts	`session.list_prompts()`	`session.get_prompt(name, arguments)`

Resources are static documents (for example `demo://resource/static/document/architecture.md`), and reading one returns a **lot** of text.

To simulate what a host does, the notebook builds one "dump" from everything the server offers:

```
# Tools → LLM tool format (as before)
ollama_tools = [...]

# Resources: read the ACTUAL content of every resource; this is what really
 ↪  bloats context
resource_dump = "=== RESOURCES ===\n"
for r in resources_result.resources:
    content = await session.read_resource(r.uri)
    resource_dump += f"--- {r.name} ({r.uri})
 ↪  ---\n{content.contents[0].text}\n\n"

# Prompts: name and description
prompt_dump = "=== PROMPTS ===\n"
for p in prompts_result.prompts:
    prompt_dump += f"{p.name}: {p.description}\n"
```

Then it measures how many input tokens the model consumes for the same one-word message under three conditions:

```
def count_input_tokens(messages, tools=None):
    response = ollama.chat(model=MODEL, messages=messages, tools=tools)
    return response.prompt_eval_count       # tokens the model actually
     ↪  consumed as input

user_msg = [{"role": "user", "content": "HI"}]

baseline        = count_input_tokens(user_msg)                          # 1.
 ↪  plain "HI"
with_tools      = count_input_tokens(user_msg, tools=ollama_tools)      # 2.
 ↪  + MCP tools
polluted_msg    = [{"role": "system", "content": resource_dump + "\n" +
 ↪  prompt_dump}] + user_msg
```

```
with_everything = count_input_tokens(polluted_msg, tools=ollama_tools)   # 3.
 ↪  + resources + prompts
```

Input	Input Tokens
"HI" alone	68
"HI" + MCP tools	709
"HI" + tools + resources + prompts	**8,040**

The user typed two letters, yet the model processed about 8,000 tokens. The tools added roughly 640 tokens; the resources and prompts added the remaining ~7,300. Resources are full documents, so they weigh far more than tool definitions. The demo deliberately dumps everything to make the effect visible, but it is exactly the pattern the host follows when it loads a server's context into the LLM.

Bottom line: MCP's job ends once the host is connected and holds the server's elements. The context problem appears when the host dumps them into the LLM's context window. The next lecture covers how to prevent or fix it.

19 Solving the Context Problem in MCP

MCP is not what pollutes the context window. Its only job is **seamless communication** between host and server, and it does that well: the Gmail server let Claude use Gmail's tools with no tool-writing, and the notebook pulled tools, resources, and prompts from a server with a few calls.

The pollution starts in the **host**, when it dumps everything it received into the LLM's context window. So the fix also belongs in the host: **design the application so that MCP elements never reach the LLM unfiltered.**

> 💡 **Insight — MCP Adds No New Kind of Problem**
> The same problem has appeared all through this course: the application holds everything, yet the LLM should see only what is relevant. Textual data is handled by **RAG**, and tools by **deferred loading, programmatic tool calling, and pagination**. MCP just adds one more *source* of tools and text, so nothing new is needed here.

Split what arrives from the server into two categories and apply the matching technique:

Category	MCP Elements	Technique from Earlier Modules
Tools	Server tools	Deferred loading, programmatic tool calling, pagination
Textual data	Resources, prompts	RAG

Together these form a **filtering layer** between the host and the LLM.

> 🧠 **Analogy — The Watchman**
> The filtering layer is a watchman standing between the application and the LLM. Everything can pile up in the application, but the watchman lets through only the tools and text the current query needs.

Modern AI applications are expected to **support MCP**. What matters is the engineering around it: the host keeps all the data, but the data leaving the host for the LLM is always filtered.

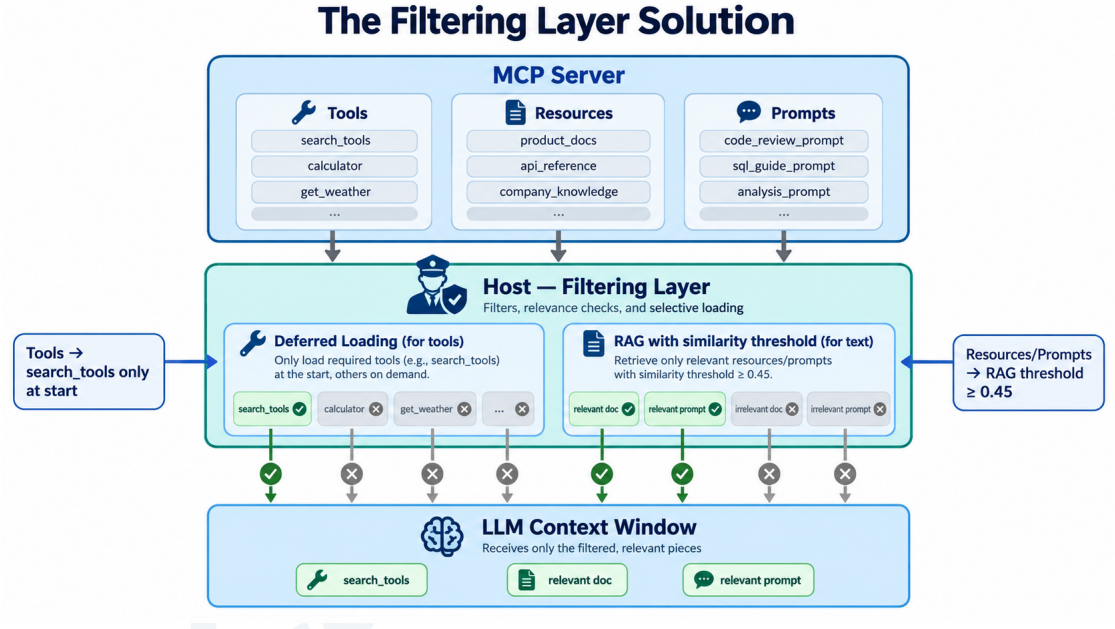

Figure 46. A filtering layer keeps all MCP data in the host while sending only query-relevant tools and text to the LLM.

19.1 Demo: Building the Filtering Layer

The demo reuses the "Everything" server from the previous lecture. Sending everything to the LLM cost roughly 8,000+ tokens there. Now the goal is to send only what each query needs. The notebook uses Ollama with a Gemma chat model and EmbeddingGemma for embeddings.

1 Ingest everything into the host

The host connects, then fetches all tools (13), resources (7), and prompts (4). At this point all of it sits in the host, and **nothing has gone to the LLM**. The filtering layer will now sit in between.

2 Tools: deferred loading

This is the same code as the tools module.

1. Embed each tool's **name + cleaned description** once.
2. Build a `search_tools` function that ranks tools by a **hybrid score** (keyword match + embedding similarity).
3. At the start, give the LLM **only the `search_tools` schema**. Matching tools are added to the callable list only after the LLM searches for them.

```
def search_tools(query, max_results=3):
    query_emb = get_embedding(query, task_type="search_query")
    query_words = set(query.lower().split())

    scored = []
    for t in MCP_TOOLS:
        text = (t["name"] + " " + clean_docstring(t["description"])).lower()
        keyword_score = len(query_words & set(text.split())) /
↪   max(len(query_words), 1)
        embedding_score = cosine_similarity(query_emb,
↪   TOOL_EMBEDDINGS[t["name"]])
        scored.append((0.6 * keyword_score + 0.4 * embedding_score,
↪   t["name"]))   # hybrid score

    scored.sort(reverse=True)
    return [name for _, name in scored[:max_results]]    # names only; full
↪   schemas load later
```

3 Textual data: RAG with a similarity threshold

Resources and prompts go into an in-memory "database": a list where each entry stores its **text and its embedding**. The developer also added two policy documents (refund and shipping) so retrieval could be tested, because the dummy server has nothing meaningful to query.

> 💡 **Insight — You Often Have to Add Your Own Knowledge**
> In real projects, the server rarely supplies everything the application needs. Some knowledge has to be added from your side, exactly as the two policy documents were here.

Threshold instead of top-k. Normal retrieval returns the top k chunks by similarity. If even the best match is poor (all scores near zero), top-k still returns k irrelevant chunks and dumps them into the context window. Using a **minimum similarity score** returns only chunks that clear the bar, and returns **nothing** when nothing is relevant.

```python
KNOWLEDGE_THRESHOLD = 0.45   # tunable: raise or lower it during testing

def search_knowledge(query, threshold=KNOWLEDGE_THRESHOLD):
    qv = get_embedding(query, task_type="search_query")
    scored = sorted(((cosine_similarity(qv, e["vec"]), e) for e in
 ↪  knowledge),
                    key=lambda x: x[0], reverse=True)
    return [(s, e) for s, e in scored if s >= threshold]   # only chunks
     ↪  above the bar
```

For “Can I return an item I bought two weeks ago?”, only the refund policy comes back (score about 0.5), and no unrelated chunks.

4 System prompt: route by input type

The system prompt has the LLM classify every message into one of three cases:

Case	Input Type	Behaviour
1	Knowledge / policy question	Answer only from the `<reference_information>` block; add no facts of its own
2	Action or live data	Starts with only `search_tools`: search first, then call the tool that gets loaded; never act from memory
3	General conversation	Answer directly; no search and no reference information

The rules also forbid inventing tool names, calling tools that are not currently available, and making up facts.

CampusX

5 The agent loop

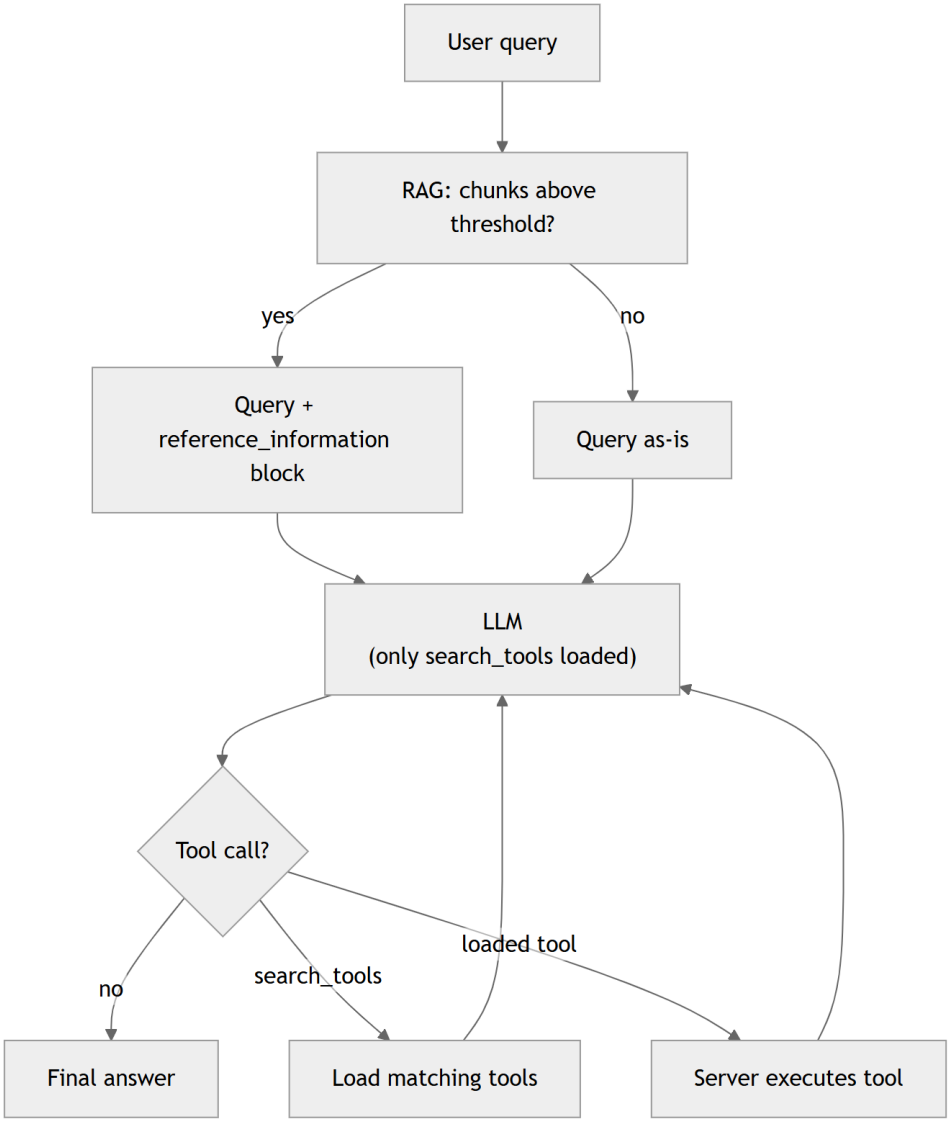

Figure 47. Diagram for 10. Demo: Building the Filtering Layer

```python
async def run_agent(user_query, model=CHAT_MODEL, max_steps=6):
    # 1. RAG: keep only knowledge above the threshold
    hits = search_knowledge(user_query)
    if hits:
        block = "\n\n".join(f"[{e['source']}] {e['text']}" for _, e in hits)
```

```
        user_content =
↪  f"<reference_information>\n{block}\n</reference_information>\n\n{user_query}"
    else:
        user_content = user_query          # nothing relevant: send the query
↪  unchanged

    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_content}]
    available = [SEARCH_TOOL]              # the ONLY tool the LLM sees at
↪  the start

    async with stdio_client(SERVER, errlog=errlog) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            for step in range(max_steps):  # hard cap on the loop
                resp = ollama.chat(model=model, messages=messages,
↪  tools=available)
                msg = resp.message
                messages.append(msg)

                if not msg.tool_calls:     # no tool needed: this is the final
                 ↪  answer
                    return msg.content

                available_names = {t["function"]["name"] for t in available}
                for tc in msg.tool_calls:
                    name, args = tc.function.name, tc.function.arguments

                    if name == "search_tools":                 # deferred
                     ↪  loading step
                        found = search_tools(args["query"],
↪  args.get("max_results", 3))
                        for n in found:
                            if n not in available_names:
                                available.append(ollama_def(n))    # callable
↪  from the next turn
                        out = f"Loaded (now callable): {', '.join(found)}"

                    elif name in available_names:              # a loaded
                     ↪  tool
                        res = await session.call_tool(name, args)  # the
↪  SERVER runs it
                        out = res.content[0].text if res.content else ""

                    else:                                      # the LLM
                     ↪  invented a tool name
```

```
                        out = f"ERROR: '{name}' is not loaded. Use
↪  search_tools first."

                    messages.append({"role": "tool", "content": out})
    return "Stopped: reached max steps."
```

In an MCP setup, **the server runs the tools**. The host only tells the server which tool to run with which arguments, receives the output, and passes it to the LLM. The guard for invented tool names also matters: the LLM gets a correction telling it to search first instead of a silent failure.

6 Results

Query	What Ran
"Can I return an item I bought two weeks ago?"	RAG retrieved the refund policy; no tool call; **589 tokens**
"Hi, how are you today?"	RAG skipped, no tool call; direct reply
"Run a long background operation and tell me when it completes"	`search_tools` loaded the matching tool and the server ran it (about 10 seconds, 5 progress steps); **1,060 tokens**
"Does your shipping policy allow delivery to a PO box?"	RAG retrieved the shipping policy; no tool call; answer: no
"How does this system work internally?"	RAG retrieved the relevant resources; answered from them

Across the five runs, input stayed between roughly **530 and 1,200 tokens**, compared with **8,000+** when everything is dumped. For each prompt, neither all the tools nor all the text reached the LLM, only what that query required.

Long-running agents. Context still accumulates over many turns. Apply the same **summarization** technique from the tools module to compress the accumulated context over time.

19.2 Golden Rules

1. **MCP connects; it does not manage context.** Its job ends when the host holds the server's elements. What reaches the LLM is an application design decision.
2. **Holding is not loading.** The host may keep every tool, resource, and prompt; the LLM should see only what the current query needs.
3. **Put a filtering layer between host and LLM.** Filter tools with deferred loading (or similar) and textual data with RAG.
4. **Use a relevance threshold, not a fixed top-k.** Returning nothing is better than injecting the "best of the bad" matches.

CampusX

5. **Route by input type.** Small talk needs no retrieval and no tools; policy questions need retrieved text; actions need tools.
6. **Never let the LLM call a tool that is not loaded.** Guard against invented tool names and tell the model to search first.
7. **Measure input tokens before and after.** Compare token counts with and without the filtering layer to confirm the pollution is actually gone.

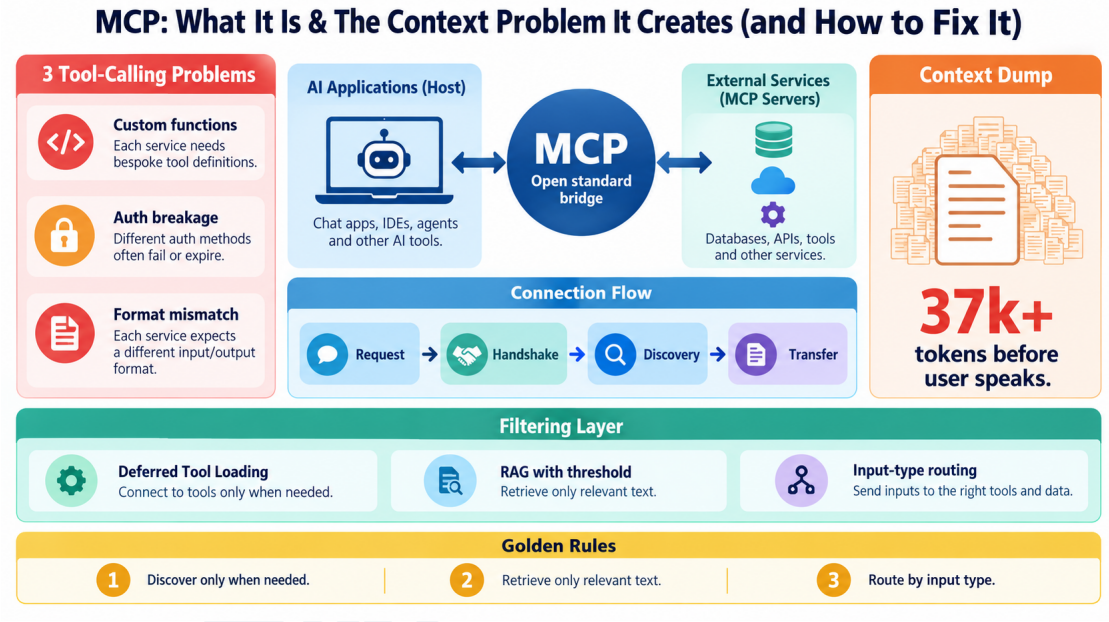

Figure 48. Complete MCP context-management overview: connect servers to the host, filter tools and text, and load only what each query needs.

CampusX

COURSE MODULE

Memory Management

20 Intro to Memory in LLM Apps

20.1 Why Memories Are Needed

LLMs are inherently **stateless** — an LLM is fundamentally just a *next-token predictor*, with no built-in mechanism to remember anything across interactions. Left on its own, an LLM cannot recall who you are, what you discussed earlier, or what you prefer.

Yet real AI applications (ChatGPT, Claude, Gemini, etc.) clearly *do* remember things about you — your name, your ongoing projects, your preferences. This apparent memory is not a property of the LLM itself; it is engineered around the LLM through a **memory system**.

> 🧠 **Analogy — LLMs as Gajni**
> LLMs are like the movie character Gajni, who forgets everything. Without an external memory system, every conversation starts from a blank slate.

20.2 What Is Memory?

Memory is a system that:

1. **Stores** information about a particular user, and
2. **Retrieves** the relevant piece of that stored information back into the LLM's context window exactly when it's needed.

Without memory, an AI application degrades into a plain question-and-answer board — no continuity, no recall of prior conversation, no personalization.

Memory systems are built **outside the LLM**. The LLM itself never "remembers" anything; the surrounding engineering pulls the right stored information into its context window at the right time.

CampusX

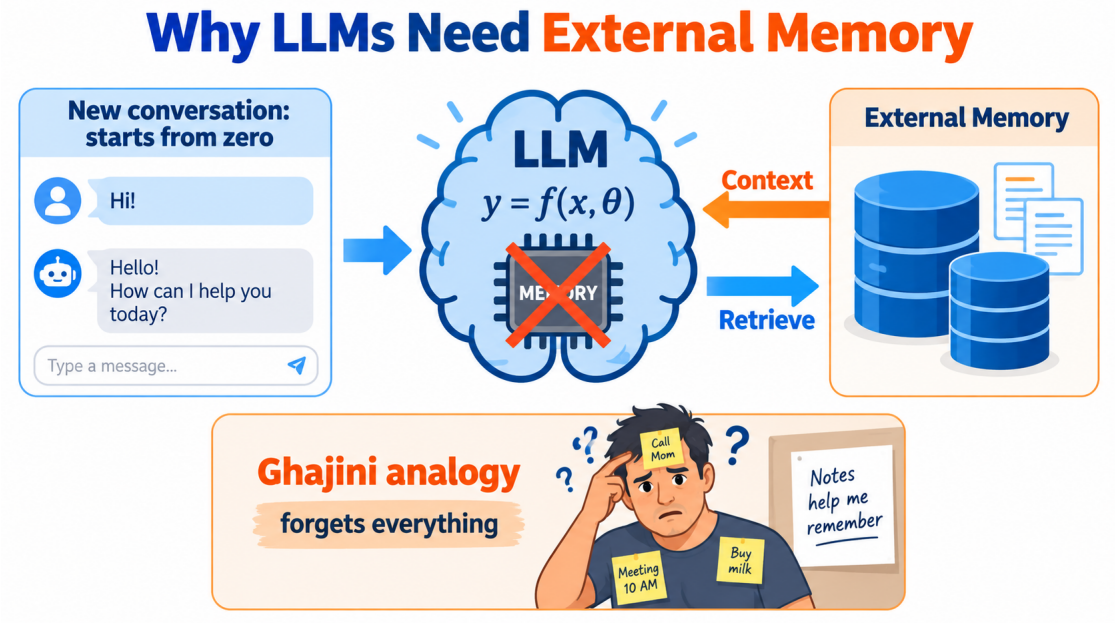

Figure 49. Because LLMs are stateless, an external memory layer must store durable information and retrieve it into the context window when needed.

20.3 Why Memory Matters: Core Benefits

Benefit	What It Means
Recognition	The application knows who you are (role, projects, hobbies) without you repeating yourself every session — typically populated once you log in with an identity like email.
Continuity & saved effort	You can leave a conversation mid-task (e.g., partway through discussing a dense research paper) and resume days later without re-explaining context or what's already been covered.
Personalization	Over time, the application learns your preferred answer format, level of detail, example style, and code style, and adapts its responses accordingly.
Improved output quality over time	As preferences and patterns accumulate, generated content (ideas, code, answers) becomes cleaner and more targeted to the user.

CampusX

> 💡 **Insight — Why Switching LLMs Feels Bad**
> When you switch to a new LLM/tool, it initially feels worse — wrong format, over-verbose code, answers that don't match your style. This isn't the model being weaker; it's that the new system's memory has no information about you yet. Use it for a while and it starts to feel as good as your old tool, because your information gets fed into its memory over time.

20.4 Why Memory *Management* Matters

Knowing memory is necessary isn't enough — **poorly managed memory actively harms an AI application**. Two properties of memory drive this:

Memory grows over time

Like human memory, the more a user interacts with the system, the more information accumulates. Unmanaged growth leads to:

- **Context bloat / context rot** — too much (possibly irrelevant) information gets pushed into the context window, degrading quality even though technically "useful" data exists somewhere in it.

Memory comes in different types

Different types of memory hold different kinds of information (covered in detail in the next lecture). If these types aren't handled properly, you risk:

- **Context loss** — information the user *did* provide never actually reaches the LLM at the moment it's needed.

> ⚠ **Mentor's Rule — The Cost of Context Loss**
> Example: a user tells your AI application several times that they prefer R over Python. If this preference isn't properly stored and retrieved, the application keeps generating Python code anyway — despite having been told the correct preference repeatedly. The eventual outcome is that the user abandons the application for a competitor. Reliable memory retrieval is not optional polish; it directly affects user retention.

Retrieval itself is a hard problem

Since memories live outside the LLM, getting them *into* the context window at the right time and in the right amount is a non-trivial engineering task:

- Pull in **too much** → context bloat / context rot.
- Pull in **too little** → context loss.

CampusX

Balancing this trade-off is a core challenge of memory management.

20.5 Module Roadmap

The module builds and manages memory in two stages:

1. **Manually** — building memory storage, saving memories, retrieving them, and injecting them into the context window, all without libraries.
2. **With libraries** — implementing the same capabilities using existing tooling, along with the practical problems that come up and how to engineer around them.

21 LLMs Don't Have Memory - So How Do They Remember?

To understand why memory has to be engineered *around* an LLM rather than living inside it, it helps to look at what an LLM actually is at inference time.

The Inference Equation

An LLM at inference is just a **parameterized mathematical function**:

$$y = f(x, \theta)$$

A parameterized function is one whose output depends not only on its input, but also on a set of parameters. The classic example is linear regression: $y = mx + b$, where m (slope) and b (intercept) are parameters learned from data during training — not supplied by the user at prediction time.

An LLM follows the same pattern, just at a vastly larger scale:

Symbol	Meaning
θ	Billions of trained parameters (fixed after training — hence "a 70B parameter model")
x	Input tokens — the prompt you send at inference time
y	Output tokens — what the model generates, based on both x and θ

The key asymmetry: θ is **fixed** — a user has no control over it at inference time. x is the only thing a user can change, which is why changing the prompt changes the output.

CampusX

Why This Means No Intrinsic Memory

This function is **stateless**: a system is stateless if its output depends only on the current input, not on anything that happened before.

Concretely: calling the function with x_1 produces y_1. Calling it again with x_2 produces y_2 — and x_2's output has *no dependency* on x_1 or y_1. Each call is independent. This can be demonstrated directly: telling an LLM "My name is X" in one call, then asking "What is my name?" in a completely separate call, gets an "I don't know" response — proof that no information carried over.

> 💡 **Insight — LLMs Have No Intrinsic Memory**
> Since LLMs are stateless by mathematical definition, memory can never be something the model "does" internally. It has to be built as a system *around* the model — feeding relevant past context back in as input tokens each time.

21.1 The Core Problem and the Path to a Solution

Two facts create a deadlock:

1. LLMs have no intrinsic memory (stateless by nature).
2. Virtually no useful GenAI or agentic application can function without memory — a chatbot with no memory forces the user to repeat everything every time, which is highly frustrating and unworkable.

The resolution: since the model itself can't hold memory, **memory has to be built externally** — a system wrapped around the LLM that behaves like memory, even though the LLM itself still doesn't "remember" anything.

Building this system requires two foundational concepts: **context window** and **in-context learning**.

Context Window

The context window is the amount of text an LLM can read and hold in mind at one time before answering.

> 🧠 **Analogy — Context Window as a Camera Lens**
> The LLM is the camera; the context window is its lens. A small lens captures only a small part of the scene; a large lens captures much more. The bigger the context window, the more text the LLM can process before answering.

Modern LLMs typically have context windows of 128K tokens or more (roughly enough for a ~200-page PDF); some models (e.g., certain Gemini models) go up to 1 million tokens. The

CampusX

key takeaway: a lot of information *can* be sent as input tokens (x) — and this capacity is what memory-building will exploit.

In-Context Learning

During training, an LLM absorbs patterns from huge amounts of data, and that knowledge gets baked into its parameters (θ) — this is called **parametric knowledge**. Normally, at inference, the model answers by searching this parametric knowledge.

But as models scaled up, an emergent ability appeared: LLMs can also use information present directly *in the prompt itself* — even data they were never trained on (e.g., a 100-page private company PDF pasted into the prompt) — to answer questions. This is **in-context learning**: the model supplements its trained parametric knowledge with patterns/information supplied within the prompt at inference time.

21.2 Short-Term Memory: Using Conversation History as State

Combining the context window (large capacity) and in-context learning (ability to use prompt-supplied information), a simple solution emerges: **concatenate the entire conversation history into every new call**.

Instead of calling $y_2 = f(x_2, \theta)$, call:

$$y_2 = f(\text{concat}(x_1, y_1, x_2), \theta)$$

Each new LLM call includes everything said before. The model isn't "remembering" anything internally — the system is just feeding it continuity via the input.

Implementation pattern: maintain a `messages` list/variable. Append each new user message and each model response to it, and send the whole list on every call. This `messages` variable acts as *state* for an otherwise stateless system — often called a **conversation buffer**. Restarting the process empties it, and the LLM immediately "forgets" again — which is why this is called **short-term memory**.

Short-Term Memory in Chatbots: Thread-Scoped

Chatbot UIs (ChatGPT, Gemini, etc.) organize interactions into **conversations**, also called **threads** — one open-to-close session on a topic.

Each conversation gets its **own** short-term memory: the `messages` variable is scoped to a single thread. Opening a new chat starts a fresh, empty `messages` list; switching back to an old chat would need to reload that thread's messages separately. This is intentional — a single shared buffer across e.g. 1,000 conversations would become too long and incoherent for the LLM to use well. Hence: **short-term memory is thread-scoped/conversation-scoped**.

CampusX

21.3 Limitations of Short-Term Memory

Fragility

The `messages` variable lives in memory/process state — if the code resets or the server crashes, everything in it is lost, and the conversation context is gone permanently unless persisted.

Solution — persistence: connect short-term memory to a database. Assign each conversation a **thread ID**; before switching threads, save the current `messages` contents against that thread ID in the database. When returning to a thread, reload its messages from the database back into the `messages` variable, preserving continuity.

Context Window Overflow

As a conversation grows very long, the concatenated `messages` history can exceed the model's context window. Beyond that limit, the model can produce incoherent replies or hallucinate.

Widely used solution — trimming + summarization combined:

- **Trimming:** send only the most recent N messages (e.g., last 50 of 500) instead of the entire history, since recent context matters most.
- **Summarization:** generate a summary of the older, trimmed-out messages (via another LLM call) and attach that summary alongside the recent messages. This keeps total token count within the context window while preserving older context in compressed form.

Thread-Scoping Itself (the most critical limitation)

Because short-term memory is locked inside a single thread, three real problems follow:

- **No continuity across conversations.** Example: telling an LLM in one conversation "I prefer Python over other languages" gets remembered *within* that thread — but a new conversation two days later reverts to Java/JavaScript examples, because that preference never survived past the thread.
- **Learning never compounds over time.** Effort spent teaching the model your preferences (e.g., "give me window functions, not subqueries") has to be redone from scratch in every new conversation — the model never adapts across sessions.
- **Cross-thread reasoning is impossible.** You can't ask "what did we discuss yesterday?" or "what solution did we land on last time we hit this problem?" — that information is gone once the thread changes. Each new conversation makes the LLM a stranger again.

This makes true personalization or a genuine "personal assistant" experience impossible with short-term memory alone, since meaningful user facts and preferences accumulate *across* many separate conversations, not within just one.

CampusX

21.4 Why We Need Long-Term Memory

Solving thread-scoping requires a fundamentally different kind of memory with two key properties:

1. **Stores special, durable information** — information that survives beyond a single conversation/session, potentially useful for days, weeks, or the life of a multi-session project (e.g., facts relevant across the months it takes to write a book together with an assistant). Examples: who the user is (role, nationality, profession), how the system/assistant should behave for this user, what has worked vs. failed for them in the past, what decisions were previously made.
2. **Highly selective, not exhaustive** — rather than blindly saving entire conversations, only the most important, stable, reusable pieces of information are extracted and stored per conversation. Everything else is discarded.

Because this memory exists *outside* and *beyond* any single conversation/thread (unlike short-term memory, which is destroyed when the thread ends), it's called **long-term memory**.

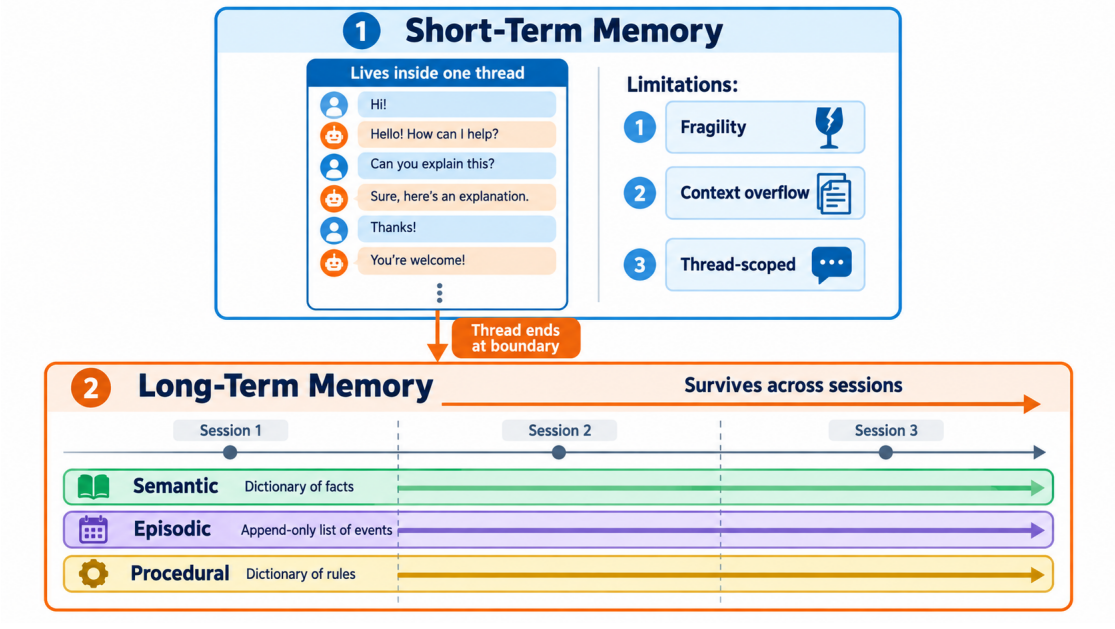

Figure 50. Short-term memory preserves one thread through conversation history, while long-term memory stores selected information across sessions.

21.5 Types of Long-Term Memory

Type	What It Stores	Purpose
Episodic memory	Past events — what happened before (e.g., a rejected solution, wrong deployment credentials used last time, what worked/didn't work previously)	Lets the current conversation answer "what did we do last time?" and avoid repeating past mistakes
Semantic memory	Facts about the user and the system (e.g., "user prefers Python," "user is a beginner," "system uses PostgreSQL," "budget constraint is ₹10,000")	Most common/important type — grounds the assistant in what's *true* about the user, system, or task
Procedural memory	Strategies, rules, and learned behaviors — *how* to do things (e.g., "avoid subqueries, use window functions," "if tool X fails, try tool Y," "always explain step-by-step for this user")	Compounds over time and is why an agent starts to feel personalized and comfortable to work with

21.6 How Long-Term Memory Works: Four-Step Architecture

Long-term memory systems operate in four steps:

Step	What Happens
1. Creation (Update)	During a conversation, decide what's worth remembering beyond it. Extract memory candidates from user/model/tool messages, filter out noise, tag the scope (user-level, app-level, or agent-level), then decide to create a new memory, update an existing one, or ignore it.
2. Storage	Save the created memory to a durable store, tagged with identifiers/metadata for easy future retrieval, so it survives restarts and crashes. Store type depends on the memory type: relational DB, key-value store, log/text file, or vector database (for semantic search) — episodic, semantic, and procedural memory may each use different storage.
3. Retrieval	When a new conversation starts, before replying, the system asks: "given the current situation, what should I remember right now?" It checks current user input, decides if memory is needed, searches the memory store, and pulls back a small relevant subset — **not** everything. Retrieval is **selective, not exhaustive**.

CampusX

Step	What Happens
4. Injection	Retrieved long-term memory is never sent to the LLM directly. It's first pulled into short-term memory (the conversation buffer/context window) alongside the user's prompt, and the combined content is sent to the model as just more input tokens.

> 💡 **Insight — Long-Term Memory Always Flows Through Short-Term Memory**
> The model never talks to long-term memory directly. Retrieval always inserts relevant long-term memories into the short-term context (the prompt) first — from the model's perspective, injected memory is indistinguishable from any other input tokens.

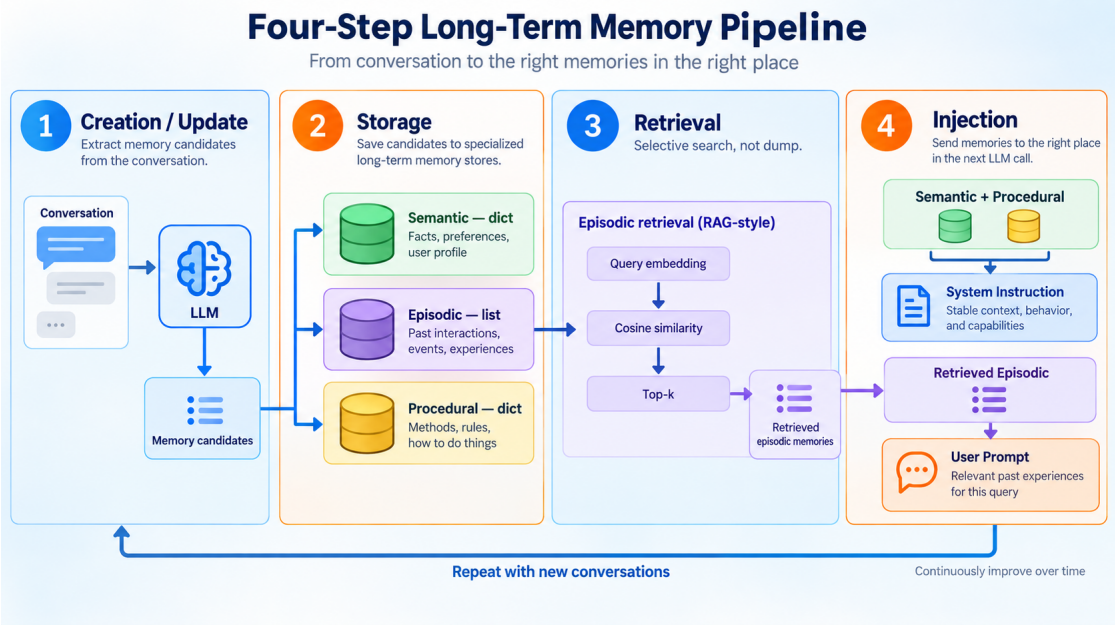

Figure 51. A long-term memory system creates, stores, retrieves, and injects selected information through a four-step pipeline.

21.7 Challenges in Building Memory Systems

Three main engineering challenges show up when building this end-to-end:

1. **Deciding what to remember** during creation — filtering signal from noise in normal conversation and judging whether something will still matter later is genuinely hard.
2. **Deciding what to retrieve** in real time — figuring out which subset of a potentially large long-term memory store is actually relevant to the current conversation.

3. **Orchestrating the whole system** — integrating memory stores, retrieval logic, and injection into an already-complex agentic system involves many moving parts and is engineering-heavy.

21.8 Tools for Memory Management

Because building this from scratch is hard, managed libraries/platforms have emerged that handle creation, storage, and retrieval so developers can focus on the application itself:

Tool	Notes
LangMem	From the LangChain family; integrates easily with LangGraph for building AI agents
Mem0	A popular memory-layer platform for GenAI apps
Supermemory	Also manages long-term memory for GenAI apps; notable for being built by a 15-year-old Indian founder

21.9 Future of Memory in LLMs

Because the root cause of all this complexity is that LLMs have no intrinsic memory, research is now exploring architectures with **built-in** memory. Google Research's **Titans** (and related work like "Miras") explore transformer-style architectures with intrinsic memory, aiming to reduce the need for these external memory systems altogether. This remains an active research area precisely because memory is essential for GenAI and agentic applications.

21.10 Designing Storage for Each Memory Type

Each of the three long-term memory types stores a different *kind* of content and therefore needs a different underlying data structure.

What Each Type Stores (Recap)

Memory Type	Stores	Governing Property
Semantic	Facts about the user — stable, general, true information (name, age, location, profession, allergies, language preference)	Every fact has exactly **one true answer at a time**, but that answer **can change over time** (e.g., city changes when the user relocates)
Episodic	Specific events tied to a particular time (a bug fixed, a course started, a lecture delivered)	Events **only accumulate** — history is never overwritten, only added to; the pile grows endlessly

CampusX

Memory Type	Stores	Governing Property
Procedural	Learned rules about *how* the AI should behave for this user (formatting preferences, explanation style, tone)	Behaves like a fact ("what output style does this user want") but is **refined through repeated feedback** over multiple turns, not stated once

Choosing the Right Data Structure

Semantic memory → Dictionary. A fact needs a unique key with a single current value, and that value must be easy to overwrite as the fact changes (**upsert**: update if the key exists, insert if it doesn't). A dictionary's key-value, mutable-value structure matches this exactly — e.g. `{"city": "Gurgaon"}` becomes `{"city": "Bangalore"}` after a move, without creating a duplicate entry.

Episodic memory → List. Events keep arriving and are never revised or deleted — only appended. A list's append-only growth pattern fits this "endless pile" naturally; nothing needs a unique key since multiple distinct events can coexist without conflict.

Procedural memory → Dictionary. Despite storing *behavioral rules* rather than personal facts, a rule like "preferred code-explanation style" still has one current value per labeled preference, and that value gets overwritten (not duplicated) as the AI learns more about the user — so it uses the same upsert-style dictionary as semantic memory.

> 💡 **Insight — Why Keep Semantic and Procedural Memory Separate?**
> Both use dictionaries and both follow "one current value per key," so it's tempting to merge them into a single store. For small or dummy projects, that's fine. But in production systems they're kept separate because they're populated differently: semantic facts usually arrive as a single stated fact ("I live in Bangalore"), while procedural rules are learned gradually across multi-turn feedback loops ("no, don't format it that way… like this instead"). Keeping them apart avoids a factual update (e.g., changing city) ever colliding with or accidentally overwriting a behavioral preference (e.g., code style).

22 Making Memories

The full pipeline has four parts: define storage classes, hold a conversation, extract memory candidates from it via an LLM, then route each candidate into the right store.

Step 1 — Storage Classes

Each memory type gets its own class wrapping the data structure decided above:

```python
class SemanticMemory:
    """Stable FACTS about the user. Built as a dict; written by upsert."""
    def __init__(self):
        self._facts = {}                  # empty dict created when the object
         ↪  is made

    def upsert(self, key, value):         # UPDATE if key exists, INSERT if
     ↪  new
        self._facts[key] = value

    def get(self, key):
        return self._facts.get(key)

    def delete(self, key):                # remove a fact once it stops being
     ↪  true
        self._facts.pop(key, None)

    def get_all(self):
        return dict(self._facts)          # return a copy - no search needed


class EpisodicMemory:
    """EVENTS that happened, time-stamped. Built as a list; append-only."""
    def __init__(self):
        self._events = []                 # empty list created when the object
         ↪  is made

    def add(self, text):
        self._events.append({             # APPEND -- old events are never
         ↪  rewritten
            "text": text,
            "time": datetime.now(timezone.utc).isoformat(),
            "vector": embed(text),         # used for similarity search later
             ↪  (next lecture)
        })

    def get_all(self):
        return list(self._events)


class ProceduralMemory:
    """Learned RULES about how to help this user - same shape as
     ↪  SemanticMemory,
    kept as a separate class so the two stay conceptually distinct."""
    def __init__(self):
        self._rules = {}
```

```
    def upsert(self, key, value):
        self._rules[key] = value

    def get(self, key):
        return self._rules.get(key)

    def delete(self, key):
        self._rules.pop(key, None)

    def get_all(self):
        return dict(self._rules)
```

The `embed()` call inside `EpisodicMemory.add()` converts text into a vector — this exists to support relevant-episode *retrieval* later (covered in the next lecture on bringing memory into the context window); at this stage it can be treated as a black box.

Step 2 — Extraction: Deciding What's Worth Remembering

A conversation naturally mixes durable, storage-worthy information with disposable noise. Example from a live chat session:

Message	Worth storing?	Why
"Hi, I'm Ajay, a data science instructor based in Delhi."	Yes	Identity fact
"Delhi is the capital of which country?"	No	One-off trivia question — noise
"I'm allergic to peanuts, keep that in mind for food suggestions."	Yes	Durable constraint
"Do peanuts contain protein?"	No	One-off factual question — noise
"Whenever you show me code, explain it step by step — I hate big code dumps."	Yes	Behavioral preference
"Last week I started building a Context Engineering course for CampusX."	Yes	Time-stamped event
"By the way, what's the capital of France?"	No	Noise
"I mostly work in Python and prefer it over R."	Yes	Language preference fact
"What's the difference between Python and R?"	No	One-off explanatory question — noise

CampusX

A practical checklist for what to look for when deciding whether something is worth extracting: identity and background, location, preferences, constraints, relationships, goals or ongoing situations, and relevant possessions; actions the user took, problems and their solutions, decisions made jointly with the AI, things tried but failed, and milestones; and format, length, depth, tone, language, working-style preferences, and dos/don'ts. All of this maps into one of the three memory types — semantic, episodic, or procedural.

This filtering-and-classifying job is delegated to the LLM itself via a system prompt:

```
EXTRACTION_PROMPT = """You extract long-term MEMORY from a conversation -
 ↪  durable facts,
events, and preferences that will help in FUTURE, separate conversations with
 ↪  this user.

Save something only if it will still matter days from now. Ignore greetings,
 ↪  small talk,
one-off questions the user just wanted answered, and anything temporary.

Classify each memory as exactly one type:

- "semantic"  = a lasting FACT about the user (who they are / what's true).
                Stays true tomorrow, no date attached.
- "episodic"  = a specific EVENT that happened, tied to a moment in time.
- "procedural"= a RULE about HOW to help this user (their preferred way of
 ↪  being helped).

Write each memory from the user's perspective: turn "I" into "the user",
 ↪  resolve vague
references, and make each item a single self-contained sentence.

Return ONLY a JSON list. Each item:
  "type":    "semantic" | "episodic" | "procedural"
  "key":     a short snake_case label for semantic/procedural; use "" for
 ↪  episodic
  "content": one short sentence describing the memory

If nothing is worth remembering, return []."""
```

Two design choices matter here:

- **Static instruction in the system role, changing conversation in the user role.** This keeps the system prompt a stable, cacheable prefix while the conversation content varies per call.
- **Rewrite from the user's perspective.** The model is told to turn "I prefer Python" into "The user prefers Python" — not "AI recommended Python" — so the stored memory is an objective third-person record, independent of how it was originally phrased.

The extraction call itself:

```
def extract_memories(conversation, system_prompt=EXTRACTION_PROMPT):
    response = ollama.chat(
        model=CHAT_MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content":
 ↪  f"Conversation:\n{conversation}\nOutput:"}
        ],
        format="json",    # ask for guaranteed valid JSON -> fewer parse
         ↪  failures
    )
    reply = response["message"]["content"]
    items = json.loads(_extract_json(reply))    # _extract_json strips ```json
 ↪  fences etc.
    return items
```

Running this on the full sample conversation produces memory candidates like:

```
[
  {"type": "semantic",   "key": "user_name",            "content": "The
 ↪  user's name is Ajay."},
  {"type": "semantic",   "key": "profession",           "content": "Works as
 ↪  a data science instructor based in Delhi."},
  {"type": "semantic",   "key": "health_allergy",       "content": "The user
 ↪  is allergic to peanuts."},
  {"type": "semantic",   "key": "language_preference",  "content": "Works
 ↪  mostly in Python and prefers it over R."},
  {"type": "procedural", "key": "code_explanation_style","content": "Explain
 ↪  code step by step and avoid large code dumps."},
  {"type": "episodic",   "key": "",                     "content": "Started
 ↪  building a Context Engineering course for CampusX last week."},
]
```

Note that every noise question (capital of Delhi/France, peanut protein content, Python-vs-R differences) was correctly dropped.

Step 3 — Routing: Storing Each Candidate

A single router function sends each candidate to its matching store, using that store's own write method:

```
def store_memory(item, semantic, episodic, procedural):
    t = item.get("type")
    if t == "semantic":
        semantic.upsert(item["key"], item["content"])      # fact  ->
 ↪  semantic store
    elif t == "episodic":
        episodic.add(item["content"])                       # event ->
 ↪  episodic store
    elif t == "procedural":
        procedural.upsert(item["key"], item["content"])     # rule  ->
 ↪  procedural store
    else:
        print("Unknown type, skipped:", item)
```

Applying this to all extracted candidates:

```
semantic   = SemanticMemory()
episodic   = EpisodicMemory()
procedural = ProceduralMemory()

for item in candidates:
    store_memory(item, semantic, episodic, procedural)
```

Resulting memory state:

```
SEMANTIC   (facts):
   user_name            -> The user's name is Ajay.
   profession           -> Works as a data science instructor based in Delhi.
   health_allergy       -> The user is allergic to peanuts.
   language_preference  -> Works mostly in Python and prefers it over R.
EPISODIC   (events):
   2026-08-21   Started building a Context Engineering course for CampusX
 ↪  last week.
PROCEDURAL (learned rules):
   code_explanation_style -> Explain code step by step and avoid large code
 ↪  dumps.
```

Out of a long, noisy conversation, only the durable, reusable signal survives into memory — this is the core payoff of memory design: when this gets injected into a future context window, the LLM sees only relevant information, not clutter, and can give a genuinely tailored response.

Memory Evolves Across Conversations

Running the same `extract_memories` → `store_memory` pipeline on a **second**, later conversation shows how memory accumulates rather than resets:

New conversation content: the user mentions starting freelance consulting alongside data science, gave a guest lecture last Friday, and asks for a one-line caption under every diagram/table shared.

Extraction produces:

```
[
  {"type": "semantic",   "key": "side_business",         "content": "The user
 ↪  does freelance consulting for startups alongside their data science
 ↪  work."},
  {"type": "episodic",   "key": "",                       "content": "The
 ↪  user gave a guest lecture on machine learning basics at a local college
 ↪  last Friday."},
  {"type": "procedural", "key": "formatting_preference",  "content": "Always
 ↪  add a one-line caption under every diagram or table shared."},
]
```

After routing these into the same store objects, the memory state becomes:

```
SEMANTIC   (facts):
   user_name            -> The user's name is Ajay.
   profession           -> Works as a data science instructor based in Delhi.
   health_allergy       -> The user is allergic to peanuts.
   language_preference  -> Works mostly in Python and prefers it over R.
   side_business        -> The user does freelance consulting for startups
 ↪  alongside their data science work.
EPISODIC   (events):
   2026-08-21   Started building a Context Engineering course for CampusX
 ↪  last week.
   2026-08-21   The user gave a guest lecture on machine learning basics at a
 ↪  local college last Friday.
PROCEDURAL (learned rules):
   code_explanation_style -> Explain code step by step and avoid large code
 ↪  dumps.
   formatting_preference  -> Always add a one-line caption under every
 ↪  diagram or table shared.
```

Existing semantic/procedural keys stay untouched (upsert doesn't disturb unrelated keys); episodic memory simply grows by one more entry. This is exactly the "durable, evolving personalization" that short-term memory alone can never achieve — the assistant keeps compounding what it knows about the user across separate conversations.

> 💡 **Insight — Manual Build, Automated in Practice**
> This manual build is done here purely to show the underlying mechanics; in practice, dedicated memory libraries handle extraction, storage, and retrieval out of the box.

23 Reading Memory

The previous lecture covered only two of the four memory-lifecycle steps: **extract** and **store**. The memories built there — semantic (facts), episodic (events), procedural (rules) — were populated but never actually used by the LLM. This part covers the remaining two steps: **retrieve** and **inject** — pulling stored memory back out and feeding it into the context window so the model can act on it, without polluting or bloating that window.

Each of the three memory types is retrieved and injected **differently**, because each stores a different kind of information with different volume and retrieval needs.

23.1 Semantic and Procedural Memory: Treated as "Constitution"

Why No Retrieval Step Is Needed

Semantic memory (facts) and procedural memory (learned rules) share two properties that change how they're handled:

- **Needed in almost every conversation.** A user's name, allergies, or preferred code-explanation style are relevant regardless of what's currently being discussed.
- **Small in volume.** Basic facts about a person and their learned preferences don't pile up the way events do.

As long as these stores stay small, stable, and broadly relevant to the session, there's no need to search or filter them — the **entire** semantic memory and **entire** procedural memory are pulled and injected directly, every time. This is the strategy while volume stays low; a semantic or procedural store that grows large, or that accumulates task-specific entries not relevant to every session, would need the same kind of selective retrieval used for episodic memory below.

Where It Gets Injected: System Instruction

Semantic and procedural memory are merged directly into the **system instruction** — the same "constitution" that governs the LLM's behavior (introduced earlier in the course). The new system instruction becomes:

New System Instruction = Original System Instruction + Rules (Procedural) + Facts (Semantic)

CampusX

```
SYSTEM_INSTRUCTION = """You are a helpful personal assistant with memory of
 ↪  this user.

You will be given memory about the user in three parts:
- RULES: how this user wants you to behave. Always follow them.
- FACTS: stable truths about the user. Treat them as true and use them when
 ↪  relevant.
- RELEVANT EVENTS: things that happened before, retrieved for this question.
 ↪  Use them
  only if they help answer the current question; ignore them if they don't
 ↪  fit.

Rely on this memory instead of guessing. If the memory doesn't cover
 ↪  something, answer
normally and don't invent details about the user."""

rules = "\n".join(f"- {v}" for v in procedural.get_all().values()) or "-
 ↪  (none)"
facts = "\n".join(f"- {v}" for v in semantic.get_all().values()) or "-
 ↪  (none)"

SYSTEM_INSTRUCTION = SYSTEM_INSTRUCTION + "\n\nRULES:\n" + rules +
 ↪  "\n\nFACTS:\n" + facts
```

As soon as a new chat instance/session starts, this combined instruction — original instructions + that user's rules + that user's facts — becomes the LLM's operating constitution for the whole session. Two users sending the identical prompt through the identical model can get differently-styled responses, because each carries a different personalized system instruction built from their own accumulated rules and facts.

> 💡 **Insight — Why This Also Protects the KV Cache**
> Placing semantic and procedural memory inside the system instruction means this block sits at the very top of the prompt and stays constant for the whole conversation. Because it doesn't change turn to turn, the model's KV (key-value) cache for this portion is preserved rather than recomputed on every call — this keeps the setup both context-efficient and computationally cheaper.

23.2 Episodic Memory: Treated as RAG

Why It Can't Be Dumped Directly

Episodic memory is fundamentally different: it stores an ever-growing, unbounded pile of past events, and only a small fraction of that pile is ever relevant to a given question. Dumping the entire episodic history into every prompt would bloat and pollute the context window. This is the

same problem RAG (Retrieval-Augmented Generation) was built to solve — so episodic memory is retrieved exactly like a RAG pipeline: **search, don't dump.**

Step 1: Memory Creation as a Searchable Store

Each episodic entry is stored with both its raw text and a vector embedding of that text (already set up in the previous lecture's `EpisodicMemory.add()`):

```
def add(self, text):
    self._events.append({
        "text": text,
        "time": datetime.now(timezone.utc).isoformat(),
        "vector": embed(text),    # embedding stored alongside the text
    })
```

Step 2: Retrieval via Cosine Similarity

When a query comes in, it's embedded, compared against every stored event's vector using cosine similarity, and only the top matches above a similarity threshold are returned:

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def search_events(query, k=2, min_score=0.45):
    events = episodic.get_all()
    hits = []

    if events:
        q = embed(query)
        scored = [(cosine(q, e["vector"]), e) for e in events]
        scored.sort(key=lambda pair: pair[0], reverse=True)    # highest
↪   similarity first
        hits = [(score, event) for score, event in scored[:k] if score >=
↪   min_score]

    return hits
```

- **Threshold (`min_score = 0.45`):** an event only counts as relevant if its similarity score is at least 0.45 (45%).
- **Top-k (`k = 2`):** at most the top 2 matching events are returned, even if more clear the threshold.

Behavior in practice: asking *"What food should I avoid?"* against an episodic store that has no diet-related event returns nothing (correctly — that fact lives in semantic memory, not episodic). Asking *"What do you know about the context engineering course?"* returns the matching event ("Started building a Context Engineering course for CampusX last week") with a similarity score around 0.64 — well above the threshold.

Step 3: Injection Into the User Prompt (Not the System Instruction)

Unlike semantic/procedural memory, retrieved episodic "hits" are injected into the **user prompt**, mixed in with the current question — the same way retrieved chunks are injected in a standard RAG setup:

```
retrieved = search_events("What do you know about the context engineering
 ↪  course?")[0][1]["text"]
user_question =
 ↪  f"<question>{question}</question><retrieved_chunks>{retrieved}</retrieved_chunks>"
```

> 💡 **Insight — Governance vs. Evidence**
> Semantic and procedural memory answer "**who is this user and how should I behave**" — that's governance, so it belongs in the system instruction, which sets the model's persona and rules before the current question is even read. Episodic memory answers "**what evidence from the past is relevant to this specific question**" — that's supporting evidence for one query, so it belongs in the user prompt, right alongside the question it supports. Keeping these two roles in separate parts of the prompt is what stops the context window from becoming polluted as conversations grow.

23.3 Combining All Three Memory Types

A single `answer()` function pulls everything together — constitution-style memory into the system role, RAG-style memory into the user role:

```
def answer(question):
    # ---- SYSTEM section: rules + facts (stable memory) ----
    rules = "\n".join(f"- {v}" for v in procedural.get_all().values()) or "-
 ↪  (none)"
    facts = "\n".join(f"- {v}" for v in semantic.get_all().values()) or "-
 ↪  (none)"
    system = {"role": "system", "content":
        SYSTEM_INSTRUCTION + "\n\nRULES:\n" + rules + "\n\nFACTS:\n" + facts}
```

```
    # ---- USER section: the question, with events added ONLY if any are
     ↪  relevant ----
    events = search_events(question)
    if events:
        event_text = "\n".join(f"- {e['text']}" for _, e in events)
        user_content = "RELEVANT EVENTS:\n" + event_text + "\n\nQuestion: " +
↪   question
    else:
        user_content = question    # no relevant event -> keep it clean, don't
↪   force-inject noise
    user = {"role": "user", "content": user_content}

    messages = [system, user]
    reply = ollama.chat(model=CHAT_MODEL,
↪   messages=messages)["message"]["content"]
    print("\nASSISTANT:\n" + reply)
```

This produces genuinely personalized, zero-shot-looking behavior without the user restating anything in the current session:

Question Asked	What Answered It	Memory Source
"What snack should I grab?"	Warns to avoid peanuts	Semantic (allergy fact)
"What course am I working on?"	Names the Context Engineering course for CampusX	Episodic (retrieved event)
"Write a function to reverse a string"	Gives Python code, explained step by step	Procedural (code style rule) + Semantic (language preference)

None of these facts were restated in the current conversation — the model behaves as if it "just knows," purely because the right memory was injected into the right part of the prompt.

CampusX

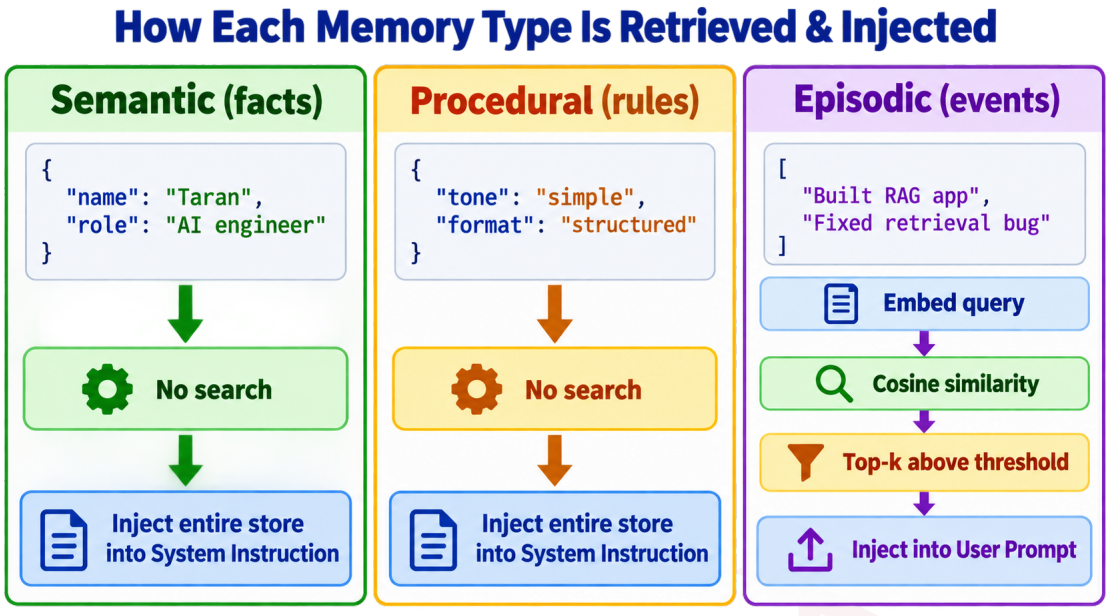

Figure 52. Semantic and procedural memory shape the system instruction, while relevant episodic memories are retrieved and injected beside the current user query.

23.4 Running It as an Agentic Loop

The final step wraps this into a persistent chat loop that layers **short-term memory** (the running conversation `history`) on top of the **long-term memory** stores built and retrieved above:

```
def run_agent():
    history = []
    print("Chat started. Type 'exit' to stop.\n")

    while True:
        question = input("You: ").strip()
        if question.lower() in ("exit", "quit", ""):
            print("Chat ended.")
            break

        # ---- SYSTEM: rules + facts (long-term, stable) ----
        rules = "\n".join(f"- {v}" for v in procedural.get_all().values()) or
↪   "- (none)"
        facts = "\n".join(f"- {v}" for v in semantic.get_all().values()) or "-
↪   (none)"
        system = {"role": "system", "content":
```

```
            SYSTEM_INSTRUCTION + "\n\nRULES:\n" + rules + "\n\nFACTS:\n" +
↪  facts}

        # ---- USER: the question, with events added only if relevant ----
        events = search_events(question)
        if events:
            event_text = "\n".join(f"- {e['text']}" for _, e in events)
            user_content = "RELEVANT EVENTS:\n" + event_text + "\n\nQuestion:
↪  " + question
        else:
            user_content = question
        user = {"role": "user", "content": user_content}

        # ---- messages = system + conversation so far + this turn ----
        messages = [system] + history + [user]
        reply = ollama.chat(model=CHAT_MODEL,
↪  messages=messages)["message"]["content"]
        print("Assistant:", reply, "\n")

        # ---- store the plain exchange in history (short-term memory) ----
        history.append({"role": "user", "content": question})
        history.append({"role": "assistant", "content": reply})
```

Sample run: without the user ever mentioning it in the session, the agent correctly warns about a peanut allergy, names the course the user is working on, writes Python code broken down step by step, and — thanks to `history` — correctly recalls "What did I just ask you?" (answering with the previous turn's question, i.e. ordinary short-term conversational continuity).

23.5 Putting the Architecture Together

Memory Type	Retrieval Strategy	Injection Point	Reason
Semantic (facts)	None, while the store stays small & broadly relevant	System instruction	Small, always relevant — a large or task-specific store would need filtering too
Procedural (rules)	None, while the store stays small & broadly relevant	System instruction	Same as semantic; behaves like a fact about "how to help"
Episodic (events)	RAG-style: embed query → cosine similarity → top-k above threshold	User prompt	Large, ever-growing, only a small relevant slice matters per query

This is the complete answer to "how do you keep the context window healthy while still being deeply personalized": small, always-needed memory goes into the stable system instruction

(protecting the KV cache); large, growing memory is filtered through retrieval and injected only when relevant, right alongside the question it supports.

24 Creating Memories with Libraries and Problems with Libraries

Everything so far (extraction, storage, retrieval, injection) was built by hand to show the underlying mechanics. In industry, this work is rarely done from scratch — the same way machine learning algorithms aren't hand-coded when scikit-learn exists, or data manipulation isn't hand-rolled when pandas and NumPy exist. AI memory has its own ecosystem of libraries that handle creation, storage, and retrieval automatically: **LangMem**, **Zep**, **Mem0**, and **Supermemory** are the most commonly cited names.

Among these, **Mem0** (written "mem0," pronounced "Mem Zero") is currently the most widely adopted — it consistently tops searches, blog rankings, and chatbot recommendations for "top libraries for creating memory in AI."

24.1 What Mem0 Is

Mem0 is an **open-source library for building a persistent memory layer around an AI application**. The core idea is deliberately simple: hand it a conversation, and it automatically extracts relevant information, stores it as reusable memory, and later retrieves the relevant pieces when needed — all without the developer writing extraction prompts, embedding logic, or a vector-store pipeline by hand.

> Mem0 is an open-source library that creates persistent memory layers for LLM applications and AI agents — it automatically extracts useful information from a conversation, stores it as reusable memory, and retrieves the relevant memories later when needed.

Why Mem0 Is So Widely Used

Reason	What It Means
Purpose-built for AI memory	Handles creation, storage, and retrieval end-to-end on the developer's behalf
Automatic extraction	Pulls out important facts/information from a conversation without manual filtering rules
Semantic (meaning-based) retrieval	Search is done by meaning, not keyword-matching
Easy to integrate	Can be dropped into an existing LLM/agent system with minimal redesign
Open source and customizable	The underlying storage and retrieval behavior can be adapted to a given application's needs

Reason	What It Means
Framework-agnostic	Works with LangGraph, LangChain, CrewAI, or a custom stack — unlike, e.g., LangMem, which is tied specifically to the LangChain framework
User-specific memory (multi-tenancy)	Every memory is tagged with a unique user/account ID, so different users' memories are stored and retrieved in isolation rather than overlapping

Practically, nearly all of this is exposed through just **two functions**: `.add()` to create memory and `.search()` to retrieve it.

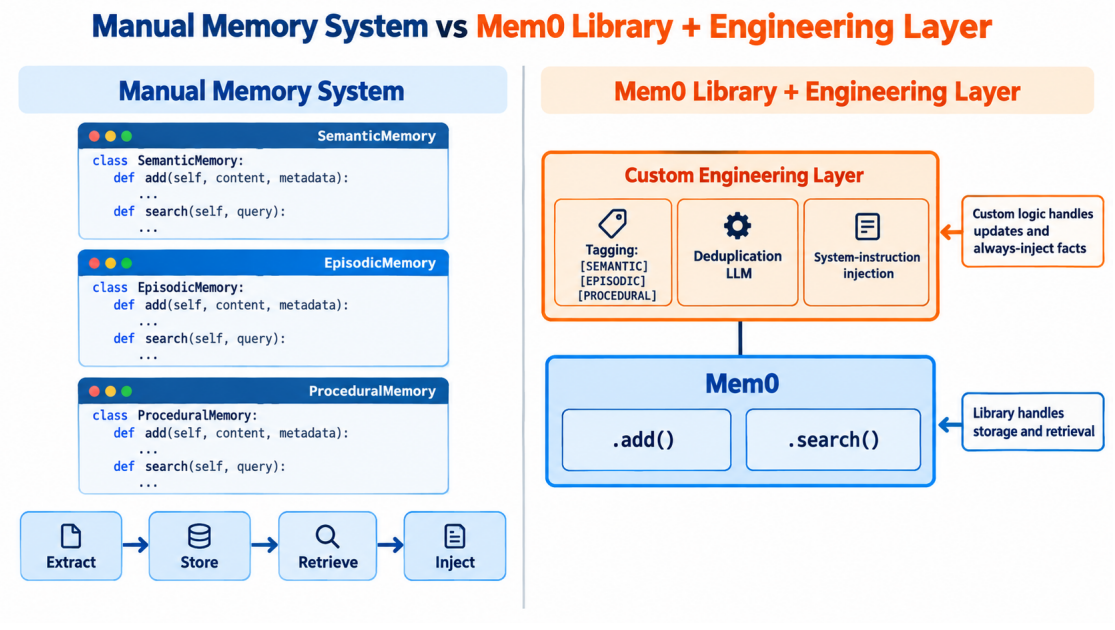

Figure 53. A manual memory pipeline exposes each extraction, storage, retrieval, and injection step, while Mem0 packages those mechanics behind a compact library interface.

24.2 Building and Using Memory With Mem0

Step 1 — Install and Import

```
!pip install -U mem0ai

import ollama
from mem0 import Memory
```

Step 2 — Create an Instance

Creating the instance is where you specify which LLM extracts facts, which embedding model turns them into vectors, and which vector database stores them (Mem0 defaults to OpenAI if you just supply an API key; a local/alternate stack needs an explicit config):

```
config = {
    "llm":       {"provider": "ollama", "config": {"model":
     ↪  "gemma4:31b-cloud"}},
    "embedder": {"provider": "ollama", "config": {"model":
     ↪  "embeddinggemma:latest"}},
    "vector_store": {
        "provider": "qdrant",                    # vector DB choice (Chroma,
         ↪  Pinecone, etc. also work)
        "config": {
            "embedding_model_dims": 768,          # must match the embedding
             ↪  model's output size
            "collection_name": "mem0_gemma",      # table name
        },
    },
}
m = Memory.from_config(config)
```

Step 3 — Create Memory: `.add()`

A raw, noisy conversation (greetings, small talk, one real facts mixed with weather chit-chat) is passed straight to `.add()`, tagged with a `user_id`:

```
m.add(conversation, user_id="ajay")
```

Behind the scenes, Mem0 uses the configured LLM to extract only the durable facts, embeds them with the configured embedding model, and stores those embeddings in the configured vector database — all in one call. Retrieving everything stored for a user:

```
for mem in m.get_all(filters={"user_id": "ajay"})["results"]:
    print("-", mem["memory"])
```

From a messy conversation containing weather small talk and traffic complaints alongside real facts, Mem0 correctly extracted only the durable pieces: Ajay lives in Delhi, works as a data

science instructor, is allergic to peanuts, prefers Python over R, wants code explained step by step, and started a Context Engineering course for CampusX — all noise dropped automatically.

Each stored memory record carries more than just text — an ID, a text field, a hash, metadata, timestamps (created/updated), and the associated `user_id`, which is exactly what enables per-user isolation: querying with one `user_id` only ever surfaces that user's own memories.

Step 4 — Retrieve Memory: `.search()`

```
m.search("What food should I take?", filters={"user_id": "ajay"})
```

Every stored memory for that user comes back with a **semantic similarity score** against the query — not a binary match/no-match:

Query	Top Retrieved Fact	Score
"What food should I take?"	User has a peanut allergy	~0.50
"Where do I live?"	User lives in Delhi	~0.60
"Which language do I prefer?"	User prefers Python over R	~0.54

Sending Retrieved Memory to the LLM

The typical pattern: take the top-k (commonly top 2–3) results by score, extract just their text, and inject that into the user prompt alongside the question — the same RAG-style pattern used for episodic memory in the manual build:

```
result = m.search(user_query, filters={"user_id": "ajay"})

top_2_text = " ".join(
    item["memory"]
    for item in sorted(result["results"], key=lambda x: x["score"],
     ↪  reverse=True)[:2]
)

user_question =
 ↪  f"<question>{user_query}</question><retrieved_chunks>{top_2_text}</retrieved_chunks>"

messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user",   "content": user_question}
]
```

CampusX

```
response = ollama.chat(model="gemma4:31b-cloud", messages=messages)
```

This correctly answers things like “I am currently working on which project?” (pulling the Context Engineering course fact) or “give me code to calculate the mean of a dataframe column” (writing it in Python, per the stored language preference) — without those facts being restated in the current session.

24.3 The Update Problem: Multi-Turn, Contradictory Facts

Simulating a user returning after time has passed, with facts that **contradict** earlier ones:

```
update_conversation = [
    {"role": "user", "content": "Hey! Big update - I moved from Delhi to
 ↪  Bangalore last month."},
    {"role": "assistant", "content": "That's a big move, congrats!"},
    {"role": "user", "content": "Also, I've switched - I mostly work in R now,
 ↪  not Python anymore."},
    {"role": "assistant", "content": "Got it, R it is now."},
]
m.add(update_conversation, user_id="ajay")
```

Querying afterward shows the problem clearly: `m.get_all()` now lists **both** “Ajay lives in Delhi” **and** “Ajay moved from Delhi to Bangalore,” and **both** “user prefers Python over R” **and** “user switched primary language from Python to R.” The old fact was never removed — it was just added alongside the new, contradicting one, leaving the LLM to sort out which is current.

24.4 Limitations of Memory Libraries (Mem0 and Others Alike)

As of August 2026, every major memory library — not just Mem0 — has solved **storage and retrieval** well, but none has fully solved **keeping memory correct over time**. Mem0’s own “State of AI Agent Memory” documentation acknowledges this directly, citing unsolved problems in temporal fact extraction and cross-session fact replacement.

The Temporal Update Problem (Fact Redundancy)

As shown above: when a fact changes (city, language preference), libraries tend to **append** the new fact rather than **replace** the old one, producing contradictory memories that the LLM has to reconcile on its own.

No Automatic Forgetting

Libraries are good at remembering but not at discarding information that's become irrelevant or obsolete — leading to a context window that accumulates clutter over time.

Facts and Instructions Can Get Missed During Retrieval

Because retrieval in these libraries pulls chunks based on **query similarity**, an important standing fact (e.g., a peanut allergy) can rank low or get excluded for a query that isn't obviously related to it (e.g., "which language do I prefer?") — even though that fact should arguably always be visible to the model. This happens because these libraries treat *everything* as retrieval-based, with no separate "always-inject" channel the way the earlier manual design used the system instruction for semantic/procedural memory.

Multi-Turn Conversation History Isn't Well Managed

Beyond single-fact contradictions, the broader flow of a multi-turn relationship (what was discussed, in what order, evolving context) isn't handled with the same care as raw fact storage.

Retrieval Isn't Always the Efficient Choice

Running a full RAG-style search for *every* query — even a simple one like "Who am I?" — has real latency cost, and that cost grows with the size of the vector store.

> ⚠ **Mentor's Rule — Don't Retrieve What Should Just Be Injected**
> A direct benchmark: answering "Who am I?" by retrieving from memory took ~6.2 seconds; answering with the same information placed directly in the system instruction took ~1.7 seconds — a 72% difference. This was with only 8–9 stored vectors; the gap only widens with a larger memory store. Static, always-needed facts belong in the system instruction, not behind a retrieval call — retrieval should be reserved for the dynamic, growing portion of memory (as episodic memory was handled in the manual build).

25 Creating Memories Using Libraries and Rectifying Their Errors

The previous lecture identified two concrete problems with using a memory library like Mem0 as-is: (1) there was no way to separate semantic/procedural memory out so it could live in the system instruction, and (2) outdated facts weren't being replaced when new, contradicting facts arrived. This part solves both through engineering layered on top of the library — not by abandoning the library.

Solving problem 1 (getting semantic and procedural memory into the system instruction) takes three steps: **tag** each memory at creation time, **separate** tagged memories into their own containers, then **integrate** those containers into the system instruction.

Step 1 — Tagging Memories at Creation Time

Mem0 exposes a `custom_instructions` parameter (best supported in API version `v1.1`) that lets you tell the extraction LLM exactly how to classify and label what it pulls out. Every extracted memory is instructed to start with one of three tags — `[SEMANTIC]`, `[EPISODIC]`, or `[PROCEDURAL]`:

```
CLASSIFY_INSTRUCTIONS = """
For every memory you extract, classify it into EXACTLY ONE of these three types:
[SEMANTIC], [EPISODIC], or [PROCEDURAL]. Put the classification tag at the VERY
START of the memory.

[SEMANTIC] - enduring facts, attributes, preferences, identity, skills.
  "What is true about the user?"
  e.g. "[SEMANTIC] User lives in Bangalore", "[SEMANTIC] User knows Python"

[EPISODIC] - specific events, experiences, time-bounded activities/projects.
  "What happened? What was the user working on, and when?"
  e.g. "[EPISODIC] User started building a Context Engineering course for
  CampusX during the week of August 22, 2026"

[PROCEDURAL] - instructions/rules for future behavior.
  "How should I behave?"
  e.g. "[PROCEDURAL] Always explain code step by step"

Use exactly ONE tag per memory, placed first, followed by a space.
"""

config = {
    "llm":      {"provider": "ollama", "config": {"model": "gemma4:31b-cloud"}},
    "embedder": {"provider": "ollama", "config": {"model": "embeddinggemma:latest"}},
    "vector_store": {
        "provider": "qdrant",
        "config": {"embedding_model_dims": 768, "collection_name": "mem0_gemma3"},
    },
    "custom_instructions": CLASSIFY_INSTRUCTIONS,
```

```
    "version": "v1.1",
}
m = Memory.from_config(config)
```

> 💡 **Insight — Ongoing Activities Are Still Episodic**
> A subtlety worth internalizing: something being *currently true* doesn't automatically make it semantic. "User is building a Context Engineering course" describes a specific, time-bounded project with a recognizable start — that's episodic, not semantic, even while it's ongoing. The distinguishing test: semantic answers *"what is true about the user"*; episodic answers *"what happened / what were they working on, and when."* An event can also produce a resulting semantic fact — "User moved to Bangalore" (episodic, the move) versus "User lives in Bangalore" (semantic, the resulting state) — both can legitimately be extracted from the same sentence.

With this in place, every memory returned by `m.add()` now carries a tag, e.g. `[SEMANTIC] User's name is Ajay`, `[PROCEDURAL] Always explain step by step`, `[EPISODIC] User started building a context engineering course...`.

Step 2 — Separating Tagged Memories

Once tagged, memories are split into dedicated containers by iterating over everything returned by `get_all()`:

```
from datetime import datetime

semantic = []
procedural = []

for row in m.get_all(filters={"user_id": "ajay"})["results"]:
    text = row["memory"]
    dt = datetime.fromisoformat(row["created_at"])
    date, time = dt.strftime("%d-%m-%Y"), dt.strftime("%I:%M:%S %p")

    if text.startswith("[SEMANTIC]"):
        semantic.append({"text": text.replace("[SEMANTIC]", "").strip(),
↪   "date": date, "time": time})
    elif text.startswith("[PROCEDURAL]"):
        procedural.append({"text": text.replace("[PROCEDURAL]", "").strip(),
↪   "date": date, "time": time})
```

Note: `date`/`time` are captured alongside each memory specifically to support the de-duplication step next. Also note that **no separate container is built for episodic memory** here — that's intentional, covered below.

CampusX

Solving the Duplication Problem

Splitting memories this way surfaces the second problem directly: the semantic list may contain both "User mostly codes in Python and prefers it over R" *and* "User has switched their primary language from Python to R" — genuinely contradictory facts sitting side by side.

Important caveat first: duplication isn't automatically wrong. "Used to prefer Python, now prefers R" is itself a valid piece of memory — some applications (audit trails, developmental tracking, medical/legal bots) *want* to retain that full history rather than collapse it. Whether to de-duplicate is an application design decision, not a universal rule.

For applications that do want a single current "source of truth" per fact, a small, fast LLM is used purely for this deduplication job — it reads the memory list, identifies which entries represent the *same underlying fact*, and keeps only the most recent one by timestamp:

```
import ollama

def deduplicate_memories(memory_list):
    if not memory_list:
        return ""

    system_instruction = """
You are a memory deduplication system. Your task is to clean the given
 ↪  memories.

1. Identify memories that represent the SAME underlying fact, preference,
   behavior, rule, state, or attribute.
2. If multiple memories represent the same fact, keep ONLY the latest one
   according to date and time.
3. If memories represent different facts, keep all of them.
4. Pay attention to changes over time (e.g. "lives in Delhi" -> "moved to
   Bangalore" are the same fact - keep only the latest).
5. Do not merge unrelated memories.
6. Return ONLY the final memory text, one per line - no JSON, dates, bullets,
   or explanations.
"""
    user_prompt = f"Memories:\n\n{memory_list}\n\nReturn only the final
 ↪  memories, one per line."

    response = ollama.generate(model="gemma4:31b-cloud",
 ↪  system=system_instruction, prompt=user_prompt)
    return response["response"].strip()

clean_semantic   = deduplicate_memories(semantic)
clean_procedural = deduplicate_memories(procedural)
```

Getting good results here depended heavily on how detailed the classification instructions were — an under-specified prompt misclassified a location-change event (e.g., putting the Bangalore move under the wrong tag); adding more explicit examples and semantic-vs-episodic distinctions in `CLASSIFY_INSTRUCTIONS` fixed it. After deduplication, contradictions like "lives in Delhi" / "lives in Bangalore" collapse correctly to just "lives in Bangalore."

Step 3 — Integrating Into the System Instruction

With clean, deduplicated semantic and procedural text in hand, they're merged into the system instruction exactly as in the manual build:

```
SYSTEM_INSTRUCTION = """You are a helpful personal assistant with memory of
 ↪  this user.

You will be given memory about the user in three parts:
- RULES: how this user wants you to behave. Always follow them.
- FACTS: stable truths about the user. Treat them as true and use them when
 ↪  relevant.
- RELEVANT EVENTS: things that happened before, retrieved for this question.
 ↪  Use them
  only if they help answer the current question; ignore them if they don't
 ↪  fit.

Rely on this memory instead of guessing. If the memory doesn't cover
 ↪  something, answer
normally and don't invent details about the user."""

SYSTEM_INSTRUCTION = SYSTEM_INSTRUCTION + "\n\nRULES:\n" + clean_procedural +
 ↪  "\n\nFACTS:\n" + clean_semantic
```

Asking the model "tell me something about myself" against this system instruction correctly returns the current, de-duplicated facts (data science instructor, based in Bangalore — not Delhi, allergic to peanuts) with no contradiction visible to the model. Importantly, the underlying Mem0 store can still be "dirty" (containing the old, superseded entries) — the point of this engineering layer is to *purify* what actually reaches the LLM, not necessarily to clean the raw store itself.

25.1 Handling Episodic Memory: Reuse the Library's Own Retrieval

No dedicated container was built for episodic memory above — deliberately. The Mem0 instance itself is treated *as* episodic memory, because its `.add()`/`.search()` functions already do exactly what a manual episodic-memory RAG pipeline would: `.add()` embeds text and stores it in a vector store; `.search()` embeds a query and finds semantically similar stored vectors. Rebuilding that manually would just be reimplementing what the library already provides.

The only addition needed: filter `.search()` results down to entries tagged `[EPISODIC]`, above a similarity threshold, then take the top-k:

```python
def get_top_episodic_memories(user_query, user_id="ajay", threshold=0.30,
 ↪  top_k=2):
    result = m.search(user_query, filters={"user_id": user_id})

    episodic = [
        r for r in result["results"]
        if r["memory"].startswith("[EPISODIC]") and r["score"] > threshold
    ]

    top_episodic = sorted(episodic, key=lambda x: x["score"],
 ↪  reverse=True)[:top_k]

    return [
        {
            "text": r["memory"].replace("[EPISODIC]", "").strip(),
            "date": r["created_at"].split("T")[0],
            "time": r["created_at"].split("T")[1].split("+")[0],
        }
        for r in top_episodic
    ]
```

The retrieved episodic chunks are then injected into the **user prompt** (not the system instruction), delimited alongside the question — the same RAG-style pattern used throughout this module:

```python
episodic = get_top_episodic_memories(user_query)
user_question =
 ↪  f"<question>{user_query}</question><retrieved_chunks>{episodic}</retrieved_chunks>"

messages = [
    {"role": "system", "content": SYSTEM_INSTRUCTION},
    {"role": "user",   "content": user_question}
]
response = ollama.chat(model="gemma4:31b-cloud", messages=messages)
```

25.2 One Function, All Three Memory Types

Everything above combines into a single `answer_with_memory()` function: fetch all memories → tag-split into semantic/procedural → deduplicate each → build the system instruction from them → search + filter + rank episodic hits for the current query → assemble the user prompt → call the LLM.

```python
def answer_with_memory(user_query, user_id="ajay"):
    memories = m.get_all(filters={"user_id": user_id})["results"]

    semantic, procedural = [], []
    for row in memories:
        text = row["memory"]
        dt = datetime.fromisoformat(row["created_at"])
        date, time = dt.strftime("%d-%m-%Y"), dt.strftime("%I:%M:%S %p")
        if text.startswith("[SEMANTIC]"):
            semantic.append({"text": text.replace("[SEMANTIC]", "").strip(),
↪   "date": date, "time": time})
        elif text.startswith("[PROCEDURAL]"):
            procedural.append({"text": text.replace("[PROCEDURAL]",
↪   "").strip(), "date": date, "time": time})

    clean_semantic   = deduplicate_memories(semantic)
    clean_procedural = deduplicate_memories(procedural)

    SYSTEM_INSTRUCTION = ("You are a helpful personal assistant with memory
↪   of this user.\n\n"
        "RULES: how this user wants you to behave. FACTS: stable truths about
         ↪  the user. "
        "RELEVANT EVENTS: retrieved past events - use only if they help.\n\n"
        "RULES:\n" + clean_procedural + "\n\nFACTS:\n" + clean_semantic)

    result = m.search(user_query, filters={"user_id": user_id})
    episodic = [r for r in result["results"] if
↪   r["memory"].startswith("[EPISODIC]") and r["score"] > 0.30]
    top_2_episodic = sorted(episodic, key=lambda x: x["score"],
↪   reverse=True)[:2]
    episodic = [{"text": r["memory"].replace("[EPISODIC]", "").strip(),
                 "date": r["created_at"].split("T")[0],
                 "time": r["created_at"].split("T")[1].split("+")[0]} for r
                  ↪  in top_2_episodic]

    user_question =
↪   f"<question>{user_query}</question><retrieved_chunks>{episodic}</retrieved_chunks>"
    messages = [{"role": "system", "content": SYSTEM_INSTRUCTION}, {"role":
↪   "user", "content": user_question}]
    return ollama.chat(model="gemma4:31b-cloud", messages=messages)
```

Calling `answer_with_memory("I am currently working on which project?")` correctly surfaces the Context Engineering course episode; calling it with `"What do you know about me?"` correctly surfaces name, profession, location, allergy, and language preference from the system instruction — all handled through one function, with each memory type routed

through its own correct mechanism. This approach also measurably uses fewer tokens than a naive blind-RAG-everything approach, since stable facts aren't repeatedly re-retrieved.

25.3 How This Maps to Real Chatbots (ChatGPT, Claude, etc.)

The same architecture underlies production chat assistants, just automated end-to-end:

- **During an active session**, ongoing conversation is handled by an in-memory `messages`/history list — this is short-term memory, exactly as built earlier in this module.
- **When a session ends**, that message history is processed through the same extract → tag → separate → deduplicate pipeline described above, and the results are written into long-term storage.
- **When a new session starts**, the cleaned semantic and procedural memory is "cold-loaded" directly into the system instruction before the first user message is even processed — so the assistant opens the conversation already knowing the user's stable facts and preferences, without a single retrieval call.

This is why a chat assistant with memory enabled can reference something from days or weeks earlier without the user restating it — the mechanism is the same extract/tag/store/inject pipeline built manually throughout this module, just running automatically around every session boundary.

25.4 No Universal Rule for Memory Design

There is no single fixed formula for how every AI application should structure memory. Whether duplicates should be removed or preserved, which memory types matter for a given product, how aggressively to deduplicate, what threshold to use for episodic retrieval — all of this depends on the specific AI application being built. Memory architecture should be designed around the application's actual needs, not applied as a rigid template.

25.5 Golden Rules for Memory Management

- **LLMs have no memory of their own.** Every "memory" behavior in an AI application is engineered externally — the model itself is stateless and only ever sees whatever text is placed in its context window on a given call.
- **Match the retrieval strategy to the memory's shape.** Information that stays small, stable, and broadly relevant (facts, learned behavioral rules) can be injected in full into the system instruction; information that's large, ever-growing, or only relevant to specific queries (events, or a semantic/procedural store that's grown large or task-specific) should be filtered through similarity search and injected only when relevant, into the user prompt.
- **Never dump everything into the context window "just in case."** Indiscriminate injection pollutes and bloats the context, degrades response quality, and wastes tokens and latency — selective, purposeful injection is what keeps a context window healthy.

- **Stale facts must be actively resolved, not silently accumulated.** When a fact changes, decide deliberately whether the old version should be overwritten or preserved as history — letting contradictory facts pile up forces the model to guess which one is current.
- **A library is a foundation, not a complete solution.** Memory libraries solve storage and retrieval mechanics well, but not correctness over time (deduplication, forgetting, ensuring critical facts aren't missed by pure similarity search) — production-grade memory systems pair a library with custom engineering around it.
- **Design memory around the application, not the other way around.** There is no one-size-fits-all memory architecture; decisions like whether to deduplicate, which memory types to track, and how aggressively to retrieve should follow from what the specific application actually needs.

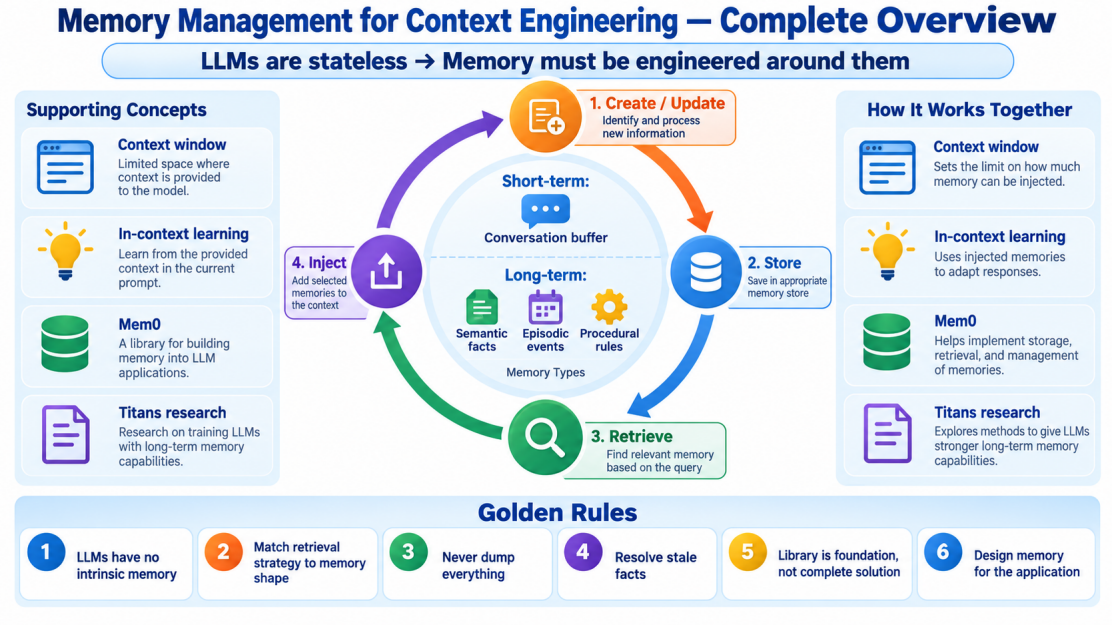

Figure 54. Complete overview of memory management for context engineering, from short-term state and long-term memory creation to retrieval, injection, and library-assisted operation.

COURSE MODULE

Context Compression

26 Context Compression and Lossless Compression

26.1 Why Context Compression Exists

Everything covered so far in context engineering — RAG, tools, memory, MCP — was about putting a *healthy* context into the context window. But even with all these good practices followed, a subtler problem remains: over time, the context window keeps filling up.

In a multi-turn conversation, every prompt, response, tool call, tool result, and retrieval gets appended to the context. None of these individual additions are "unhealthy," but their accumulation is unavoidable. At the same time, every LLM has a **hard token limit**. This creates a structural dilemma: context keeps growing, but the window has a ceiling.

Context compression is the discipline of managing this growth — making the text in the context window smaller without losing the information that actually matters.

> 🧠 **Analogy — The 3-Hour Movie**
> Context compression is like watching a 3-hour movie and explaining it to a friend in 30 minutes — covering the storyline, key scenes, dialogues, and plot, just without the padding. You're re-expressing the same essential content using far fewer "tokens."

26.2 The Four Reasons to Compress Context

Driver	Why It Matters
Cost	Every token sent to the LLM (system instructions, memory, history) is billed. A larger context window means higher cost per call.
Latency	Larger inputs take longer for the LLM to process. Compressed input means faster responses.
Context rot	An overfilled window causes the "lost in the middle" phenomenon and general context bloat, degrading output quality.
Hard wall	Every LLM has a maximum context limit. Uncontrolled growth eventually hits this wall, causing errors or silent loss of important information.

Context compression isn't just about "better-looking output" — it simultaneously reduces cost, improves speed, and improves agent quality by avoiding context rot. Claude Code is cited as a strong real-world example of an agent that combines good memory management with continuous context compression to avoid these problems.

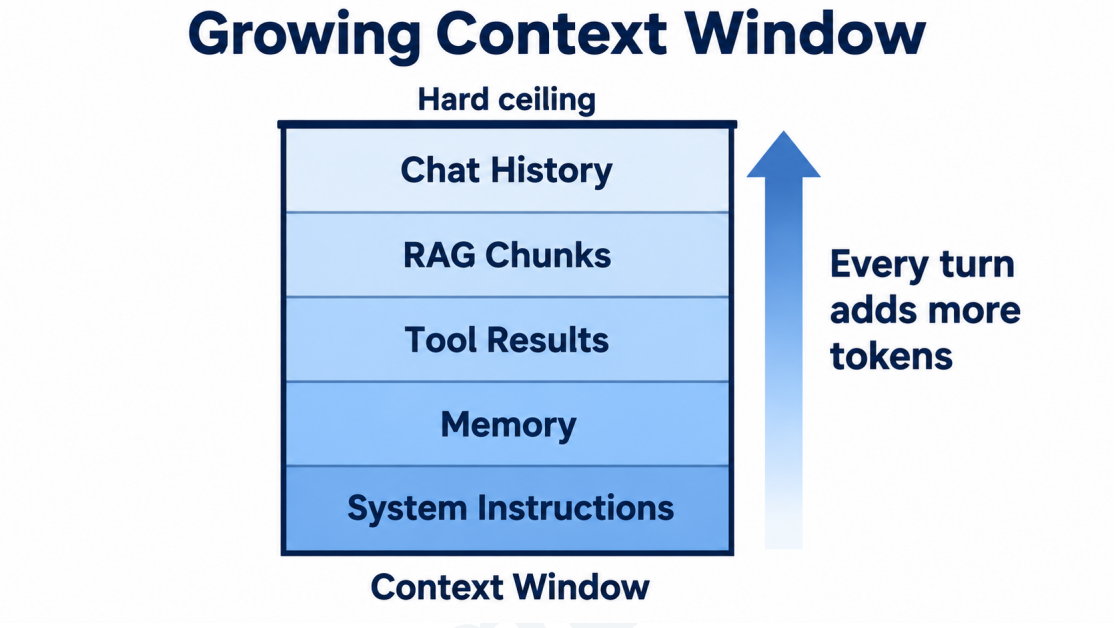

Figure 55. A growing multi-turn context accumulates prompts, responses, retrievals, and tool results until cost, latency, context rot, and hard limits become unavoidable.

26.3 Two Approaches to Compression

There are two broad ways to compress context:

1. **Lossless compression** — the subject of this lecture.
2. **Lossy compression** — covered in a later lecture.

Lossless Compression: Definition

Lossless compression reduces the token count **without losing any information**. The key distinction is between *information* and *meaning*:

- **Information** = the actual hard facts, preserved exactly.
- **Meaning** = the gist — e.g., summarizing a 10-line paragraph into a 4-line version that captures the same idea but drops the literal content.

Lossless compression preserves *information*, not just meaning. It works by cleaning up **verbose representations** of the same data — nothing is thrown away, only the wasteful formatting around it.

26.4 Catalog of Lossless Techniques

Technique	What It Does
JSON → CSV	Converts tool output from verbose JSON into CSV, preserving every field while cutting structural tokens (brackets, repeated keys, quotes).
Deduplication	Removes duplicate rows/records. The information already exists once; repeated copies add no new value.
JSON minification	Converts pretty-printed (indented, multi-line) JSON into a single compact line.
XML tag dropping	Strips XML tags while keeping the values inside them.
Decorative character removal	Removes ASCII borders, separators, dashes — common in RAG sources built from documents/databases.
Whitespace management	Strips extra spaces, blank lines, trailing whitespace, deep indentation.
ANSI/timestamp/HTML stripping	Removes ANSI escape/color codes, repeated timestamps, and HTML wrapper tags that carry no semantic value.

A live token-count comparison in the lecture showed a JSON object going from 49 tokens (pretty-printed) to 33 tokens (minified) with identical information — and an XML-tagged snippet dropping from 42 tokens to 23 tokens after tag removal.

26.5 Where Verbose Elements Come From

Almost all of this "verbose noise" enters the context window through only two channels: **tool calls** and **RAG**.

- **System instructions** and **memory** are written/curated carefully, so they rarely introduce noise.
- **Message history** may technically contain messy content, but it is intentionally left untouched.

> ⚠ **Mentor's Rule — Never Clean the User's Prompt**
> Even if a user's prompt looks "dirty" or unstructured, don't clean it. You never know the intent behind how a user phrased or formatted their input — cleaning it risks silently altering meaning. Compression targets tool and RAG output, not the user's own words.

CampusX

Two Sources of Tools/RAG: Manual vs. MCP

- **Manually built tools/RAG:** You control the design, so you can build them to output information in the most token-efficient form from the start (e.g., designing tool outputs to minimize tokens, cleaning documents before embedding them for RAG).
- **MCP-sourced tools/RAG:** Tools, resources, and prompts come from a third-party MCP server. You don't control their output format, so verbosity (XML wrappers, excess whitespace, decorative characters) can leak in unpredictably.

Workflow for MCP-sourced context:

1. Connect to the MCP server.
2. Use its tools/resources/prompts to build your RAG pipeline, and test it.
3. During testing, identify concrete verbosity problems (e.g., a tool returning HTML-scraped text, excessive XML, inconsistent spacing).
4. Write a cleaning function targeting exactly those identified patterns.
5. Insert that function between the MCP output and the point where it's appended to the LLM's context window.

26.6 Implementation: `clean_context`

The cleaning function uses the `re` library to strip specific noise patterns identified during profiling — without touching the actual data values.

```python
import re

def clean_context(text: str) -> str:
    """
    Sequential pipeline for lossless compression on retrieved context.
    Strips structural noise while preserving the underlying facts.
    """
    # 1. Strip ANSI escape codes (terminal colors/formatting)
    text = re.sub(r'\x1B(?:[@-Z\\-_]|\[[0-?]*[ -/]*[@-~])', '', text)

    # 2. Strip HTML/XML tags, keep the inner content
    text = re.sub(r'<[^>]+>', '', text)

    # 3. Remove timestamps (e.g. 2024-01-01T12:00:00.000Z)
    text = re.sub(r'\d{4}-\d{2}-\d{2}[ T]\d{2}:\d{2}:\d{2}(?:\.\d+)?Z?', '',
    ↪ text)

    # 4. Remove decorative/border characters (===, ---, ###, etc.)
    text = re.sub(r'[#\*=\-_]{3,}', '', text)
```

CampusX

```
    # 5. Collapse multiple spaces/blank lines into one
    text = re.sub(r'\s+', ' ', text)

    return text.strip()
```

Integrating it into the agentic loop is a one-line addition at the point where retrieved data is about to be appended to context:

- **RAG loop:** after `search_knowledge()` retrieves each chunk, pass it through `clean_context()` before wrapping it in delimiter tags and adding it to context.
- **Tool loop:** after a tool call returns its raw output, pass that output through `clean_context()` before returning it to the agent.

Nothing else in the loop changes — this is a pure insertion step.

Figure 56. A lossless cleaning pipeline removes ANSI codes, markup, timestamps, decorative characters, and excess whitespace before context reaches the model.

26.7 Measured Impact

Testing the function on deliberately "dirty" samples gave the following character/token savings:

Noise Type	Savings
Pretty-printed JSON	~90%
XML tags	~71%
Decorative borders	~67%
ANSI escape codes	~61%
HTML wrappers	~43%
Whitespace/indentation	~27%

Running the same set of test queries through a "dirty" agent (no cleaning) versus a "clean" agent (with `clean_context` applied) showed per-query savings ranging roughly from 22% to 58%, averaging around **33% overall reduction in context window usage** — with zero information loss.

> 💡 **Insight — Lossless Is the First Line of Defense**
> Lossless compression is rarely used on its own — it's almost always paired with lossy compression. Think of it as the first line of defense: strip out structural noise (formatting, tags, whitespace) *before* applying any lossy technique like summarization. This way, the lossy step spends its "budget" compressing actual meaning rather than wasting it on junk syntax.

27 Lossy Compression

What Is Lossy Compression

Where lossless compression reduces tokens while preserving every hard fact, **lossy compression** deliberately allows some information to be lost. The trade-off is intentional: as tokens are reduced, specific details disappear, but the **meaning, context, and essence** of that information must survive intact.

In short: *lossy compression = deliberate information loss, with meaning preserved.*

Deletion vs. Summarization

Lossy compression is a theme with a couple of concrete techniques:

- **Deletion** — simply dropping parts of the context window over time. This is **task-specific**: useful only in certain workflows (e.g., discarding a completed sub-task's data) and not broadly applicable.
- **Summarization** — condensing information while keeping its meaning. This is the technique actually used almost everywhere, and is effectively synonymous with lossy compression in practice.

> 🧠 **Analogy — The 300-Page Book**
> Summarizing is like condensing a 300-page book into a 3-page summary. The literal text (300 pages of detail) is gone, but the plot, characters, and key events all remain recognizable. That's lossy compression: information is lost, meaning is not.

CampusX

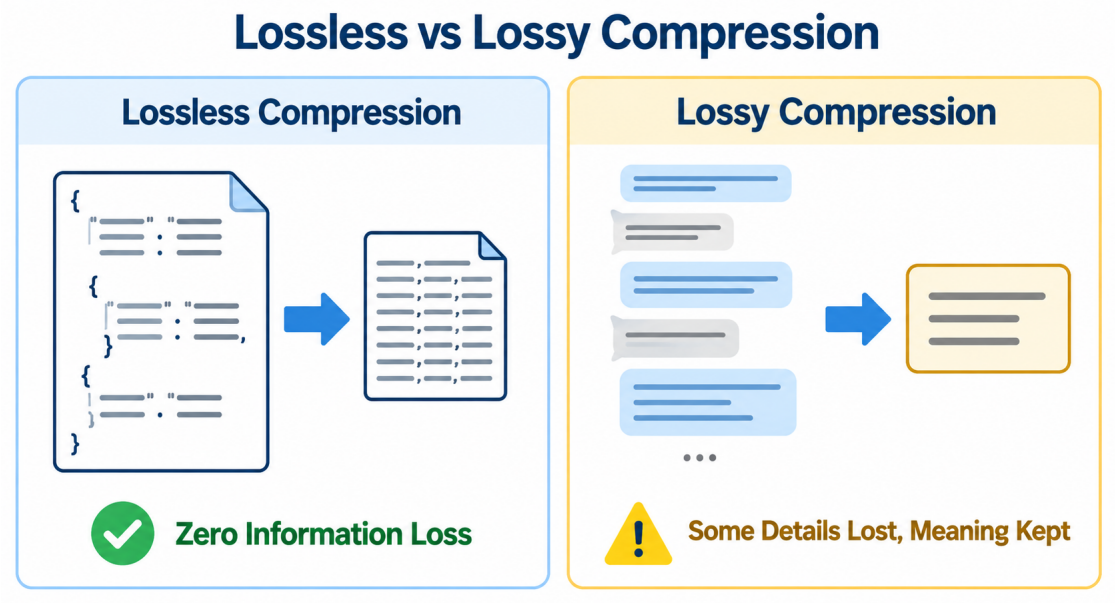

Figure 57. Lossless compression removes structural waste while preserving every fact; lossy compression deliberately removes detail while preserving essential meaning.

28 Creating Summary and Memories Together

Building automatic summarization into a chatbot or agent follows three phases:

Phase 1 — Set a Trigger

Decide when summarization should kick in. Common trigger types:

- **Token-based**: fire once the context exceeds a threshold (e.g., 1,200 tokens for a small model, 30k+ for larger windows).
- **Turn-based**: fire every N multi-turn exchanges (e.g., every 10 turns).

Token-based triggers are the more common choice.

Phase 2 — Segment the Context

Never summarize the *entire* context window. Split it into three parts:

Segment	Treatment
System instructions / tool definitions	Always excluded — kept static, never summarized.

Segment	Treatment
Middle history (everything except the above and the tail)	This is what actually gets summarized.
Last N messages (commonly 1–4)	Always excluded — kept verbatim to preserve short-term coherence.

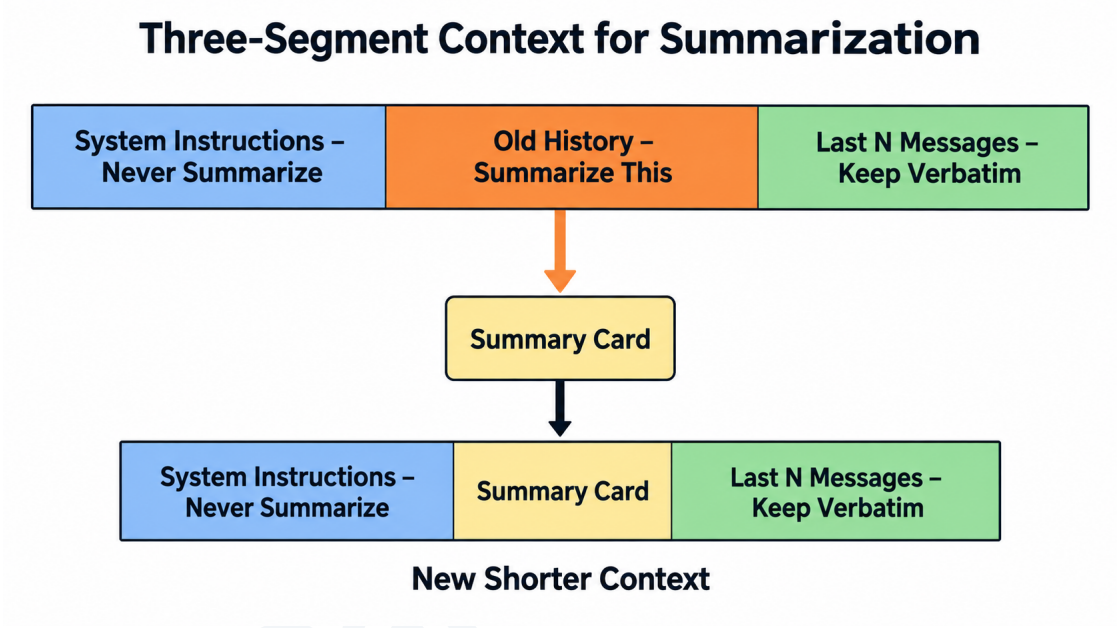

Figure 58. Safe summarization divides context into protected system instructions, compressible middle history, and recent messages kept verbatim.

Phase 3 — Summarize and Reconstruct

The excluded "middle" messages are joined into one block of text and sent to an LLM (often a smaller, cheaper model, since this is an administrative task, not the main reasoning task) with a dedicated summarization system prompt.

The new context window is then rebuilt as:

```
new_context = system_instructions + summary + last_N_messages
```

The summary is typically inserted as a single message (e.g., role `user`, content prefixed with something like "conversation so far: …") so the rest of the pipeline treats it like any other turn.

```python
def compress_with_summary(messages, trigger_tokens, token_count_fn,
 ↪  summarizer_llm):
    system_msg = messages[0]                  # always excluded
    recent = messages[-1:]                    # last N messages, always
 ↪  excluded
    old = messages[1:-1]                      # the middle - candidate for
 ↪  summarization

    if token_count_fn(messages) <= trigger_tokens:
        return messages                        # trigger not hit yet, no
         ↪  change

    old_text = "\n".join(f"{m['role']}: {m['content']}" for m in old)

    summary = summarizer_llm.generate(
        system_prompt=SUMMARY_SYSTEM_PROMPT,    # instructs what to keep, see
 ↪  below
        user_input=old_text,
    )

    summary_msg = {"role": "user", "content": f"Conversation so far:
 ↪  {summary}"}

    return [system_msg, summary_msg] + recent
```

28.1 What a Good Summary Must Capture

A generic summary isn't enough for task-oriented agents — losing the wrong detail breaks the user experience. Four things must always be preserved:

1. **Decisions made so far** — e.g., a hotel already booked and paid for. Omitting this risks the agent re-doing or contradicting completed work.
2. **Established facts / shared information** — preferences, constraints, and details the user has already given (e.g., dietary restrictions, budget, destinations of interest). Omitting this forces the agent to re-ask the user, which is a frustrating experience.
3. **Unsolved / pending questions** — what's still outstanding (e.g., flights and local transport not yet booked). Without this, the agent may wrongly assume the task is complete.
4. **Main goal and current progress** — the overarching objective and how far along it is.

> 💡 **Insight — Summaries Are State, Not Prose**
> A good agent summary isn't a narrative recap — it's a compact state record: decisions locked in, facts on file, open questions, and overall progress. Treat it like a save-file for the conversation, not a highlight reel.

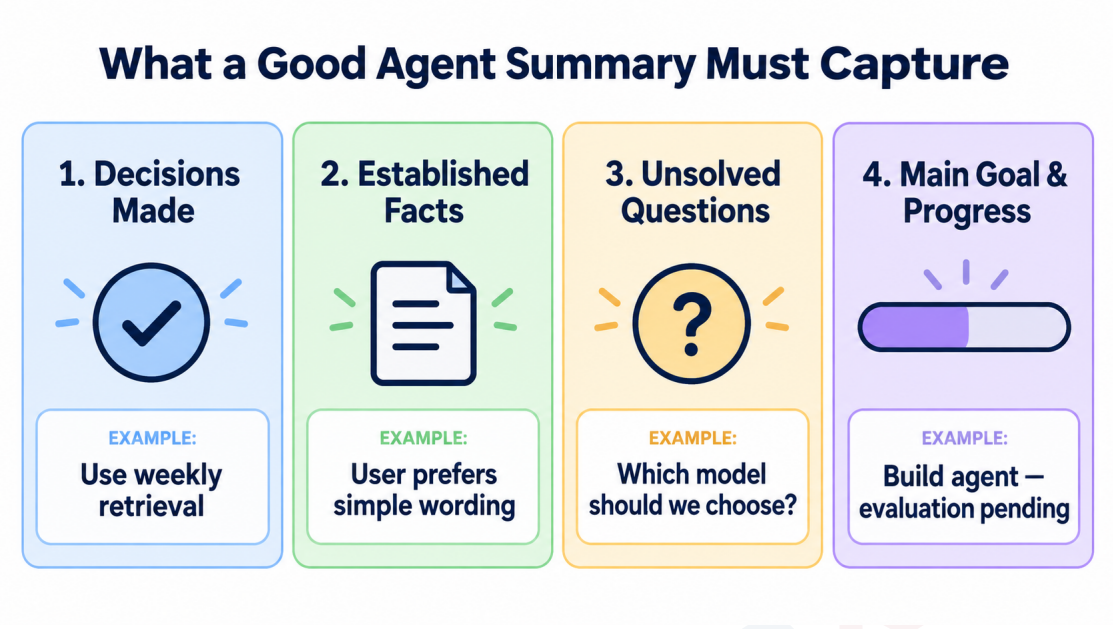

Figure 59. A useful agent summary preserves four pillars: decisions, established facts, unresolved questions, and the main goal with current progress.

28.2 Practical Result: Chatbot Summarization Experiment

A simple travel-planning chatbot ("Tara") was run with a 1,200-token trigger. Once the trigger was hit, the previous exchanges were replaced by a summary:

Stage	Tokens
Just before trigger fires	~1,272
Immediately after summarization	~736

Despite the ~42% token reduction, the agent retained the user's exact preferences (vegetarian diet, ₹2 lakh budget, interest in temples/nature over nightlife) across the compressed turn — confirming that meaning was preserved even though the literal message history was gone. The same trigger-and-summarize loop then continues automatically on every subsequent cycle: tokens accumulate, the trigger fires again, and the middle section gets re-summarized.

The identical principle applies inside AI agents that use tools and RAG (already covered in the tools module) — the mechanics of separating system/old/recent messages and summarizing the "old" portion are the same.

28.3 Lossless and Lossy Are Partners, Not Alternatives

Lossless and lossy compression are not competing choices — they're applied together, in sequence:

1. **Lossless compression runs first** (the first line of defense): strip verbose formatting, XML, whitespace, decorative characters — with zero information loss.
2. **Lossy compression runs second**: once the lossless-cleaned context still exceeds what's needed, summarization trims it further by deliberately discarding detail while preserving meaning.

This two-layer approach is what allows an agent to sustain long-running, multi-turn sessions without hitting the hard token wall or degrading into "lost in the middle" behavior.

28.4 Combining Context Compression with Memory Creation

Context compression and memory creation both start from the same raw conversation, but they are **two separate outputs**, not one derived from the other.

Rule: Never Build Memory From Compressed Context

Memory must always be built from the **raw conversation** — never from a summary. A summary has already discarded detail in order to save tokens, so building memory on top of it means building memory on top of information that's already partially lost. Memory needs the full, unsummarized exchange to extract accurate semantic, episodic, and procedural facts.

This means that at the moment of compression, the same raw "old" messages get used twice, for two different purposes:

- **Summarization** → produces a compact stand-in for the context window (covered in Part 2).
- **Memory creation** → produces persistent semantic/episodic/procedural memories (covered in the earlier memory module, using `mem0`).

28.5 The Unified Trigger Flow

The flow doesn't change from what was built for lossy compression — one branch is simply added:

1. Trigger fires (e.g., token count exceeds a threshold).
2. Split messages into **system**, **old** (unsummarized middle), and **recent** (last N), exactly as in Part 2.
3. Run **two functions** on the old segment:
   - `create_summary(old_messages)` — summarizes the old segment for the compressed context window.
   - `create_memory(raw_old_messages)` — extracts durable memories from the same old segment.
4. Rebuild the active context as `system + summary + recent_messages`.
5. Separately, store the extracted memories against the user's ID for future sessions.

CampusX

A crucial detail: `create_memory` must only ever receive messages that **haven't already been folded into a previous summary**. If a summary message were passed into memory creation, the memory system would be extracting facts from compressed (already lossy) text — which breaks the "memory only from raw conversation" rule. So the pipeline tracks which old messages are still raw and only feeds those to `create_memory`.

```python
def on_trigger(messages, summarizer_llm, memory_client, user_id):
    system_msg = messages[0]
    recent = messages[-1:]                    # protected, same as Part 2
    old = messages[1:-1]                      # candidate for summarization
    raw_old = [m for m in old if not m.get("is_summary")]   # never-summarized
    only

    old_text = "\n".join(f"{m['role']}: {m['content']}" for m in old)
    summary = create_summary(old_text, summarizer_llm)      # -> compact
    context stand-in

    create_memory(raw_old, memory_client, user_id)          # -> persistent
    semantic/episodic/procedural facts

    summary_msg = {"role": "user", "content": f"Conversation so far:
    {summary}", "is_summary": True}
    return [system_msg, summary_msg] + recent
```

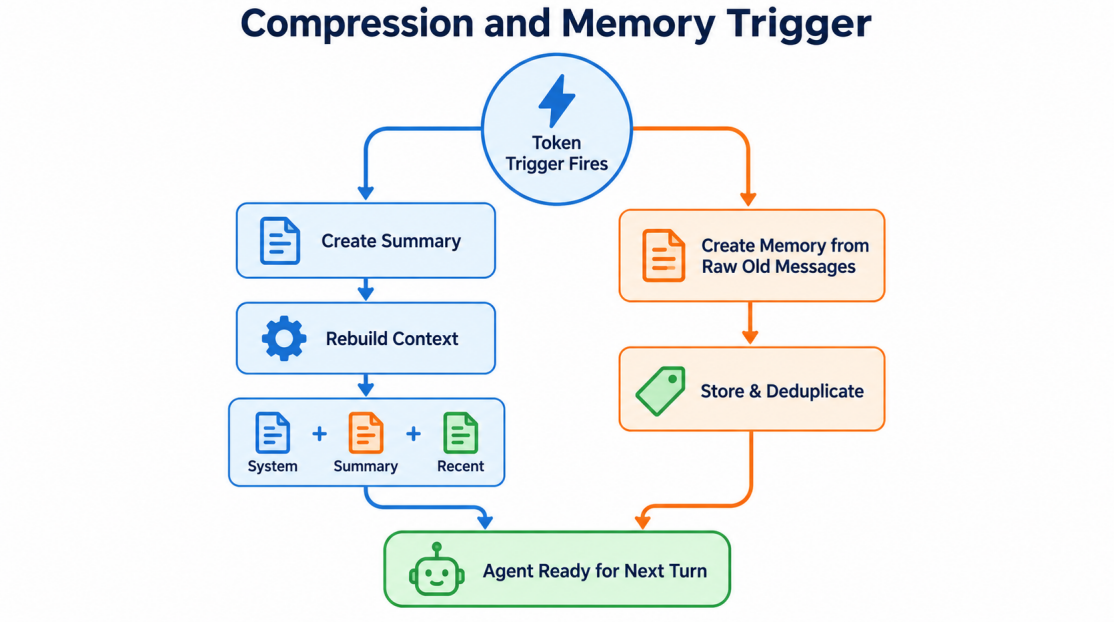

Figure 60. At the compression trigger, raw older messages branch into both a compact conversation summary and durable memory creation before the active context is rebuilt.

Practical Run: Compression + Memory Together

Running the same "Tara" travel chatbot with both branches active, against a 1,200-token trigger:

Prompt	Tokens
1	283
2	661
3	939
4	1,270 → **trigger fires** (summary + memory created)
5	779
6	1,104
7	1,430 → **trigger fires again**
8	795

Each time the trigger fires, both the summary and the memory are created from the same raw "old" segment before the context window is rebuilt.

28.6 Cleaning the Generated Memory

Memory produced directly by `mem0` (or any memory library) is **not clean by default** — it tends to contain duplicate or near-duplicate entries (e.g., "user prefers temple gardens and nightlife" saved multiple times in slightly different wording).

The cleaning process, consistent with the earlier memory module, has two steps:

1. **Separate by type** — split memories into semantic, episodic, and procedural buckets.
2. **Deduplicate** — use an LLM to identify and collapse near-duplicate entries within each bucket.

```python
def clean_memory(raw_memories, dedup_llm):
    semantic = [m for m in raw_memories if m["type"] == "semantic"]
    procedural = [m for m in raw_memories if m["type"] == "procedural"]

    clean_semantic = deduplicate(semantic, dedup_llm)
    clean_procedural = deduplicate(procedural, dedup_llm)

    return clean_semantic, clean_procedural
```

In the practical run, 5 raw semantic memories (several of them near-duplicates of "prefers temple gardens and nightlife" or "is vegetarian") were reduced to 2 clean semantic memories after deduplication; no procedural memories existed in this particular run, so that bucket came back empty.

CampusX

Feeding Cleaned Memory Back Into the Agent

Once memory is clean, it's injected into the agent's **system instruction / persona** for future turns — e.g., appending semantic and procedural rules to Tara's persona so the agent already "knows" the user is vegetarian and prefers temples and gardens without needing to be told again. Asking the agent afterward "what do you know about me?" correctly surfaces these facts, confirming the memory is actually being used, not just stored.

28.7 Golden Rules — Context Compression & Memory

- **Lossless first, lossy second.** Always strip structural noise (formatting, tags, whitespace) before summarizing. Lossless compression is the first line of defense; lossy summarization is the second — they're partners, not alternatives.
- **Protect the system instructions and the last N messages.** These should never be summarized away — static config and the most recent turns are what keep an agent coherent and consistent.
- **A summary is a state record, not a recap.** A useful conversation summary must preserve decisions made, established facts, unsolved questions, and overall goal/progress — not just a narrative gist.
- **Memory must come from raw conversation, never from compressed context.** Building memory on top of a summary compounds information loss; always extract memory from the unsummarized exchange.
- **Generated memory is not clean by default.** Always separate by type and deduplicate before using memory to shape future behavior — raw memory output tends to accumulate near-duplicate entries over time.

CampusX

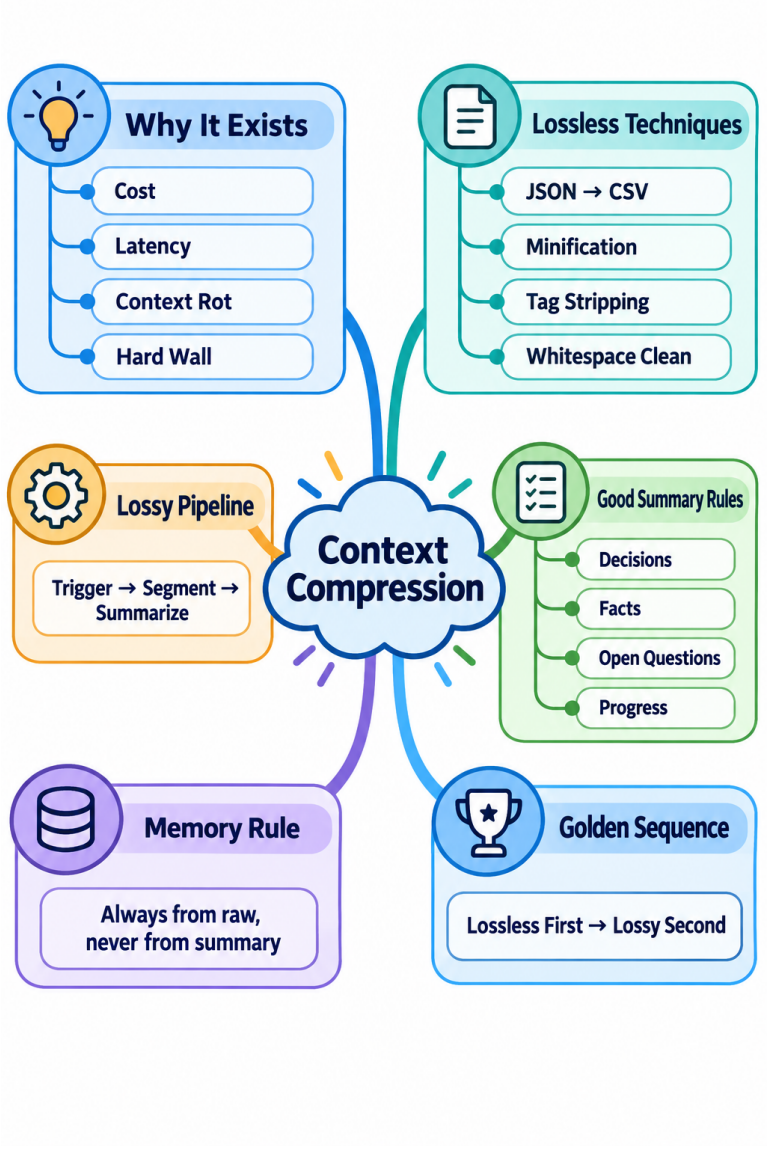

Figure 61. Complete overview of lossless cleaning, lossy summarization, protected context segments, and memory creation from raw conversation.